# Algorithms for NP-hard Problems

Eunjung KIM

June 1, 2026

# Contents

# Chapter 1

# Introduction and Parameterized algorithm

**Coping with intractability.** Most computational problems are NP-hard. Some[1] even say that "there are only six polynomial-time solvable problems, essentially. All other problems are NP-hard." Then what do we do with these intractable problems? There are a range of algorithmic paradigms designed to cope with the prevalent intractability of computational problems, each emphasizing different aspects of an efficient algorithm. In this lecture, we focus on the paradigms of parameterized algorithms, (fast) exact algorithms and approximation algorithms.

**Computational Problem and formalism.** Let us clarify what we mean by computational problems when we want to treat them in a mathematically rigorous manner. It depends on the context. When you talk about a decision problem, a computational problem is considered as a language over some alphabet $\Sigma$, i.e. a subset $L \subseteq \Sigma^*$ of all (finite-length, possibly zero) strings over $\Sigma$. Here, the alphabet $\Sigma$ is a finite set of symbols (e.g. consisting of 0 and 1) that we use to encode[2] an input of the problem as a string. In the context of a decision problem, the language $L$ is viewed as the set of (the encodings of) YES-instances to a problem at hand. In fact, the set of YES-instances as a language is THE VERY DEFINITION of a decision problem. For example, VERTEX COVER is a problem which takes as an input $G$ and a non-negative integer $k$, and the answer is YES if $G$ has a vertex cover of size at most $k$ and is NO otherwise. This is a human way of describing the problem VERTEX COVER. On the other hand, a machine (formal) way of describing VERTEX COVER is to give the set $\{(G, k) : k$ is a non-negative integer and $G$ is a graph having a vertex cover of size at most $k\}$, with a suitable encoding using some alphabet $\Sigma$. With the former description, we are looking for an algorithm to decide whether the input instance is YES-instance or not. With the latter description, we are looking for an algorithm to test a membership of a string $x \in \Sigma^*$ in a language $L$, i.e. $x \in L$ or not. We usually use the human description, but it is useful to be aware of the formal description.

A Turing machine is nothing but an algorithm which executes a membership of an input string $x \in \sigma^*$ for a language $L$, where the algorithm is described not in a human way but in a machine way[3]. When you look into what happens in a Turing machine testing if $x \in L$, there is something called a witness. A witness

[1] I heard this assertion a few times, but could not trace down the source. But let me unashamedly posit it here for a dramatic effect :)

[2] It is presumed that there is a predetermined rule of how to encode an input to the problem at hand.

[3] To learn how to rewrite an algorithm in a machine-like way is the subject of automata theory. Notice that this is a must in order to implement our 'algorithmic idea' at the hardware level, and that is why Turing machine is called the mathematical model of a computer.

$w \in \Sigma^*$ is an additional sequence of symbols, on top of the input $x \in \Sigma^*$, which describes an 'path' for the input string $x$ from the initial state to an accepting state. A vertex cover of size at most $k$ of a graph $G$ (given as an input string) can be seen as such a witness as it witnesses that $(G, k)$ is a YES-instance; such a vertex cover can be converted into a string which assists a Turing machine which solves the problem VERTEX COVER to reach an accepting state. Therefore one can rewrite a decision problem $L \subseteq \Sigma^*$ in the form $\{x \in \Sigma^* : \text{exists } y \in \Sigma^* \text{ such that } R(x, y) = 1\}$. Here, $R(x, y) = 1$ is the indicator function of a relation $R \subseteq \Sigma^* \times \Sigma^*$, i.e. $R(x, y) = 1$ if and only if $(x, y) \in R$.

An optimization problem is formally described by a set of relations $R \subseteq \Sigma^* \times \Sigma^*$, a cost function[4] $c : R \to \mathbb{N}_0$, and $\mathsf{goal} \in \{\min, \max\}$. For example, an optimization version of the problem VERTEX COVER can be described with $R$ as the set of all $(x, y)$ where $x$ is (the encoding of) a graph $G$, and $y$ is (the encoding of) a vertex cover of $G$, the cost function $c$ on $(x, y)$ equals the size of the vertex cover $y$, and $\mathsf{goal} = \min$.

There are other forms of computational problems (search problem, promise problem, function problem, counting problem, etc) and often one can be cast in the form of the other. For example, a decision problem can be seen as a special form of an optimization problem while an optimization problem can be seen as a decision problem as well. Usually which formalism of computational problem shall be chosen depends on the algorithmic framework as well as the core difficulty of the problem. That is, for parameterized problem we mostly presume decision problems and for fast exact algorithms and approximation algorithms, we presume an optimization problem. For PRIMALITY TESTING, in which we decide whether a given number $N$ is a prime or not, it hardly makes sense to consider it as an optimization problem.

**Parameterized problems.** Consider the problem VERTEX COVER, i.e. as a language

$$L := \{(G, k) : k \text{ is a non-negative integer and } G \text{ is a graph having a vertex cover of size at most } k\}.$$

One can parameterized VERTEX COVER by the solution size.

Formally, a parameterization of $\Sigma^*$ is a mapping $\kappa : \Sigma^* \to \mathbb{N}_0$. A parameterized problem is a pair $(L, \kappa)$, where $L \subseteq \Sigma^*$ and $\kappa$ is a parameterization of $\Sigma^*$. We say that $(L, \kappa)$ is the problem $L$ parameterized by $\kappa$. For instance, consider the parameterization of VERTEX COVER (as a decision problem) by the solution size (budget). In this case, the parameterization $\kappa$ at hand shall map an instance $(G, k)$ of VERTEX COVER to $k$ (a string $x \in \Sigma^*$ which is not well-formed will be mapped to an arbitrary non-negative integer, say 0). The parameterization $\kappa$ does not necessarily map an instance $x \in L$ to an integer already explicit in the input $x$. For example, we may parameterize VERTEX COVER by $k - \mathtt{lp}^*(G)$, that is, an instance $(G, k)$ of VERTEX COVER is mapped to $k - \mathtt{lp}^*(G)$ by $\kappa$. Such a mapping $\kappa$ is well-defined for a given instance $(G, k)$.

**Definition 1.0.1.** *A parameterized problem $(L, \kappa)$ is fixed-parameter tractable if there is an algorithm which, for every $x \in \Sigma^*$, decides if $x \in L$ in time $f(\kappa(x)) \cdot |x|^{O(1)}$.*

There are many different ways of parameterizing a decision problem, and some might make more sense than others. One may even consider a parameterization by multiple parameters.

A remark: instead of defining a parameterized problem as a pair of a decision problem (a set of strings) and a mapping $\kappa$, one can define a parameterized problem as a decision problem itself, i.e. a set of strings. In this perspective, we say that $P \subseteq \Sigma^* \times \mathbb{N}_0$ is a parameterized problem, and an instance of a parameterized problem is of the form $(x, k)$. Note that this is a string; one can encode the second component $k$ with a

[4]For simplicity, we assume that the cost is a non-negative integer, but it can be over the reals, etc

special unary alphabet $\mathbf{1}$ and $(x, k)$ is a string over $\Sigma \cup \{\mathbf{1}\}$. Then the parameterized problem $(L, \kappa)$ defined above can be rewritten as a subset of $\Sigma^* \times \mathbb{N}_0$ by taking

$$\{(x, \kappa(x)) : x \in L\}.$$

Often, the parameterized problem is defined as a decision problem in this way, and an instance of a parameterized problem is written as $(x, k) \in \Sigma^* \times \mathbb{N}_0$. In this case, the second component of the instance $(x, k)$ is called the parameter. According to this version, an instance to VERTEX COVER parameterized by the solution size is of the form $((G, k), k)$.

# Chapter 2

# Branching Algorithms

## 2.1 $\mathcal{O}^*(2^k)$-time algorithm for VERTEX COVER

The procedure **VC** in Algorithm 9 takes as an input instance a graph $G$ and a non-negative integer $k$, and outputs YES or NO correctly.

**Algorithm 1** Algorithm for VERTEX COVER

```
1: procedure VC(G, k)
2:     if G has no edge then return YES.
3:     else if k = 0 then return NO.
4:     else                                          ▷ G has an edge and k > 0
5:         Pick an edge uv.
6:         return VC(G − u, k − 1) or VC(G − v, k − 1).
```

Below, we prove the correctness and the running time of the algorithm **VC**[1].

**Lemma 2.1.1.** *Given an input instance $(G, k)$ of* VERTEX COVER*, the procedure* **VC** *correctly decides whether an input instance $(G, k)$ is* YES *or* NO *in time $\mathcal{O}(2^k \cdot poly(n))$.*

**Proof:** First, let us show the correctness of **VC**; that is, **VC**(G,k) returns YES if and only if $(G, k)$ is a YES-instance. We prove this by induction on $k$. Notice that when $k = 0$, the procedure **VC** always returns some output (and do not branch). Therefore, in any recursive call during the execution of the procedure, the parameter $k$ remains non-negative. If $k = 0$ or $|E| = \emptyset$, then it is trivial to verify that Lines 4 and 3 respectively returns a correct output. Therefore, we consider the case when $|E| > 0$ and $k > 0$.

If $(G, k)$ is a YES-instance, that is $G$ has a vertex cover $X$ of size at most $k$, then at least one of $u$ and $v$ in Line 5 is included in $X$. Observe that at least one of the instances $(G - u, k - 1)$ and $(G - v, k - 1)$ is a YES-instance because $X \setminus u$ (respectively $X \setminus v$) is a vertex cover of $G - u$ (respectively $G - v$). By induction hypothesis, **VC** returns a correct output upon each of $(G - u, k - 1)$ and $(G - v, k - 1)$. This means that, by the fact that one of them is a YES-instance, the returned output [2] at Line 6 will be YES and

[1] A detailed proof is given for this algorithm to illustrate what a correctness proof in full should be like. For other algorithms that we'll encounter later, not so much details might be delineated.

[2] Here, we construe 'or' as the boolean operation OR by equating YES to True and NO to False.

thus the output of $\mathbf{VC}(G,k)$ is correct.

Now suppose that $(G,k)$ is a NO-instance. Then the instance $(G-u,k-1)$ (likewise $(G-v,k-1)$) is a NO-instance. Indeed, if $G-u$ has a vertex cover $X'$ of size at most $k-1$ then $X' \cup \{u\}$ is also a vertex cover of $G$ and we have $|X' \cup \{u\}| \leq k$, a contradiction. Therefore, by induction hypothesis **VC** outputs NO to both instances called in Line 6. Therefore, Line 6 returns NO. This proves the correctness of the algorithm.

To see the running time, observe that the search tree depicting the relations of the recursive calls has depth at most $k$. This is because each instance created during the recursive calls have $k \geq 0$ and every branching decreases the parameter value by 1. Therefore, there are at most $2^k$ leaves and $\mathcal{O}(2^k)$ nodes in the search tree. As the algorithms spends $poly(n)$ steps at each node, the running time follows. □

## 2.2 $\mathcal{O}^*\big((3k)^k\big)$-time algorithm for FEEDBACK VERTEX SET

**Lemma 2.2.1.** *For any feedback vertex set $X$ of $G=(V,E)$, it holds that*

$$\sum_{v\in X}(d(v)-1) \geq |E|-|V|+1.$$

**Proof:** All the edges of $G$ are either counted as part of the forest $G-X$, or incident with $X$. Now,

$$\begin{aligned}|E| &= \#\text{ edges with both endpoints in } V-X + \#\text{ edges incident with } X\\ &\leq (|V-X|-1)+\sum_{v\in X} deg(v).\end{aligned}$$

Here, we use the fact that the number of edges in an $n$-vertex forest is at most $n-1$. Note also that $\sum_{v\in X} deg(v)$ (over)counts the number of edges incident with $X$. This yields the claimed inequality. □

**Lemma 2.2.2.** *Let $G=(V,E)$ be a graph of degree at least three. Let $\{v_1,v_2,\ldots,v_n\}$ be the vertex set of $G$ such that*

$$deg(v_1) \geq deg(v_2) \geq \cdots \geq deg(v_n).$$

*Then for any feedback vertex set $X$ of size at most $k$, we have $X \cap \{v_i : i \leq 3k\} \neq \emptyset$.*

**Proof:** Suppose not. Observe

$$\sum_{i\leq 3k}(deg(v_i)-1) \geq 3\sum_{v\in X}(deg(v)-1) \geq 3\big(|E|-|V|+1\big),$$

because each vertex of $X$ has degree at most $deg(v_{3k})$. Also, due to $X \subseteq \{v_{3k+1},\ldots,n\}$

$$\sum_{i>3k}(deg(v_i)-1) \geq \sum_{v\in X}(deg(v)-1) \geq |E|-|V|+1.$$

Summing the two equalities, we get

$$\sum_{i=1}^{n}(deg(v_i)-1) = 2|E|-|V| \geq 4\big(|E|-|V|+1\big),$$

and thus

$$3|V| > 2|E| = \sum_{v\in V} deg(v).$$

This contradicts the assumption that each $v$ has degree at least three. □

## 2.3 $\mathcal{O}^*(4^{k-\mathtt{lp}^*(G)})$-time algorithm for VERTEX COVER

Here, we consider the problem VERTEX COVER parameterized above LP. An input instance and the question is the same as in VERTEX COVER, but the parameter under consideration is $k-\mathtt{lp}^*(G)$. Here $k$ is the budget for vertex cover in the input, and $\mathtt{lp}^*(G)$ is the value of an optimal fractional solution to the following linear program LPVC for VERTEX COVER.

$$
\begin{aligned}
[\mathsf{LPVC}] \qquad & \min \sum_{u \in V(G)} x_u & \\
& x_u + x_v \geq 1 & \forall (u,v) \in E(G) \\
& x_u \geq 0 & \forall u \in V(G)
\end{aligned}
$$

Observe that if an optimal solution to the above LP is integral, then it corresponds to an optimal vertex cover. In general, an optimal solution $x^*$ to LPVC is not necessarily integral.

The basis of this parameterization is the following fact:

$$\text{the size of an optimal vertex cover of } G \geq \mathtt{lp}^*(G).$$

Therefore, the parameter $k - \mathtt{lp}^*(G)$ can be assumed to be non-negative; otherwise, it holds that $k < \mathtt{lp}^*(G)$, which means that any vertex cover of $G$ has size strictly larger than $k$. Especially $G$ does not have a vertex cover of size at most $k$. We can correctly declare $(G, k)$ as a NO-instance in this case.

**Lemma 2.3.1.** *If $k < \mathtt{lp}^*(G)$, then the input instance $(G, k)$ is a* NO*-instance.*

### Half-integrality of LP for VERTEX COVER

Now we establish the half-integrality of LPVC, that is, the property that there is always an optimal fractional solution $x^*$ to LPVC such that $x^*_v \in \{0, \frac{1}{2}, 1\}$ for every $v \in V$.

Let $x^*$ be an optimal fractional solution fo LPVC, which is not necessarily half-integral. We partition[3] $V(G)$ according to their values in $x^*$.

- $H_0 := \{u \in V(G) : x^*_u > 0,5\}$
- $C_0 := \{u \in V(G) : x^*_u < 0,5\}$
- $R_0 := \{u \in V(G) : x^*_u = 0.5\}$

It is known that LP can be solved in polynomial time, hence the partition $(R_0, H_0, C_0)$ can be found in polynomial time. We shall see that $x^*$ can be converted into a half-integral solution, i.e. $x_u \in \{0, 0.5, 1\}$ while maintaining the optimality, and thus showing that there exists a half-integral optimal solution to LP formulation of VERTEX COVER.

**Observation 2.3.1.** *Any edge incident with $C_0$ is incident with $H_0$. In particular, $C_0$ is an independent set of $G$.*

**Lemma 2.3.2.** *For every subset $H'_0 \subseteq H_0$, we have $|H'_0| \leq |N(H'_0) \cap C_0|$.*

[3] $H, C$ and $R$ represent 'Head', 'Crown' and 'Rest'.

**Proof:** Suppose not, i.e. there exists $H_0' \subseteq H_0$ such that $|H_0'| > |N(H_0') \cap C_0|$. Take $\epsilon = \min\{x_u^* - 0.5 : u \in H_0'\}$ and note that $\epsilon > 0$. Consider a solution $x'$ defined as:

$$x'_u = \begin{cases} x_u^* - \epsilon & \text{if } u \in H_0' \\ x_u^* + \epsilon & \text{if } u \in N(H_0') \cap C_0 \\ x_u^* & \text{otherwise} \end{cases}$$

It is easy to verify that $x'$ is a feasible LP solution. The objective value of $x'$ equals

$$\sum_{u \in V(G)} x_u^* + \epsilon(-|H_0'| + |N(H_0') \cap C_0)|) < \sum_{u \in V(G)} x_u^*.$$

This contradict the optimality of $x^*$, thus our claim follows. □

**Lemma 2.3.3.** *The following solution $\tilde{x}$ is optimal to* LPVC*:*

$$\tilde{x}_u = \begin{cases} 0.5 & \textit{if } u \in R_0 \\ 1 & \textit{if } u \in H_0 \\ 0 & \textit{if } u \in C_0 \end{cases}$$

**Proof:** Let $H_0'$ be the set of vertices in $H_0$ with $x_u^* < 1$ and $C_0'$ be the set of vertices in $C_0$ with $x_u^* > 0$. Among all optimal solutions to LP yielding the partition $(R_0, H_0, C_0)$, choose $x^*$ so as to minimize the set $H_0' \cup C_0'$. We shall show that $H_0' \cup C_0' = \emptyset$, which establishes $x^* = \tilde{x}$ and thus the statement.

For the sake of contradiction, $H_0' \cup C_0' \neq \emptyset$. Take $\delta = \min\{1 - x_u^* : u \in H_0'\} \cup \{x_u^* : u \in C_0'\}$ and note that $\delta > 0$. Consider a solution $x'$ defined as:

$$x'_u = \begin{cases} x_u^* + \delta & \text{if } u \in H_0' \\ x_u^* - \delta & \text{if } u \in C_0' \\ x_u^* & \text{otherwise.} \end{cases}$$

As we chose minimum $\delta$, it holds $x' \geq 0$. It is straightforward to verify that $x'$ is feasible.

Now we argue that $x'$ is an optimal LP solution. From Lemma 2.3.2, we know $|H_0'| \leq |N(H_0') \cap C_0|$. Also observe that $N(H_0') \cap C_0 \subseteq C_0'$ since otherwise, there would be an edge $(u, v)$ with $u \in H_0'$ and $v \in C_0 \setminus C_0'$ such that $x_u^* + x_v^* < 1 + 0 = 1$, contradicting the feasibility of $x^*$. We derive $|H_0'| \leq |C_0'|$. Then, the objective value of $x'$ equals

$$\sum_{u \in V(G)} x_u^* + \delta(|H_0'| - |C_0'|),$$

which does not exceed $\sum_{u \in V(G)} x_u^*$. Therefore, $x'$ is an optimal solution such that $\{u \in H_0 : x'_u < 1\} \cup \{u \in C_0 : x'_u > 0\} \subsetneq H_0' \cap C_0'$, a contradiction.

Lastly, observe that at least one vertex from $H_0' \cup C_0'$ now has a new LP value set to 1 or 0. In particular, the increase / decrease of the LP solution by $\delta$ does not change the underlying partition $(R_0, H_0, C_0)$. This contradicts the choice of $x^*$ as a minimizer of $|H_0' \cup C_0'|$. This complete the proof. □

From now on we assume that $x^*$ is a half-integral solution as in Lemma 2.3.3.

### Preprocessing: how to get all-$\frac{1}{2}$ optimal solution

From the previous discussion, we know that a half-integral optimal solution $x^*$ for VERTEX COVER can be obtained in polynomial time. We define the set $V_a(x^*)$ as the set of all vertices $v$ such that $x^*_v = a$ for $a \in \{0, .5, 1\}$. When $x^*$ is obvious from the context, we drop it. From the half-integrality of $x^*$, it is obvious that $(V_0, V_1, V_{\frac{1}{2}})$ forms a partition of $V$.

**Lemma 2.3.4.** *For the graph $G[V_{\frac{1}{2}}]$, all-$\frac{1}{2}$ is an optimal fractional solution.*

**Proof:** If all-$\frac{1}{2}$ solution, i.e. $(x^*_v = 0.5)_{v \in V_{\frac{1}{2}}}$, is not an optimal fractional solution to LPVC of $G[V_{\frac{1}{2}}]$, consider an optimal fractional solution $z^*$ of $G[V_{\frac{1}{2}}]$ and observe

$$\sum_{v \in V_{\frac{1}{2}}} z^*_v < \sum_{v \in V_{\frac{1}{2}}} x^*_v.$$

We can obtain a new feasible solution to LPVC of $G$ by replacing the value of $x^*_v$ by $z^*_v$ for all $v \in V_{\frac{1}{2}}$ (and keep other values unchanged). It is not difficult to verify that the new fractional solution is feasible and has strictly smaller objective value than $x^*$, contradicting the optimality of $x^*$. □

**Lemma 2.3.5.** *There exists an optimal vertex cover containing $V_1$.*

**Proof:** Let $S$ be an optimal vertex cover of $G$ and let $S_{\frac{1}{2}} S_1, S_0$ be its intersections with $V_{\frac{1}{2}}, V_1$ and $V_0$. We take a new set

$$S' := (S \setminus S_0) \cup V_1,$$

i.e. the set obtained from $S$ by removing all of $C_0$ and adding all of $V_1$. This is a vertex cover of $G$ because any edge incident previously covered by $S_0$, thus incident with $V_0$ must be also incident with $V_1$, see Observation 2.3.1. We claim that $S'$ is again an optimal vertex cover. To see this, it suffices to show that the vertices newly added are not more than those removed, that is, $|V_1 - S_1| \leq |S_0|$.

Observe that $N(V_1 - S_1) \cap C_0 \subseteq S_0$ since otherwise $S_0$ fails to cover all edges incident with $V_1 - S_1$. Applying Lemma 2.3.2 for $V_1 - S_1$, we have

$$|V_1 - S_1| \leq |N(V_1 - S_1) \cap V_0| \leq |S_0|.$$

That is, the new vertex cover $S'$ containing $V_1$ is not larger than $S$, thus is an optimal vertex cover. □

**Lemma 2.3.6.** *The instance $I' = (G', k')$ is equivalent to $I = (G, k)$, where $G' = G[V_{\frac{1}{2}}]$ and $k' = k - |V_1|$. Furthermore, it holds that $k' - \mathtt{lp}^*(G[V_{\frac{1}{2}}]) = k - \mathtt{lp}^*(G)$.*

**Proof:** The proof of the first statement follows easily from Lemma 2.3.5. The second statement is derived from the following.

$$\begin{aligned}
\mathtt{lp}^*(G) &= \sum_{v \in V} x^*_v = \sum_{v \in V_0} x^*_v + \sum_{v \in V_1} x^*_v + \sum_{v \in V_{\frac{1}{2}}} x^*_v \\
&= \frac{1}{2} \cdot |V_{\frac{1}{2}}| + |V_1| \\
&= \mathtt{lp}^*(G[V_{\frac{1}{2}}]) + |V_1|,
\end{aligned}$$

where the last equality holds because of Lemma 2.3.4. □

Notice that Lemma 2.3.4 guarantees that all-$\frac{1}{2}$ is an optimal fractional solution to $G[V_{\frac{1}{2}}]$, but it does not guaranteed that it's the unique optimal solution. So how can we achieve the uniqueness? We do it greedily, namely, try each vertex $v$ and fix its LP value to one and see if it leads to yet another optimal fractional solution. To be precise, here is the reduction rule. We write $|y^*|$ to denote $\sum_{v \in V} y^*_v$.

**Reduction Rule 2.3.1.** *If there is a vertex $v \in V$ and a half-integral solution $y^*$ to* LPVC *such that $y^*_v = 0$ and $|y^*| = \mathtt{lp}^*(G)$, then output the new instance $I' = (G', k')$ with $G' = G[V_{\frac{1}{2}}(y^*)]$ and $k' = k - |V_1(y^*)|$*

The soundness of Reduction Rule 2.3.1 is derived immediately from Lemma 2.3.6. It is easy to see that after applying Reduction Rule 2.3.1 exhaustively, all-$\frac{1}{2}$ is the unique optimal fractional solution to LPVC of the resulting graph. Indeed, for an irreducible instance $(G, k)$ with respect to Reduction Rule 2.3.1, if there is a vertex $v$ with $x^*_v = 1$ for an optimal fractional solution $x^*$, then by Lemma 2.3.2 there must be at least one vertex whose value in $x^*$ equals 0. We just state this observation as the next statement.

**Observation 2.3.2.** *If $(G, k)$ is irreducible with respect to Reduction Rule 2.3.1 (i.e. the reduction is not applicable to $(G, k)$ anymore), then $x^*_v = \frac{1}{2}$ for all $v \in V$ is the unique fractional optimal solution to* LPVC.

### Branching + Preprocessing with runtime analysis

**Algorithm 2** Algorithm for VERTEX COVERabove LP

```
1: procedure VC-above-LP(G, k)
2:     Apply Reduction Rule 2.3.1 until it is no longer applicable
3:                                  ▷ Now, (G, k) has all-1/2 as the unique fractional optimal sol.
4:     if k < lp*(G) then return NO.                                   ▷ Lemma 2.3.1
5:     else if k ≥ 2 · lp*(G) then return YES.                         ▷ Lemma 2.3.7
6:     else                                  ▷ G has an edge and k − lp*(G) ≥ 0
7:         Pick an edge uv.
8:         return VC-above-LP(G − u, k − 1) or VC-above-LP(G − v, k − 1).
```

The algorithm for the problem VERTEX COVERabove LP is presented as **VC-above-LP**. The correctness of Line 2.3.1 is due to Lemma 2.3.1. To see the correctness of Line 4, we need the next lemma [4].

**Lemma 2.3.7.** *For any graph $G$, one can find a vertex cover $Z$ of size at most $2 \cdot \mathtt{lp}^*(G)$ in polynomial time. In particular, an instance $(G, k)$ to* VERTEX COVER *is a* YES*-instance if $k \geq 2 \cdot \mathtt{lp}^*(G)$.*

**Proof:** First compute an optimal half-integral solution $x^*$ to LPVC of $G$ and consider the usual partition $(V_0, V_1, V_{\frac{1}{2}})$. We take $Z = V_1 \cup V_{\frac{1}{2}}$. Since $V_0$ is an independent set, $Z$ is a vertex cover. Now

$$|Z| = |V_1| + |V_{\frac{1}{2}}| \leq 2(|V_1| + 0.5 \cdot |V_{\frac{1}{2}}|) = 2 \cdot \mathtt{lp}^*(G).$$

The second statement follows immediately. □

The rest of the correctness proof is rather tedious. To analyze the running time of **VC-above-LP**, consider the measure

$$\mu(G, k) = k - \mathtt{lp}^*(G).$$

[4] From $\mathtt{lp}^*(G) \leq$ the size of an optimal vertex cover, we deduce that the size of $Z$ is at most twice the size of an optimal vertex cover. That is, Lemma 2.3.7 presents a 2-approximation algorithm for VERTEX COVER

We want to argue that the measure decreases by at least $\frac{1}{2}$ in each branching of Line 12. The key observation is the following inequality (in fact, equality holds)

$$\texttt{lp}^*(G - v) \geq \texttt{lp}^*(G) - \frac{1}{2}.$$

Suppose this is not the case, that is, $\texttt{lp}^*(G - v) \leq \texttt{lp}^*(G) - 1$ and let $y^*$ be an optimal fractional solution of $G - v$, i.e. $|y^*| = \texttt{lp}^*(G - v)$. We can extend $y^*$ to a fractional solution $z^*$ of $G$ by setting the value at $v$ equal to 1. Then

$$|z^*| = |y^*| + 1 = \texttt{lp}^*(G - v) + 1 \leq \texttt{lp}^*(G),$$

which means that $z^*$ is an optimal fractional solution of $G$ which is not all-$\frac{1}{2}$. This contradicts that Reduction Rule 2.3.1 has been applied exhaustively in Line 3. Now

$$\mu(G - v, k - 1) = (k - 1) - \texttt{lp}^*(G - v) \leq k - 1 - (\texttt{lp}^*(G) - \frac{1}{2}) = \mu(G, k) - \frac{1}{2}.$$

By symmetry, the branching at the endpoint $u$ also decreases the measure by $\frac{1}{2}$.

During the branching algorithm, the measure $\mu$ can drop down to $-0.5$ at most (in which case, we declare as NO-instance). Therefore, the length of a longest root-to-leaf path in the search tree is $2\mu(G, k) = 2(k - \texttt{lp}^*(G))$. It follows that the number of leaves in the search tree is at most $4^{k-\texttt{lp}^*(G)}$, and the running time is $\mathcal{O}^*(4^{k-\texttt{lp}^*(G)})$.

Because of Line 4 (see Lemma 2.3.7 as well), we may assume that $k < 2 \cdot \texttt{lp}^*(G)$ since otherwise, we can immediately declare the given instance as YES, in constant time. Therefore, $k - \texttt{lp}^*(G) \leq \frac{k}{2}$. We summarize the result in the next statement.

**Lemma 2.3.8.** *The algorithm* **VC-above-LP** *solves, given an instance $(G, k)$, the problem* VERTEX COVER*above LP in time $\mathcal{O}^*(4^{k-\texttt{lp}^*(G)})$ as well as $\mathcal{O}^*(2^k)$-time* [5].

## Problems List

VERTEX COVER
**Instance:** a graph $G = (V, E)$, a positive integer $k$.
**Question:** Does $G$ have a vertex cover of size at most $k$, i.e. a set $X$ of vertices such that $G - X$ is an independent set?

ODD CYCLE TRANSVERSAL
**Instance:** a graph $G = (V, E)$, a positive integer $k$.
**Question:** Does $G$ have a vertex cover of size at most $k$, i.e. a set $X$ of vertices such that $G - X$ is bipartite?

[5] Recall that we already have a simple branching algorithm for VERTEX COVER running in time $\mathcal{O}^*(2^k)$. Therefore, the second running time might seem unnecessary. However, almost identical algorithm work for problems such as Multiway Cut and others, which are generalizations of VERTEX COVER. For those problems, this LP-based branching approach is the only known way to achieve $\mathcal{O}^*(2^k)$-time algorithm.

FEEDBACK VERTEX SET
**Instance:** a graph $G = (V, E)$, a positive integer $k$.
**Question:** Does $G$ have a feedback vertex set (fvs) of size at most $k$, i.e. a set $X$ of vertices such that $G - X$ is acyclic?

$d$-HITTING SET
**Instance:** a universe $V$, a family $\mathcal{E}$ of subsets of $V$ each of which having size at most $d$, a positive integer $k$.
**Question:** Is there a hitting set of $\mathcal{E}$, i.e. a subset $X$ of $V$ such that $X \cap e \neq \emptyset$ for every $e \in \mathcal{E}$?

$k$-PATH
**Instance:** a graph $G = (V, E)$, a positive integer $k$.
**Question:** Does $G$ have a path on $k$ vertices?

# Chapter 3

# Kernelization

## 3.1 Kernelization 101

A kernelization of $P$ is a polynomial-time (in $|x|$ and $k$) algorithm which transforms an instance $(x, k)$ of $P$ into another instance $(x', k')$ of $P$ satisfying

- equivalence: $(x, k) \in P$ if and only if $(x', k') \in P$
- size bound: $|x'| \leq g(k)$ and $k' \leq g(k)$ for some computable function

An instance $(x', k')$ obtained after applying kernelization is called a kernel. The function $g(k)$ is the size of a kernel.

Usually, a kernelization consists in applying a sequence of reduction rules. A reduction rule for $P$ is a polynomial-time (in $|x|$ and $k$) algorithm which transforms an instance $(x, k)$ of $P$ into an equivalent instance $(x', k')$ of $P$. The equivalence of $(x, k)$ and $(x', k')$ is also referred to as the soundness or safeness of the reduction rule. Notice that the size of the resulting instance of a reduction rule is not necessarily bounded. We say that an instance $(x, k)$ is irreducible with respect to a reduction rule $R$ if $R$ cannot be applied to $(x, k)$ anymore (or equivalently, applying $R$ does not change the instance).

**Lemma 3.1.1.** *A decidable parameterized problem $P$ is FPT if and only if it admits a kernelization.*

**Proof:** ($\Leftarrow$) If $P$ admits a kernel of size $g(k)$, then run the kernelization algorithm (takes polynomial time in $|x| + k$) and then do an exhaustive search on the obtained kernel to decide whether $(x', k') \in P$. The whole procedure is an FPT-algorithm.
($\Rightarrow$) Let $P$ has an FPT-algorithm $\mathcal{A}$ running in time $f(k) \cdot |x|^c$ for some constant $c$. Then run $\mathcal{A}$ on the given instance $(x, k)$ for time $|x|^{c+1}$. If it ouputs YES/NO answer, then produce a constant-size instance of $P$ accordingly. This would be the kernel. If $\mathcal{A}$ does not terminate in time $|x|^{c+1}$, this means $|x| < f(k)$. That is, the given instance $(x, k)$ is already a kernel, and thus output $(x, k)$. □

Devising a kernelization with small size bound $g(k)$ (usually, polynomial $g$) is an important research topic in parameterized complexity. Kernelization design involve the following steps.

- Devise reduction rules.
- Prove that the above reduction rules are safe.

- Prove that when an instance $(x', k')$ of $P$ is irreducible w.r.t the reduction rules, $|x'| \leq g(k)$ and $k' \leq g(k)$. The smaller the function $g$ is, the better. Notice that the equivalence of kernelization is automatically guaranteed by the safeness of reduction rules.

## 3.2 Simple kernelization: VERTEX COVER

We look at a simple kernelization for VERTEX COVER yielding $O(k^2)$ vertices.

Reduction Rule 1: If a vertex $v$ is isolated in $G$, then delete $v$. The new parameter is $k' := k$.

Reduction Rule 2: If a vertex $v$ is incident with at least $k+1$ edges in $G$, then delete $v$ and set $k' := k-1$.

It is trivial to see that Reduction Rule 1 is safe. To see that Reduction Rule 2 is safe, notice that any vertex cover of size at most $k$ in $G$ must contain $v$. Hence, if $(G, k)$ is a yes-instance, $(G - v, k - 1)$ is also yes-instance. The opposite direction of equivalence is straightforward.

Consider an instance $(G', k')$ for which none of Reduction Rules 1 and 2 can be applied, and let us analyze the size of $G'$. Since the parameter does not increase with the reduction rules, we know that $k' \leq k$.

Suppose $G'$ is a yes-instance, and $C$ is a vertex cover of $G'$ with $|C| \leq k'$. Since $(G', k')$ is irreducible with respect to Reduction Rule 2, every $v \in C$ is incident with at most $k'$ edges of $G$. As each edge of $G'$ is incident with $C$ and $k' \leq k$, there are at most $|C| \cdot k' \leq k^2$ edges in $G$. Due to Reduction Rule 1, there's no isolated vertex in $G'$ and thus every vertex in $V(G) \setminus C$ is adjacent with some vertex of $C$. For $deg(v) \leq k'$ for all $v \in C$, we can assign each vertex of $V(G') \setminus C$ to a vertex of $C$ so that each vertex of $C$ is assigned with at most $k'$ vertices. Now, we have $|V(G')| = |V(G')) \setminus C| + |C| \leq |C| \cdot k' + k' = k(k+1)$. To bound $|E(G')|$, we similarly observe that every edge of $G'$ is incident with (at least) one vertex of $C$, leading to $|E(G')| \leq |C| \cdot k' \leq k^2$.

Hence, if $|V(G')| > k(k+1)$ or $|E(G')| > k^2$, we know that $(G', k')$ is a no-instance and output a constant-size no-instance as a kernel. Otherwise, $(G', k')$ is a kernel with $|V(G')| \leq k(k+1)$ and $|E(G')| \leq k^2$.

## 3.3 LP-based kernelization for VERTEX COVER

**Theorem 3.3.1** (Nemhauser-Trotter Theorem, NT Theorem in short)**.** *Given a graph $G$, a partition $(R, H, C)$ satisfying the following can be computed in polynomial time.*

*(a.) For any vertex cover $S_r$ of $G[R]$, $S_r \cup H$ is a vertex cover of $G$.*

*(b.) There exists an optimal vertex cover containing $H$.*

*(c.) Any vertex cover of $G[R]$ is of size at least $\frac{1}{2}|R|$.*

Before presenting an algorithm for computing such a partition $(R, H, C)$, let's think how to use the above NT Theorem for computing a kernel. We propose the following reduction rules.

**Reduction Rule 1:** Let $(R, H, C)$ be a partition such that (a)-(c) of NT Theorem is met and $H \cup C \neq \emptyset$. Then delete $H \cup C$ from $G$ and set $k' := k - |H|$, i.e. the new instance is $(G[R], k - |H|)$.

**Lemma 3.3.1.** *Reduction Rule 1 is safe; $(G, k)$ is a* YES*-instance if and only if $(G[R], k - |H|)$ is a* YES*-instance.*

**Proof:** Suppose $(G, k)$ is a yes-instance and let $S$ be an optimal vertex cover. Notice that $|S| \leq k$. By condition (b) of NT Theorem, we can assume that $H \subseteq S$. Take $S_r := S \cap R$ and observe that $S_r$ is a vertex cover of $G[R]$. Due to condition (a), $S_r \cup H$ is a vertex cover of $G$. Since $S_r \cup H \subseteq S$ and is a vertex cover of $G$, the optimality of $S$ implies that $S \cap C = \emptyset$. Hence, $|S_r| = |S| - |H| \leq k - |H|$ and $(G[R], k - |H|)$ is a yes-instance.

For the opposite direction, suppose $(G[R], k - |H|)$ is a yes-instance and let $S_r$ be a vertex cover of $G[R]$ of size at most $k - |H|$. By condition (a), $S_r \cup H$ is a vertex cover of $G$ and its size is at most $k$. That is, $(G, k)$ is a yes-instance. □

**Lemma 3.3.2.** VERTEX COVER *admits a kernel containing at most* $2k$ *vertices.*

**Proof:** Consider the following (kernelization) algorithm: find a partition $(R, H, C)$ as in NT Theorem in polynomial time and apply Reduction Rule 1 if possible. Let $(G', k') = (G[R], k - |H|)$ be the resulting instance, which can be identical to $(G, k)$ if $H \cup C = \emptyset$ and Reduction Rule 1 was not applied. The algorithm then performs the following.

- If $|R| > 2k$ or $k < 0$, output a constant-size no-instance.
- Otherwise, output $(G', k')$.

In the first case, observe that any vertex cover of $G[R]$ contains more than $k$ vertices by condition (c) of NT Theorem, and thus $(G', k')$ is a no-instance. Therefore the output instance is equivalent to $(G', k')$, The equivalence of the output instance to $(G, k)$ in both cases follows by Lemma 3.3.1. It is clear that the output instance has at most $2k$ vertices and that the algorithm runs in polynomial assuming the partition $(R, H, C)$ of NT Theorem can be found in polynomial time. □

How can we find a partition $(R, H, C)$ as in NT Theorem? There are several nice proofs of NT Theorem, and we have already seen a version using LP Relaxation of VERTEX COVER during the lecture in Week 01 (See the lecture note). Here is a reminder.

$$\begin{aligned} \min \sum_{u \in V(G)} x_u & \\ x_u + x_v \geq 1 & \qquad \forall (u, v) \in E(G) \\ x_u \geq 0 & \qquad \forall u \in V(G) \end{aligned}$$

Define

- $R_0 := \{u \in V(G) : x^*_u = 0.5\}$
- $H_0 := \{u \in V(G) : x^*_u > 0, 5\}$
- $C_0 := \{u \in V(G) : x^*_u < 0, 5\}$

LP can be solved in polynomial time, hence the partition $(R_0, H_0, C_0)$ can be found in polynomial time. We already have seen that this partition satisfies the conditions (a)-(c) of NT Theorem in the previous lectures. Here is a recap.

**Lemma 3.3.3.** *The partition* $(R_0, H_0, C_0)$ *meets the condition (a).*

**Proof:** Observe that $C_0$ is an independent set: indeed if there is an edge between $u, v \in C_0$, we have $x^*_u + x^*_v < 0.5 + 0.5 = 1$, violating the corresponding inequality in LP. For the same reason, there is no edge between $C_0$ and $R_0$. This means that $N(C_0) \subseteq H_0$, from which condition (a) holds. □

**Lemma 3.3.4.** *The partition $(R_0, H_0, C_0)$ meets the condition (b).*

**Proof:** By Lemma 2.3.5. □

**Lemma 3.3.5.** *The partition $(R_0, H_0, C_0)$ meets the condition (c).*

**Proof:** Recall that $y^*_v = 0.5$ for all $v \in R_0$ is an optimal fractional solution to LPVC of $G[R_0]$, see Lemma 7 of Week 01 Lecture Note. As any vertex cover of a graph has size at least the objective value of an optimal fractional solution to LPVC, we conclude that the size of an optimal vertex cover of $G[R_0]$ is at least $\sum_{v \in R_0} y^* = 0.5|R_0|$. □

## 3.4 $4k$-vertex kernel for VERTEX COVER using Crown Decomposition

A crown decomposition of a graph $G = (V, E)$ is a partition $(R, H, C)$ of $V$ satisfying the following conditions.

(i) $H \neq \emptyset, C \neq \emptyset$,

(ii) $C$ is an independent set,

(iii) $N(C) \subseteq H$ ("$H$ separates $C$ from $R$")

(iv) For every $H' \subseteq H$, it holds that $|H'| \leq |N(H') \cap C|$ ("Hall's condition holds from $H$ toward $C$").

Hall's condition is important in the context of maximum matching / vertex cover in a bipartite graph. We recall the celebrated Hall's theorem. We say that a vertex $v$ of $G$ is saturated by a matching $M$ if $v$ is incident with some edge of $M$. A vertex set is saturated by a matching $M$ if every vertex in the set is saturated by $M$.

**Theorem 3.4.1** (Hall's theorem)**.** *Let $G$ be a bipartite graph on the bipartition $X$ and $Y$. There is a matching saturating $X$ if and only if for every $X' \subseteq X$, we have $|X'| \leq |N(X') \cap Y|$.*

**Theorem 3.4.2** (König's theorem)**.** *The size of an optimal vertex cover equals the size of a maximum matching on a bipartite graph.*

The proof of the next lemma is omitted as it is essentially the same proof as in Lecture Note for Week 01.

**Lemma 3.4.1.** *If $(R, H, C)$ is a crown decomposition of $G$, then there exists an optimal vertex cover of $G$ containing all vertices of $H$.*

We can consider the following reduction rule.

**Reduction Rule 3.4.1.** *If there is a crown decomposition $(R, H, C)$ of $G$, output $(G', k')$ where $G' := G - (H \cup C)$ and $k' := k - |H|$.*

**Lemma 3.4.2.** *Reduction Rule 3.4.1 is safe.*

**Proof:** Suppose that $(G, k)$ is a YES-instance. By Lemma 3.4.1, there exists a vertex cover $X$ of size at most $k$ which contain $H$ entirely. Now, $X - H$ is a vertex cover of $G[R] = G - (H \cup C)$ of size at most $k - |H|$.

Conversely, if $X'$ is a vertex cover of $G[R]$ of size at most $k - |H|$, let us consider the vertex set $X' \cup H$. Clearly its size is at most $k$ and it is routine to verify that $X' \cup H$ is indeed a vertex cover of $G$. □

**Theorem 3.4.3.** *Let $G$ be a graph without isolated vertices on at least $4k + 1$ vertices. In polynomial time, one can*

- *either find a matching of size at least $k + 1$,*
- *or find a crown decomposition*

**Proof:** Consider the algorithm 3 **VC-CROWN**$(G, k)$. It suffices to prove that $(R, H, C)$ at line 13 is a crown decomposition, and that deciding if there is a set $H'$ as in the while-condition at line 8 and finding one if exists can be done in polynomial time.

First, we prove that $(R, H, C)$ at line 13 is a crown decomposition. To begin with, we show that each partitions $(R_i, H_i, C_i)$ obtained when performing the while-loop maintains the invariant (i)-(iii) of the crown decomposition, and additionally the following invariant $(\star)$.

$$(\star) \qquad |H_i| < |C_i|.$$

Consider $(R_0, H_0, C_0)$. Note that $|H_0| \leq |V(M)| \leq 2k$ since otherwise the algorithm would have terminated at line 6. From $|C_0| = |V| - |V(M)| \geq 2k + 1$ (see line 3), we know that $|H_0| < |C_0|$, the invariant $(\star)$. This implies $C_0 \neq \emptyset$, which in turn implies that $H_0 = N(C_0) \neq \emptyset$ because no vertex of $G$ is isolated, (see line 2). The invariant (ii) is satisfied because $M$ is a maximal matching (line 5). The invariant (iii) holds by construction at line 7.

We show that $(R_i, H_i, C_i)$ maintains the invariant (i)-(iii) and $(\star)$ for every $i \geq 0$ counted during the while-loop. This is the case for $i = 0$ and it suffices to establish the following.

$(\diamond)$ If $(R_i, H_i, C_i)$ satisfies (i)-(iii) and $(\star)$, then $(R_{i+1}, H_{i+1}, C_{i+1})$ satisfies (i)-(iii) and $(\star)$.

Regarding $(\star)$, note that

- $|H_i| < |C_i|$ $\quad \because (R_i, H_i, C_i)$ satisfies $(\star)$ by induction hypothesis,
- $|H'| > |N(H') \cap C_i|$ $\quad \because$ the condition of the while-loop at line 8,

which, combined with the construction of $C_{i+1}$ and $H_{i+1}$, leads to $(\star)$. This also establishes (i) as well by the same argument as in the case of $i = 0$. That $C_{i+1}$ is independent follows from $C_{i+1} \subseteq C_i$. To see (iii), suppose the contrary, i.e. $C_{i+1}$ is adjacent with $R_{i+1}$. By induction hypothesis, $C_{i+1}$ is adjacent with $R_{i+1} - R_i$, that is, with $H' \cup (N(H') \cap C_i)$. This is impossible because $C_i$ is independent by the invariant (ii) for $i$ and if $C_{i+1}$ has a vertex adjacent with $H'$, then such a vertex is included in $N(H') \cap C_i$ and should be moved to $R_{i+1}$. This establishes the claim $(\diamond)$.

Now, the partition $(R, H, C)$ at line 13 has no vertex set $H' \subseteq H$ with $|H'| > |N(H') \cap C|$. That is, the partition at hand satisfies (iv) on top of (i)-(iii) and it is a crown decomposition.

Secondly, let us see that deciding if there is a set $H'$ as in the while-condition at line 8 and finding one if exists can be done in polynomial time. We use the known polynomial-time algorithm (e.g. Hopcroft-Karp,

there are many) for the MINIMUM VERTEX COVER problem on the bipartite graph $G[H_0, C_0]$, namely the bipartite graph obtained from the subgraph of $G$ induced on $H_0 \cup C_0$ by removing all edges within $H_0$. Let $X$ be an optimal vertex cover of $G[H_0, C_0]$.

If $H_0 \subseteq X$ (implying $H_0 = X$), by Theorem 3.4.2 there is a matching saturating $H_0$. This means that there is no such $H'$ as in line 8 by Theorem 3.4.1. If $H_0 - X \neq \emptyset$ (i.e. these are vertices of $H_0$ not in the vertex cover $X$), note that $N(H_0 - X) \cap C_0$ must be all contained in the vertex cover $X$. Take $H' := H_0 - X$. If $|H'| \leq |N(H') \cap C_0|$, we can alternatively take a new vertex cover $X' := (X - N(H') \cap C_0) \cup H'$; note that $|X'| \leq |X|$ and it is an easy exercise to verify that $X'$ is indeed a vertex cover of $G[H_0, C_0]$. Now $H_0 \subseteq X'$ and there is no such $H'$ by Theorem 3.4.1. If $|H'| > |N(H') \cap C_0|$, we have found a vertex set $H'$ as required at line 8. □

One can improve the bound of Theorem 3.4.3 to $3k + 1$ instead of $4k + 1$. The proof can be found on page 26 of the textbook by Cygan et al. "Parameterized Algorithms."

**Algorithm 3** Kernelization for VERTEX COVER using crown decomposition

1: **procedure VC-CROWN**$(G, k)$
2: Remove all isolated vertices.
3: **if** $|V| \leq 4k + 1$ **then return** $(G, k)$.
4: **else**
5: Let $M$ be a maximal matching.
6: **if** $|M| \geq k + 1$ **then return NO**
7: $C_0 \leftarrow V - V(M)$, $H_0 \leftarrow N(C_0)$, $R_0 \leftarrow V - (C_0, H_0)$, and $i \leftarrow 0$
8: **while** there exists $H' \subseteq H_i$ with $|H'| > |N(H') \cap C_i|$ **do**
9: $C_{i+1} \leftarrow C_i - N(H') \cap C_i$
10: $H_{i+1} \leftarrow H_i - H'$
11: $R_{i+1} \leftarrow R_i \cup H' \cup (N(H') \cap C_i)$
12: $i \leftarrow i + 1$
13: Let $(R, H, C) = (R_i, H_i, C_i)$.
14: Return **VC-CROWN**$(G - (H \cup C), k - |H|)$

## 3.5 Quadratic kernel for FEEDBACK VERTEX SET

**Basic Preprocessing.** The input graph $G$ for FEEDBACK VERTEX SET may have a loop (i.e. an edge whose both endpoints are identical) or parallel edges (i.e. multiple edges between a pair of vertices) as we may create loops and parallel edges while applying reduction rules. Graphs without loops and parallel edges are called simple graphs, and graphs with possible loops and parallel edges are called multigraphs. For formal definition of multigraphs, see Chapter 1.10 of Diestel's book. Very often, we assume that the graphs under consideration are simple graphs, but there are situations where multigraphs are more natural objects to consider. The kernelization of FEEDBACK VERTEX SET is such a situation.

In a multigraph, the degree of a vertex $v$ counts the number of edges incident with $v$, where a loop at $v$ contributes 2 to the degree of $v$. One can have a cycle of length 1 (a loop) or of length 2 (two parallel edges).

In each of the following reduction rules, the obtained graph is referred to as $G'$.

- Reduction Rule 1: If a vertex $v$ has a loop, then delete $v$ and set $k' := k - 1$.

- Reduction Rule 2: If a vertex $v$ has degree at most 1 in $G$, then delete $v$ and $k' := k$.
- Reduction Rule 3: If there are more than two parallel edges between $u$ and $v$, delete all edges but two of them and set $k' := k$.
- Reduction Rule 4: If a vertex $v$ is incident with exactly two edges $vx$ and $vy$, delete $v$ and add an edge $xy$. Let $k' := k$. Note that this operation can create parallel edges between $x$ and $y$ (if they are already adjacent) or a loop (if $x = y$).

The next two lemmas are straightforward and their proof is left to the readers.

**Lemma 3.5.1.** *Each of Reduction Rule 1-4 is safe and can be applied in polynomial time.*

**Lemma 3.5.2.** *Let $(G, k)$ be an instance to* FEEDBACK VERTEX SET *which is irreducible with respect to Reduction Rules 1-4. Then we have $deg(v) \geq 3$.*

**What next? An overview.** Suppose that $(G, k)$ is irreducible with respect to the basic Reduction Rules 1-4. If the maximum degree of $G$ is bounded by $d$, then one can easily bound the size of $G$.

**Lemma 3.5.3.** *Let $(G, k)$ be an instance to* FEEDBACK VERTEX SET *which is irreducible with respect to Reduction Rules 1-3. If $deg(v) < d$, then $|V| < (d+1)k$ and $|E| < 2dk$ unless $(G, k)$ is a* NO*-instance.*

**Proof:** If $(G, k)$-is a NO-instance, we have nothing to prove. Suppose that $X$ is a feedback vertex set of $G$ of size at most $k$. Let $E(V - X, X)$ be the set of edges with one endpoint in $V - X$ and another endpoint in $X$, and $E(V - X)$ is the set of edges with both endpoints in $V - X$. The former is upper bounded by $d|X|$ due to $deg(v) \leq d$ and the latter is upper bounded by $|V - X| - 1$ because $G[V - X]$ is acyclic. Now

$$3 \cdot |V - X| \leq \sum_{v \in V - X} deg(v) = |E(V - X, X)| + 2 \cdot E(V - X) < d|X| + 2(|V - X| - 1),$$

from which we deduce $|V| < (d+1)|X| \leq (d+1)k$ and

$$\begin{aligned} |E| &= |\text{edges incident with } X| + |\text{edges with both endpoints in } V - X| \\ &< d|X| + |V - X| - 1 \\ &\leq |V| + (d-1)|X| < 2dk. \end{aligned}$$

□

Based on Lemma 3.5.3, our strategy for getting an $O(k^2)$-vertex kernel is to upper bound the maximum degree of $G$ by $O(k)$. A priori, there is no reason why the input graph $G$ has degree $O(k)$ - a vertex may have an arbitrarily large degree. In what follows, if there is a vertex $x$ of degree large enough ( $\geq 8k$ suffices), either there is a certificate forcing $x$ in any feedback vertex set of size at most $k$ or there is a vertex set $S'_x \subseteq V - x$ such that one may assume any optimal feedback vertex set contains either $x$ or $S'_x$. Each of these two cases can be handled with suitable reduction rules.

An $x$-cycle is a cycle traversing $x$, which can be one, two or at least three in general. But as we can assume that the instance at hand is irreducible with respect to Reduction Rule 1, the length of an $x$-cycle will be at least two whenever a relevant reduction rule or lemmas are applied, unless stated otherwise.

An $x$-flower is a collection of $x$-cycles such that any pair of cycles intersect exactly at $\{x\}$. We say that $x$-flower is of order $k$ if there are $k$ cycles in the collection. When there is an $x$-flower of order at least $k+1$,

then any feedback vertex set of size at most $k$ must contain $x$ and the safeness of the next reduction rule is immediate.

- Reduction Rule 5: If there is an $x$-flower of order $k+1$, then delete $x$ and set $k' := k-1$.

What if there is no $x$-flower of order $k+1$? We shall see that one of the following happens; recall that $x$ does not have a loop.

(i) Either there an $x$-flower of order $k+1$,

(ii) or, no $x$-flower of order $k+1$ exists. In this case, there is a vertex set $S_x \subseteq V - x$ of size at most $2k$ which hits all $x$-cycles.

That one of (i) and (ii) must happen is Corollary 3.5.1. The eligible one can be decided in polynomial time by Theorem 3.5.1 and Corollary 3.5.1. When (i) is the case, one can apply Reduction Rule 5. What if (ii) is the case? Lemma 3.5.9 says that if (ii) is the case **AND** $deg(x) \geq 8k$, one can identify a set $\emptyset \neq S'_x \subseteq S_x$ such that there is an optimal feedback vertex set (which must hit all $x$-cycles) containing $x$ or there is one containing $S'_x$. This feature can be implemented as a modified instance with $x$ having double edges with all vertices of $S'_x$. Reduction Rule 6 describes the detail of how to implement this feature so as to strictly decrease some measure on the input graph.

Below we see more details, and why things work as they do. We maintain the assumption that the input instance $(G, k)$ is irreducible with respect to Reduction Rules 1-4.

**What next? Distinguishing between (i) and (ii).** For a vertex subset $A$ of $G$, we say that a path $P$ is an $A$-path if $P$ has length at least one and the vertex set of $P$ intersects with $A$ precisely at the two endpoints of $P$; i.e. both endpoints of $P$ are in $A$ and no internal vertex of $P$ is in $A$.

For $i = 1, 2$, let $A_i \subseteq N(x)$ be the set of neighbors of $x$ connected with $x$ by $i$ edges. How does a hitting set $S_x$ of all $x$-cycles avoiding $x$ look like? First of all, it must contain $A_2$ entirely and this hits all $x$-cycles of length 2. Consider an $x$-cycle $C$ of length at least three. For sure, the path $P := C - x$ must start from a vertex of $N(x)$ and end in one of $N(x)$. As $S_x$ already contains $A_2$, we may consider only the case when the endpoints of $P$ are in $A_1$. The path $P$ may traverse $A_1$ many times, but one can choose a subpath $P'$ of $P$ whose internal vertices are outside of $A_1$, that is, $P$ always contains a subpath $P'$ which is an $A_1$-path. Note that $P'$ together with $x$ forms an $x$-cycle whose vertex set is contained in $V(C)$.

This summarizes to the next observation.

**Lemma 3.5.4.** *Let $x$ be a vertex of $G = (V, E)$.*

- *The maximum order of $x$-flower equals $|A_2|$ plus the maximum number of vertex-disjoint $A_1$-paths in $G - x - A_2$.*
- *$S_x \subseteq V - x$ is a hitting set of all $x$-cycles if and only if $S_x$ contains $A_2$ and $S_x - A_2$ hits all $A_1$-paths of $G - x - A_2$.*

**Proof:** To see the $\leq$-inequality in the first statement, let $\mathcal{F}$ be an $x$-flower which minimizes $|\bigcup_{C \subseteq \mathcal{F}} C|$. it is easy to see that if $C \in \mathcal{F}$ intersects $A_2$ then $C$ is a 2-cycle consisting of $x$ and a vertex $u \in A_2$; otherwise contradicting the minimum cardinality assumption on $\mathcal{F}$. In particular, this means that all the double edges incident with $x$ (and with $A_2$) are members of $\mathcal{F}$. Therefore, for any $x$-cycle $C \in \mathcal{F}$ of length at least three, we have $C \cap N(x) \subseteq A_1$, $C \cap A_2 = \emptyset$ and the path $C - x$ connects two distinct vertices of $A_1$. Again by

the minimum cardinality assumption, we conclude that $C - x$ is an $A_2$-path. The $\geq$-inequality is immediate from the fact that $A_2$ and a collection of vertex disjoint $A_1$-paths can be extended to an $x$-flower.

The second statement is easy to verify and left to the readers. □

Let $\ell := k - |A_2|$. Thanks to Lemma 3.5.4, the case of (i) and (ii) above translates to the following two cases. Now let $A := A_1$ and $H := G - A_2 - x$.

(i') Either there are $\ell + 1$ vertex-disjoint $A$-paths in $H$,

(ii") or, the maximum number of vertex-disjoint $A$-paths in $H$ is at most $\ell$. In this case, there is a vertex set $S_x \subseteq V(H)$ of size at most $2\ell$ which hits all $A$-paths of $H$.

We use the next theorem by Gallai to show that one of the case (i') and (ii') happens, and to decide which case applies can be done in polynomial time. The proof of Gallai's Theorem can be found in Chapter 9.1.1 of the textbook by Cygan et. al.

**Theorem 3.5.1** (Gallai 1961, Schrijver 2001)**.** *Let $G = (V, E)$ and $A \subseteq V$. Then the maximum number of pairwise vertex-disjoint $A$-paths equals the minimum of*

$$|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor,$$

*where the minimum is taken over all vertex subsets $X \subseteq V$, and $\mathcal{C}(G - X)$ is the collection of all connected components of $G - X$. Moreover, both a maximum collection of pairwise vertex-disjoint $A$-paths and a vertex set $X$ minimizing the right-hand side of the equality can be computed in polynomial-time.*

**Lemma 3.5.5.** *Let $G = (V, E)$ be a graph. If the maximum number of pairwise vertex-disjoint $A$-paths is at most $k$, then is a hitting set $Z$ of all $A$-paths with $|Z| \leq 2k$, i.e. a vertex set $Z \subseteq V$ such that there is no $A$-path in $G - Z$.*

**Proof:** By Theorem 3.5.1, there exists $X \subseteq V$ such that $|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor \leq k$. We add $X$ to $Z$, and additionally $2 \cdot \frac{|A \cap C|}{2}$ vertices to $Z$ from $A \cap C$ for each connected component of $C$ of $G - X$. Note that

$$|Z| = |X| + \sum_{C \in \mathcal{C}(G-X)} 2 \cdot \lfloor \frac{|A \cap C|}{2} \rfloor \leq 2\big(|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor\big) \leq 2k.$$

It remains to see that $G - Z$ contains no $A$-path. Suppose that $P$ is an $A$-path in $G - Z$. Then there exists a connected component $C$ of $G - Z$ entirely containing $P$. As the two (distinct) endpoints of $P$ must be in $A$, the path $P$ witnesses that $|(C \cap A) - Z| \geq 2$. Recall that $C$ is in turn contained in some connected component $C^*$ of $G - X$ as $X \subseteq Z$. This means that $|(C^* \cap A) - Z| \geq 2$, which contradicts the fact that $Z$ takes all but at most one vertex from $K \cap A$ for each connected component $K$ of $G - X$. □

**Corollary 3.5.1.** *There is a polynomial-time algorithm which, given a graph $G = (V, E)$, a vertex $x$ without loop, and a non-negative integer $k$, either detects an $x$-flower of order $k + 1$ or returns a vertex set $S_x \subseteq V - x$ of size at most $2k$ hitting all $x$-cycles.*

**Proof:** Let $A_2 \subseteq N(x)$ be the set of neighbors of $x$ connected with $x$ by at least two edges. Let $\ell := k - |A_2|$ and let $H := G - A_2 - x$. By Lemma 3.5.5, there a collection of $\ell + 1$ vertex-disjoint $A$-paths in $H$ or there is a vertex set $S_x \subseteq V - A_2 - x$ of size at most $2\ell$ hitting all $A$-paths of $H$. Moreover, one can detect such a collection of $\ell + 1$ vertex-disjoint $A$-paths in $H$ or return a vertex set $S_x$ in polynomial time

by Theorem 3.5.1. In the former case, there is an $x$-flower of order $k+1$ in $G$ by Lemma 3.5.4. In the latter case, $S_x \cup A_2$ is a hitting set of all $x$-cycles of $G$ by the same lemma. It remains to note that $|S_x \cup A_2| \leq 2\ell + |A_2| \leq 2k - |A_2| \leq 2k$. □

**What next? Further into the case (ii).** Let $S_x \subseteq V - x$ be a vertex set of size at most $2k$ hitting all $x$-cycles of $G$. Our goal is to show that the next, last Reduction Rule is applicable in this case provided that $deg(x) \geq 8k$. We first present the reduction rule.

- Reduction Rule 6: Suppose there exist a vertex $x$, a vertex subset $S \subseteq V - x$ and a set $\mathcal{T}$ (not necessarily all the connected components of $G - S - x$) of connected components of $G - S - x$ such that

  1. $G[C]$ is a tree for each $C \in \mathcal{T}$,
  2. there is exactly one edge between $x$ and $C$ for each $C \in \mathcal{T}$,
  3. for every $Z \subseteq S$, there are at least $2|Z|$ connected components of $\mathcal{T}$ neighboring $Z$.

  Then the new graph $G'$ is obtained by

  – deleting the edges connecting $x$ and the connected components of $\mathcal{T}$, and

  – adding double edges between $x$ and $S$ (and deleting some edges so that there are exactly two edges between $x$ and each vertex of $S$).

  We set $k' := k$.

We need to prove two things; (a) Reduction Rule is safe, and (b) if $deg(x) \geq 8x$ and $S_x \subseteq V - x$ is a hitting set of all $x$-cycles of size at most $2k$, there exists $S'_x \subseteq S_x$ which satisfies the condition of Reduction Rule 5 by setting $S := S'_x$ unless $(G, k)$ is a trivial NO-instance.

**Lemma 3.5.6.** *Reduction Rule 6 is safe.*

**Proof:** Let $X'$ be an optimal feedback vertex set of $G'$. If $x \in X'$, then $X'$ is a feedback vertex set of $G$ as well because $G$ and $G'$ differ only at edges incident with $x$, in particular any cycle not traversing $x$ is present in $G$ if and only if it is in $G'$.

Let $X$ be an optimal feedback vertex set of $G$. If $x \in X$, the same argument in the previous paragraph applies, and $X$ is a feedback vertex set of $G'$. So, assume that $x \notin X$. We show that there is an optimal feedback vertex set $X^*$ which entirely contain $S$. If $S \subseteq X$, the claim trivially holds. If $Z := S \setminus X \neq \emptyset$, consider a vertex $z \in Z$ and the tree components of $\mathcal{T}$ neighboring $z$. There are at least two such tree components (Condition 3), and $X$ must intersect all those tree components save at most one; otherwise, the tree components free from $X$ together with $z$ and $x$ contain a cycle, contradiction that $X$ is a feedback vertex set. Let us count the number of tree components of $\mathcal{T}$ neighboring $Z$, which is

$$\leq \text{\# tree components intersecting } X + \text{\# tree components free from } X \leq |X \cap \bigcup_{T \in \mathcal{T}} V(T)| + |Z|.$$

On the other hand, Condition 3 says that the number of tree components of $\mathcal{T}$ neighboring $Z$ is at least $2|Z|$. Therefore, we have $|Z| \leq |X \cap \bigcup_{T \in \mathcal{T}} V(T)|$. Take a set

$$X^* := \left(X \setminus \bigcup_{T \in \mathcal{T}} V(T)\right) \cup Z$$

and observe that $|X^*| \leq |X|$. It remains to verify that $X^*$ is indeed a feedback vertex set of $G$. Note that any cycle of $G - X^*$ must traverse some tree component $T$ of $\mathcal{T}$. As the neighborhood of $T$ are in $\{x\} \cup S$ and $S \cup X^*$, this means that $C$ is a cycle fully contained in $\{x\} \cup V(T)$. This is impossible

Now we argue that $X^*$ is a feedback vertex set of $G'$. If there is a cycle $C$ in $G' - X^*$, then it must be an $x$-cycle. Moreover, $C$ must contain an edge $G'$ which was not present in $G$. Such edges are between $x$ and $S$, and thus $C$ must traverse a vertex of $S$, contradicting that $C$ is a cycle in $G' - X^*$ and $X^*$ contain $S$. This is impossible due to Condition 2, which subsumes that $G[\{x\} \cup V(T)]$ is a tree. □

The last piece is Lemma 3.5.9 which states that if $deg(x) \geq 8k$ and the latter case of Corollary 3.5.1 holds, then Reduction Rule 6 is applicable. For this, we need two technical lemmata.

**Lemma 3.5.7.** *Let $H$ be a bipartite graph with a bipartition $S \uplus T$. In polynomial time, one can find a set $Z \subseteq S$ with $|N(Z)| < 2|Z|$ if such $Z$ exists.*

**Proof:** Copy each vertex $v$ of $S$, say $v'$ and make it adjacent with $N(v)$. The new graph $H'$ is a bipartite graph as well. Use König Theorem (Lecture Note #3 on Kernelization Part I) to compute a maximum matching $M$ of $H'$. If $M$ saturates $S \cup S'$, we know that $|N(Z)| \geq |Z|$ for every $Z \subseteq S \cup S'$. In particular, this holds when $Z$ takes both vertices of a pair $\{v, v'\}$. This means that $|N(Z)| \geq 2|Z|$ for every $Z \subseteq S$ in $H$. □

**Lemma 3.5.8.** *Let $H$ be a bipartite graph with a bipartition $S \uplus T$ satisfying*

- *every vertex of $T$ is adjacent with a vertex of $S$,*
- $|T| \geq 2|S|$.

*Then in polynomial time one can find vertex sets $S' \subseteq S$ and $T' \subseteq T$ such that*

- $N(T') = S'$, *and*
- $|N(Z)| \geq 2|Z|$ *for every $Z \subseteq S'$.*

**Proof:** The proof uses the same argument as in the proof of Theorem 4 in Lecture Note #3 on Kernelization Part I. □

**Lemma 3.5.9.** *Let $(G, k)$ be an instance irreducible with respect to Reduction 1-4. Let $x$ be a vertex of degree at least $8x$ and assume that there is no $x$-flower of order at least $k + 1$. Then in polynomial time, one can do one of the following:*

- *correctly decide that $(G, k)$ is a* NO*-instance, or*
- *return a vertex set $S$ meeting the condition of Reduction Rule 6.*

**Proof:** By Corollary 3.5.1, one can find a vertex set $S_x \subseteq V - x$ of size at most $2k$ hitting all $x$-cycles in polynomial time.

If $x$ have more than $3k$ incident edges whose other endpoints are in $S_x$, then there are double edges between $x$ and at least $k + 1$ vertices of $S_x$. They form an $x$-flower of order at least $k + 1$, a contradiction.

Therefore, $x$ and $S_x$ are connected by at most $3k$ edges. Note that $x$ is connected with any connected component $C$ of $G - S_x - x$ by at most one edge; otherwise, $x$ and $C$ with two edges between them create a cycle avoiding $S_x$. So any $C \in \mathcal{C}$ and $x$ is connected by at most one edge. Let $\mathcal{C}$ be the set of connected components of $G - S_x - x$ joined by a single edge with $x$. There are at least $5k$ members in $\mathcal{C}$ because any

edge incident with $x$ is either incident with $S_x$ (and there are at most $3k$ such edges) or incident with some member of $\mathcal{C}$, and due to the assumption $deg(x) \geq 8k$.

There are two cases.

1. If there are at least $k+1$ connected components in $\mathcal{C}$ containing a cycle, then $(G, k)$ is a NO-instance.
2. Otherwise, the set $\mathcal{T} \subseteq \mathcal{C}$ of connected components of $G - S_x - x$ each of which forming a tree has at least $4k$ members.

In the first case, we're done. In the second case, note that $|\mathcal{T}| \geq 2|S|$. Moreover, every connected component $C$ of $\mathcal{T}$ is adjacent with at least one vertex of $S$; if not, $C$ is a tree and a leaf of the tree $C$ has degree at most two in $G$, thus Reduction Rule 2 or 4 is applicable. Now one can apply the algorithm of Lemma 3.5.8 to the auxiliary graph $H = (S, \mathcal{T})$ and obtain sets $S' \subseteq S$ and $\mathcal{T}' \subseteq \mathcal{T}$ such that

- $S'$ is exactly the set of vertices of $S$ which are adjacent with tree components of $\mathcal{T}'_0$, and
- every $Z \subseteq S'$ is adjacent with at least $2|Z|$ tree components of $\mathcal{T}'_0$.

It remains to observe that $S'$ and $\mathcal{T}'$ satisfies the condition of Reduction Rule 6. This completes the proof. □

The kernelization algorithm is summarized as Algorithm **FVS-kernel**$(G, k)$.

**Algorithm 4** Kernelization for FEEDBACK VERTEX SET

1: **procedure FVS-kernel**$(G, k)$
2: **if** $k < 0$ **then return** NO
3: **if** One of Reduction Rules 1-4 is applicable with output $(G', k')$ **then return FVS-kernel**$(G', k')$
▷ Now, $G$ is loopless and every vertex has degree at least 3
4: **if** $|V| < 8k^2 + k$ **then return** $(G, k)$.
5: **else if** $deg(v) < 8k$ for every $v \in V$ **then return** NO.
6: **else**
7: Pick a vertex $x$ of degree at least $8k$.
8: **if** there is an $x$-flower of order $k+1$ **then** return **FVS-kernel**$(G - x, k-1)$
9: **else**
10: Let $S_x \subseteq V - x$ be a hitting set of all $x$-flowers of size at most $2k$.
11: ▷ $x$ is joined with $S_x$ with more than $3k$ edges.
12: Let $\mathcal{C}$ be the connected components of $G - S_x - x$ joined with $x$.
13: ▷ $|\mathcal{C}| \geq 5k$.
14: **if** there are more than $k$ components of $\mathcal{C}$ containing a cycle **then** return NO
15: Let $\mathcal{T} \subseteq \mathcal{C}$ be the tree components of $G - S_x - x$ joined with $x$ by a single edge.
16: ▷ $|\mathcal{T}| \geq 4k$.
17: Let $\mathcal{T}' \subseteq \mathcal{T}$ and $S' \subseteq S$ as described in Lemma 3.5.8.
18: Let $G'$ be the output of Reduction Rule 6. **return FVS-kernel**$(G', k)$

**Correctness of FVS-kernel**$(G, k)$.The equivalence of $(G, k)$ and the output instance at lines 2, **??** is trivial. Line 3 is correct due to the correctness of Reduction Rules 1-4. The output at line 4 is correct due to Lemma 3.5.3. The correctness of Reduction Rule 5 ensures that line 8 is correct. At line 10, the existence of such $S_x$ is given by Lemma 3.5.1.

At line 15, the number of components in $\mathcal{T}$ is at least $4k$. Indeed $x$ can have at most $3k$ edges incident with

$S_x$ since otherwise, there is an $x$-flower (consisting of 2-cycles) of order $k+1$. All other edges incident with $x$ must have the other endpoint in one of the members of $\mathcal{C}$, and thus we have $|\mathcal{C}| \geq 5k$. As there are at most $k$ connected components of $\mathcal{C}$ containing a cycle, it follows that $|\mathcal{T}| \geq 4k$.

Therefore, the second condition of Lemma 3.5.8 are satisfied by $S_x$ and $\mathcal{T}$ in the bipartite graph obtained by contracting each connected component of $\mathcal{T}$ to a single vertex (and ignoring all edges between vertices of $S_x$). The first condition of Lemma 3.5.8 is satisfied, note that each member $T \in \mathcal{T}$ has exactly one edge connecting $T$ and $x$ because $S_x$ is a hitting set of all $x$-cycles. If $T$ is not adjacent with $S_x$, then $x+T$ is a tree and one can find a leaf and Reduction Rule 2 is applicable. Hence, both conditions of Lemma 3.5.8 are satisfied, and one can obtain in polynomial time $S'_x \subseteq S_x$ and $\mathcal{T}' \subseteq \mathcal{T}$ which meet Condition 3 of Reduction Rule 6 (line 17). Condition 1 of Reduction Rule 6 is met by $x, S'_x$ and $\mathcal{T}'$ clearly. To verify that $\mathcal{T}'$ is a set of connected components of $G - S'_x - x$, it suffices to recall that the neighboring vertices of $\mathcal{T}'$ in $S_x$ are precisely $S'_x$ by Lemma 3.5.8. This, together with the fact that each $T/in\mathcal{T}$ is adjacent with at least one vertex of $S_x$, also means that each connected component of $\mathcal{T}'$ is adjacent with at least one vertex of $S'_x$. The correctness of line 18 follows from Lemma 3.5.6.

That the algorithm outputs a kernel on at most $8k^2 + k$ vertices on polynomial time is straightforward, left to the readers.

## 3.6 Remarks

In Dell and van Melkebeek (2010), it was proved that for both VERTEX COVER and FEEDBACK VERTEX SET, the quadratic-size kernels are essentially the best possible. For FEEDBACK VERTEX SET, the best known kernelization ends up with $2k^2 + k$ vertices and $4k^2$ edges. Whether one can obtain a kernel with $o(k^2)$ vertices for FEEDBACK VERTEX SET remains open. This result on quadratic-size lower bound for both VERTEX COVER and FEEDBACK VERTEX SET builds on the machinery for proving a lower bound of a kernel by Dell and van Melkebeek (2010).

- Holger Dell, Dieter van Melkebeek, Satisfiability allows no nontrivial sparsification unless the polynomial-time hierarchy collapses, STOC 2010, J. ACM 61(4): 23:1-23:27 (2014). Link

The kernelization for FEEDBACK VERTEX SET can be obtained in linear time.

- Yoichi Iwata, Linear-time Kernelization for Feedback Vertex Set, ICALP 2017: 68:1-68:14. Link

# Chapter 4

# Iterative Compression

## 4.1 $\mathcal{O}^*(5^k)$-time algorithm for FEEDBACK VERTEX SET

The iterative compression technique solves a parameterized problem P by iteratively solving a compression version of P. This is where the name comes from. We study this technique with an exemplary algorithm for FEEDBACK VERTEX SET. Consider the following compression version of FEEDBACK VERTEX SET.

COMPRESSION FVS
**Instance:** a graph $G = (V, E)$, a feedback vertex set $X \subseteq V$.
**Parameter:** $|X|$
**Question:** Does $G$ have a feedback vertex set $Y$ such that $|Y| < |X|$?

Suppose that you're given a fvs $X_i$ of size at most $k$ for a graph $G_i$. If $G_i$ expands to a slightly bigger graph $G_{i+1}$, where $G_{i+1}$ contains precisely one more vertex $v_{i+1}$ on top of $G_i$, then we know that $X_i \cup \{v_{i+1}\}$ is again a fvs of $G_{i+1}$. But its size might exceed our allowed budget $k$ (if not, $X_i \cup \{v_{i+1}\}$ is a trivial solution to COMPRESSION FVS). Our goal is to look for an alternative fvs of $G_{i+1}$ of size at most $k$, that is, a fvs whose size is strictly smaller than the given fvs. This extra information of a fvs $X_i \cup \{v_{i+1}\}$ of small size at hand, albeit a bit exceeding our budget, makes the task of algorithm design way easier.

**Lemma 4.1.1.** *If there is an algorithm $\mathcal{A}$ of* COMPRESSION FVS *running in time $c^{|X|} \cdot n^d$, then there is an algorithm for* FEEDBACK VERTEX SET *running in time $c^k \cdot n^{d+1}$.*

**Proof:** Let $v_1, \ldots, v_n$ be the vertices of $G$. For each $1 \leq i \leq n$, we define $G_i = G[\{v_1, \ldots, v_i\}]$, that is, $G_i$ is the subgraph of $G$ induced by the first $i$ vertices. Suppose that $X_i$ is a fvs of $G_i$ of size at most $k$. Such $X_i$ exists for $i$ up to $k$. For $i + 1 \leq n$, note that $X_i \cup \{v_{i+1}\}$ is a fvs of $G_{i+1}$ and $(G_{i+1}, X_i \cup \{v_{i+1}\})$ is a legitimate instance to COMPRESSION FVS. Now we run the algorithm $\mathcal{A}$ on $(G_{i+1}, X_i \cup \{v_{i+1}\})$. If $\mathcal{A}$ returns a fvs $X_{i+1}$ of $G_{i+1}$ of size at most $k$, we can proceed to the next iteration for $i + 2$ or declare it as a fvs of $G$ in case $i + 1 = n$. On the other hand, if $\mathcal{A}$ returns NO, then this means that not only $G_{i+1}$ is a NO-instance but $G_n$ is a NO-instance as well: indeed, if $G_n$ has a feedback vertex set $X_n$ of size at most $k$, then $X_n \cap \{v_1, \ldots, v_{i+1}\}$ is a feedback vertex set of $G_{i+1}$ and its size is clearly at most $k$. Therefore, we can correctly return NO as an output of the algorithm. To see the running time, notice that the aforementioned algorithm executes $\mathcal{A}$ at most $n$ times. □

Thanks to Lemma 4.1.1, now we can focus on designing an efficient fpt-algorithm for COMPRESSION FVS[1]. We expect that designing an algorithm for COMPRESSION FVS would be easier than designing an algorithm for FEEDBACK VERTEX SET because the latter problem is at least as hard as the former. In fact, COMPRESSION FVS can be even further reduced to the following variant of FEEDBACK VERTEX SET.

DISJOINT FVS
**Instance:** a graph $G = (V, E)$, a feedback vertex set $\tilde{X} \subseteq V$, and an integer $k$.
**Question:** Does $G$ have a feedback vertex set $\tilde{Y}$ such that $|\tilde{Y}| < k$ and $\tilde{Y} \cap \tilde{X} = \emptyset$?
**Parameter:** $k + |cc(G[\tilde{X}])|$.

The basis of reducing[2] COMPRESSION FVS to DISJOINT FVS is to rewrite a feasible solution $Y$ as a disjoint union of two sets $I := Y \cap X$ and $\tilde{Y} := Y \setminus X$. Furthermore, if $|Y| < |X|$ then $|Y \setminus X| < |X \setminus Y|$. So, in order to find a solution $Y$ to COMPRESSION FVS, we can 'guess' $Y \cap X$ by enumerating all subsets $I$ of $X$, remove the guessed part $I$ from $G$, and then find a fvs $\tilde{Y}$ of $G - I$ such that $\tilde{Y}$ is disjoint from $X \setminus I$ and has strictly smaller size than $X \setminus I$.

**Algorithm 5** Algorithm for DISJOINT FVS

1: **procedure dfvs**$(G, \tilde{X}, k)$
2: Let $F = G[V \setminus \tilde{X}]$ and $\mathcal{C}$ be the set of connected components of $G[\tilde{X}]$.
3: Delete all vertices having degree at most 1 in $G$.
4: If $v \in F$ has degree 2 in $G$ and has one neighbor in $F$, bypass it (delete $v$ and connect its neighbors).
5: **if** $G$ is acyclic **then return** $\emptyset$.
6: **else if** $G[\tilde{X}]$ contains a cycle **then return** NO.
7: **else if** $G$ contains a cycle and $k = 0$ **then return** NO.
8: Choose a leaf $v$ of $F$. ▷ $k > 0$
9: **if** $v$ has two neighbors in a single components of $\mathcal{C}$ **then**
10: **return dfvs**$(G - v, \tilde{X}, k - 1) \cup \{v\}$
11: **else** ▷ $v$ has two neighbors belonging to distinct components of $\mathcal{C}$
12: **return dfvs**$(G - v, \tilde{X}, k - 1) \cup \{v\}$ or **dfvs**$(G, \tilde{X} \cup \{v\}, k)$.

**Lemma 4.1.2.** *The algorithm* **dfvs***, given an instance* $(G, \tilde{X}, k)$*, solves* DISJOINT FVS *correctly in time* $\mathcal{O}^*(2^{\mu(I)})$*, where* $\mu(I) = k + |cc(G[\tilde{X}])|$.

**Proof:** We omit the correctness proof (which is rather straightforward, see the textbook by Cygan et. al. for details). To analyze the running time, we introduce a measure $\mu(I)$ of an instance $I = (G, \tilde{X}, k)$ to DISJOINT FVS.

$$\mu(G, \tilde{X}, k) = k + |cc(G[\tilde{X}])|.$$

In Line 12, each branching decreases the measure $\mu$ by at least one. Indeed, $\mu(G - v, \tilde{X}, k - 1) = k - 1 + |cc(G[\tilde{X}])| = \mu(I) - 1$, and the measure decreases by one in the first branching. In the second branching,

[1]Instead of using iterative compression, we can obtain an approximate feedback vertex set of size at most $2k$ using a 2-approximation algorithm for FEEDBACK VERTEX SET and apply the algorithm for COMPRESSION FVS at most $k$ times. That is, starting from $X$, we obtain a smaller solution if possible and feed it to the next instance of COMPRESSION FVS. The running time will be $\mathcal{O}^*(c^{|X|} \cdot n^d \cdot k)$ in this case.

[2]Creating a connection between the two problems so that by solving instances of the latter, one can obtain a solution to the former.

recall that $v$ is adjacent with (at least) two distinct components of $G[\tilde{X}]$ and thus by adding $v$ to $\tilde{X}$, we decreases the number of connected components by at least one. That is, $\mu(G, \tilde{X} \cup \{v\}, k) \leq \mu(I) - 1$. From $\mu(I) \geq 1$, the depth (as the number of branching nodes where Line 12 is invoked) of a search tree algorithm is at most $\mu(I)$ and the running time follows. □

**Lemma 4.1.3.** *There is an algorithm $\mathcal{B}$ for* COMPRESSION FVS *running in time* $5^{|X|} \cdot n^d$.

**Proof:** Given an instance $(G, X)$ to COMPRESSION FVS, we create an instance $(G', \tilde{X}, k')$ to DISJOINT FVS for every $I \subsetneq X$ as follows:

$$G' = G - I, \tilde{X} = X \setminus I \text{ and } k' = |\tilde{X}| - 1.$$

The algorithm $\mathcal{B}$ on the input instance $(G, X)$ is described below.

**Algorithm 6** Algorithm for COMPRESSION FVS

```
1: procedure B(G, X)
2:     for all I ⊊ X do
3:         if dfvs(G', X̃, |X̃| − 1) ≠ NO then
4:             Let Ỹ = dfvs(G', X̃, |X̃| − 1) and return Ỹ ∪ I
5:     return NO
```

We first observe that if an instance $(G', \tilde{X}, |\tilde{X}| - 1)$ of DISJOINT FVS is a YES-instance at Line 3, then the output $\tilde{Y} \cup I$ is indeed a solution to $(G, X)$ for COMPRESSION FVS. Indeed, $|\tilde{Y}| \leq |\tilde{X}| - 1$ implies that $|\tilde{Y}| + |I| < |\tilde{X}| + |I| = |X|$. Moreover, $\tilde{Y} \cup I$ is a fvs of $G$ because of $G - (I \cup \tilde{Y}) = (G - I) - \tilde{Y} = G' - \tilde{Y}$; $G' - \tilde{Y}$ is acyclic as $\tilde{Y}$ is a fvs of $G'$.

Therefore, to see the correctness of the algorithm $\mathcal{B}$ we only need to settle the claim:

$$\text{if } \mathcal{B} \text{ returns NO, then } (G, X) \text{ is a NO-instance to COMPRESSION FVS.}$$

If $(G, X)$ is a YES-instance to COMPRESSION FVS, let $Y$ be a fvs of $G$ such that $|Y| < |X|$. Then for $I := Y \cap X$, the corresponding instance $(G', \tilde{X}, k')$ defined as $G' : G - I$, $\tilde{X} := X \setminus I$ and $k' := |\tilde{X}| - 1$, the vertex set $\tilde{Y} := Y \setminus I$ is a fvs of $G'$: indeed $G' - \tilde{Y} = G - I - \tilde{Y} = G - Y$ is acyclic due to the assumption that $Y$ is a fvs of $G$. Moreover, $|\tilde{Y}| + |I| = |Y| < |X| = |\tilde{X}| + |I|$ implies that $|\tilde{Y}| < |\tilde{X}|$. Clearly, $\tilde{Y}$ is disjoint from $\tilde{X}$. Therefore, $\tilde{Y}$ is a solution to $(G', \tilde{X}, |\tilde{X}| - 1)$ for DISJOINT FVS. In particular, there exists some $I^*$(not necessarily the same $I$) such that the corresponding instance of DISJOINT FVS is YES, and therefore the condition of Line 3 is satisfied. Accordingly, the output of $\mathcal{B}(G, X)$ is not NO. This proves the correctness of the algorithm $\mathcal{B}$.

Finally, we observe that for all $I \subsetneq X$ of size $i$, an instance $I$ to DISJOINT FVS with $\mu(I) = |X| - i - 1 + |cc(G[\tilde{X}])| \leq 2(|X| - i) - 1$ is created. For each such $I$, the algorithm **dfvs** runs in time $\mathcal{O}^*(2^{\mu(I)})$, thus in $\mathcal{O}^*(4^{|X|-i})$ time. Therefore, the algorithm $\mathcal{B}$ runs in time

$$\sum_{i=0}^{|X|-1} \binom{|X|}{i} \cdot 4^{|X|-i} \cdot n^d \leq (4+1)^{|X|} \cdot n^d.$$

□

# Chapter 5

# Randomized Algorithms

## 5.1 Simple randomized algorithms: Feedback Vertex Set

We present a randomized algorithm for Feedback Vertex Set running in time $O^*(4^k)$. We first apply reduction rules first.

**Reduction Rule 1:** If $u$ has has a loop in $G$, then delete $v$ and decrease $k$ by one.
**Reduction Rule 2:** If $u$ has degree at most 1 in $G$ (and does not have a loop), then delete $v$.
**Reduction Rule 3:** If $u$ has degree 2 with neighbors $v, w$ in $G$ (possibly $v = w$), by delete $u$ and add an edge $(v, w)$.

Notice that application of Reduction Rule 3 may create parallel edges and loops - cycles of length two and one. Hence, $G$ is a graph with parallel edges in the remainder of this subsection. The degree of a vertex is the number of incident edges, not the number of neighbors. Provided Reduction Rules 0-2 have been applied exhaustively, we can assume that $G$ has minimum degree at least three.

The key observation behind the randomized $O^*(4^k)$-algorithm is sparsity of a forest. Let $S$ be a feedback vertex set of $G$. Then $G - S$ have at most $|V(G) \setminus S| - 1$ edges, which accounts for at most two among the minimum degree 3 of the vertices in $V(G) \setminus S$. Hence, more than $|V(G)| - |S|$ edges are lying between $S$ and $V(G) \setminus S$. This is formalized in the lemma below.

**Lemma 5.1.1.** *Let $G$ be a graph with minimum degree three. Then for any feedback vertex set $S$, at least half of $E$ are incident with $S$.*

**Proof:** We let $F := V(G) \setminus S$. Let us denote by $E(S)$ the set of edges whose both endpoints are in $S$ and by $E(S, F)$ the set of edges which has precisely one endpoint in each of $S$ and $F$. Let us count the sum $\sum_{v \in F} deg(v)$ in a different way. The only edges that contribute to this sum are $E(S, F) \cup E(F)$. Observe that an edge of $E(S, F)$ counts precisely once in this sum, and an edge of $E(F)$ is counted twice. Therefore, with the minimum degree condition on $V$ it holds that

$$\sum_{v \in F} deg(v) = |E(S, F)| + 2|E(S)| \geq 3|F|$$

It follows that

$$|E(S, F)| \geq 3|F| - 2|E(F)| \geq 3(|E(F)| + 1) - 2|E(F)| > |E(F)|,$$

where the second inequality is due to the fact that the number of edges in a tree $T$ is at most the number of vertices of $T$ minus one. Observe that the edge set incident with $S$ is $E(S) \cup E(F, S)$. From

$$|E(S)| + |E(S, F)| = \frac{1}{2}(2|E(S)| + 2|E(S, F)|) > \frac{1}{2}(|E(S)| + |E(S, F)| + |E(F)|) = \frac{1}{2}|E|,$$

we know that at least half of the edge set $E$ is incident with $S$. This complete the proof. □

Thanks to Lemma 5.1.1, a randomly chosen edge $e$ is incident with a (prescribed) feedback vertex set $S$ with probability at least $\frac{1}{2}$. By again randomly choosing one endpoint of $e$, we choose one of $S$ with probability at least $\frac{1}{4}$.

**Algorithm 7** Algorithm for FEEDBACK VERTEX SET

```
1: procedure FVS(G, k)
2:     Apply Reduction Rules 2-3 exhaustively.                    ▷ k remains the same
3:     if G contains a cycle and k = 0 then return NO and terminate.
4:     else if k ≥ 0 and G is acyclic then return ∅.
5:     else if G has a loop at some vertex v then return FVS(G − v, k − 1) ∪ {v}.
6:     else                          ▷ G is loopless, has a cycle, k > 0 and deg(v) ≥ 3 for every v ∈ V.
7:         Pick an edge e uniformly at random.
8:         Pick an endpoint of e uniformly at random. Let v be the chosen vertex.
9:         return FVS(G − v, k − 1) ∪ {v}.
```

**Lemma 5.1.2.** *On an input instance $(G, k)$ to* FEEDBACK VERTEX SET*, the procedure* $\mathsf{FVS}(G, k)$

*(i) runs in polynomial time,*

*(ii) outputs either* NO *or a feedback vertex set of $G$ of size at most $k$,*

*(iii) outputs a (feedback) vertex set of size at most $k$ with probability at least $\frac{1}{4^k}$ if $(G, k)$ is* YES.

**Proof:** The running time is straightforward. Before proceeding with the proof of (ii)-(iii), we point out that any input $(G', k')$ to the procedure FVS for the subsequent calls incurred by $\mathsf{FVS}(G, k)$ is a legitimate instance of FEEDBACK VERTEX SET: that is, $k' \geq 0$. Indeed, we decrease the parameter $k$ by one every time we make a call to FVS, and when $k = 0$ an output is returned at Lines 3-4, which means no subsequent call is made.

We prove (ii) by induction on $k$. It suffices to prove that if $\mathsf{FVS}(G, k)$ returns a vertex set $S$, then $S$ is a feedback vertex set of $G$ of size at most $k$. When $k = 0$, the fact that some vertex set $S$ is returned means that $(G, k)$ does not satisfy the condition of Line 3 and thus $G$ is acyclic. Now that $(G, k)$ meets the condition of Line 4, we know that $S = \emptyset$. It is clear that $S = \emptyset$ is a feedback vertex set of an acyclic graph $G$ of size at most $k = 0$. Consider $k > 0$ and notice that $S$ is returned at either Line 5 or 9. Especially, this means that the output of $\mathsf{FVS}(G - v, k - 1)$ is $S \setminus v$. By induction hypothesis, $S \setminus v$ is a feedback vertex set of $G - v$ of size at most $k - 1$. Hence, $G - v - S \setminus v$ is acyclic and thus $S$ is a feedback vertex set of $G$. Clearly $|S| \leq k$. This proves (ii).

Now we prove (iii). Suppose that $(G, k)$ is a YES-instance. If $G$ is acyclic, then (iii) trivially holds with probability 1. In particular, $G$ is always acyclic when $k = 0$ due to the assumption that $(G, k)$ is a YES-instance. Therefore, we may assume that $G$ has a cycle and $k > 0$. We claim that the input $(G - v, k - 1)$ to a subsequent call at Line 5 or 9 is YES with probability at least $\frac{1}{4}$. If $G$ has a loop at $v$, then $v$ must be included in any solution, and $(G - v, k - 1)$ is again a YES-instance with probability 1. By induction hypothesis the call at Line 5 returns a feedback vertex set of size at most $k - 1$ with probability $\frac{1}{4^{k-1}}$ and the current call returns a solution with the same probability. If $G$ does not have a loop, then Line 8 chooses a vertex $v$ contained in a solution $S$ of size at most $k$ with probability $\frac{1}{4}$. Indeed, Lemma 5.1.1 implies that the edge $e$ chosen at Line 7 is in $E(S) \cup E(S, V \setminus S)$ with probability 0.5. In case $e \in E(S)$, the probability that a random endpoint $v$ of $e$ is in $S$ is 1. In case $e \in E(S, V \setminus S)$, the probability is 0.5. Therefore,

$$\begin{aligned}\mathsf{Pr}[v \in S] = \mathsf{Pr}[e \in E(S) \cup E(S, V \setminus S)] \times \mathsf{Pr}[v \in S | e \in E(S) \cup E(S, V \setminus S)] \\ \geq 0.5 \times 0.5\end{aligned}$$

Notice that when $v \in S$, then $S \setminus v$ is a feedback vertex set of size at most $k - 1$ of $G - v$. That is, $(G - v, k - 1)$ is a YES-instance. Therefore, the created instance $(G - v, k - 1)$ at Line 9 is a YES-intance with probability at least $\frac{1}{4}$, as claimed.

To finalize the proof of (iii), we recall that by induction hypothesis, $\mathsf{FVS}(G - v, k - 1)$ returns a vertex set with probability at least $\frac{1}{4^{k-1}}$ when $(G - v, k - 1)$ is YES. Now[1],

$$\begin{aligned}&\mathsf{Pr}[\mathsf{FVS}(G, k) \text{ returns a vertex set}] \\ &\qquad \geq \mathsf{Pr}[\mathsf{FVS}(G, k) \text{ returns a vertex set at Line 9}] \\ &\qquad = \mathsf{Pr}[(G - v, k - 1) \text{ is YES and } \mathsf{FVS}(G - v, k - 1) \text{ returns a vertex set}] \\ &\qquad \geq \mathsf{Pr}[(G - v, k - 1) \text{ is YES}] \\ &\qquad \times \mathsf{Pr}[\mathsf{FVS}(G - v, k - 1) \text{ returns a vertex set } | (G - v, k - 1) \text{ is YES}] \\ &\qquad \geq \frac{1}{4} \times \frac{1}{4^{k-1}} = \frac{1}{4^k}.\end{aligned}$$

This completes the proof. □

By repeating $\mathsf{FVS}(G, k)$ $4^k$ times, we obtain an algorithm summarized in the lemma below.

**Lemma 5.1.3.** *There is an algorithm running in time $O(4^k \cdot poly(n))$ time which, given an input $(G, k)$ to* FEEDBACK VERTEX SET*, outputs*

*(i)* NO *if $(G, k)$ is a* NO*-instance, and*

*(ii) outputs a solution of size at most $k$ with probability $1 - e^{-1}$ if one exists.*

**Proof:** The algorithm $\mathcal{A}$ works as follows: on the input instance $(G, k)$, we run the procedure $\mathsf{FVS}$ $4^k$ times. If a run of FEEDBACK VERTEX SET returns a vertex set $S$, then return $S$ as an output of $\mathcal{A}$. If all $4^k$ executions of $\mathsf{FVS}$ on $(G, k)$ returns NO, then $\mathcal{A}$ returns NO. Clearly the algorithm $\mathcal{A}$ runs in the claimed running time because $\mathsf{FVS}$ runs in polynomial time and $\mathcal{A}$ invokes $\mathsf{FVS}$ as a subroutine $4^k$ times. That $\mathcal{A}$

[1] In the inequality, all probabilities are conditional on that $(G, k)$ is YES. We assumed this at the beginning of the proof of (iii).

satisfies (i) follows immediately from Lemma 5.1.2. To see (ii), observe that the probability that $\mathcal{A}$ returns NO when $(G, k)$ is YES equals

$$\mathsf{Pr}[\mathsf{FVS}(G,k) \text{ returns NO while } (G,k) \text{ is YES}]^{4^k} \leq (1 - \frac{1}{4^k})^{4^k} \approx e^{-1} (\approx 0.36),$$

where the equality holds because each run of FVS is independent, and the second inequality holds due to Lemma 5.1.2. The property (ii) follows. □

We consider a parameterized version of the LONGEST PATH problem. The problem $k$-PATH is, give a graph $G$ and an integer $k$, to find a simple path on $k$ vertices, if one exists.

$k$-PATH
**Instance:** a graph $G = (V, E)$, a positive integer $k$.
**Question:** Does $G$ have a path on $k$ vertices?

This problem is NP-complete. We give a randomized FPT-algorithm for $k$-PATH. The underlying idea is to transform (in a randomized way) the given graph into a graph in which detecting a $k$-path becomes a simpler task. It is known that on a directed acyclic graph(DAG), finding a longest (directed) path can be solved in time $O(|E|)$ using dynamic programming. So, we shall transform $G$ into a DAG $\vec{G}$ so that a (directed) $k$-path in $\vec{G}$ corresponds to a $k$-path in $G$. A $k$-path in $G$ does not necessarily yield a $k$-path in $\vec{G}$. The hope is that if we perform the transformation sufficiently many times, but not too many times to stay within the running time bound of FPT-algorithm, we will hit on $\vec{G}$ which contains a $k$-path corresponding to one in $G$.

Let $\pi : V(G) \to [n]$ be a random permutation of $V(G)$. A DAG $\vec{G}_\pi$ can be defined from $\pi$: it has $V(G)$ as the vertex set, and

$(u, v)$ is an arc of $\vec{G}_\pi$ if and only if $\pi(u) < \pi(v)$.

Notice that if $G$ contains a $k$-path $P$, and $\pi$ happens to order the vertices of $P$ in an orderly manner (two possible ways), then $P$ can be detected by a longest path algorithm on $\vec{G}_\pi$. If $G$ does not contain a $k$-path, then no permutation $\pi$ will allow $\vec{G}_\pi$ to contain a $k$-path.

The probability that a random permutation $\pi$ turns a $k$-path into a directed $k$-path in $\vec{G}$ is $\frac{2}{k!}$. Therefore, the expected number of random permutations to hit a sucessful $\vec{G}$ is $\frac{k!}{2}$. For each random permutation $\pi$, we test[2] whether $\vec{G}_\pi$ contains a $k$-path in time $O(|E|)$. Therefore, the expected running time of to detect $k$-path in $G$, if $G$ contains one, is $O(k! \cdot |E|)$.

## 5.2 Color-coding technique: $k$-PATH

We can improve the running time of $k$-PATH from $O^*(k!)$ to $2^{O(k)}$ time using color coding introduced by Alon, Yuster and Zwick. Color coding is a technique to transform a problem of detecting an object in a graph into a problem of colored object in a colored graphs, which is hopefully an easier task. In color coding for the problem $k$-PATH, we randomly color the vertices of $G$ with $k$ colors and the hope is that in the colored graph, a $k$-path becomes colorful. We say that a path in a colored graph is colorful if all vertices have distinct colors.

[2]The problem LONGEST PATH is in P on acyclic digraphs. First, we obtain a topological order of the vertex set of $\vec{G}_\pi$, then solve compute the length of a longest path to each vertex via dynamic programming over this ordering.

One pass of our color coding algorithm consists of two steps:

A. Color the vertices of $G$ with $\{1, \ldots, k\}$ uniformly at random. Let $c : V(G) \to [k]$ be the coloring.

B. Find a colorful $k$-path in $G$, if one exists. Otherwise, report that none was found.

**Step A.** The probability that step A. make a $k$-path $P$ colorful is

$$\frac{\#\text{of colorings in which } P \text{ becomes colorful}}{\#\text{ of all possible colorings}} = \frac{k!}{k^k} \approx \frac{1}{e^k}$$

So, the expected number of runs of A. before a $k$-path $P$ becomes colorful is $e^k$. Notice that any colorful $k$-path is also a $k$-path in $G$. Below, we provide an algorithm for B. running in time $O(2^k \cdot |E|)$.

**Step B: Detecting a colorful $k$-path.** Now we present an algorithm for detecting a colorful $k$-path given a vertex partition $V_1, \ldots, V_k$ of $V(G)$, where each $V_i$ are the vertices colored in $i$. We aim to set the values of indicator variables $P[C, u]$ for every color subset $C \subseteq \{1, \ldots, k\}$ and for every vertex $u \in V(G)$, so that

$P[C, u] = 1$ if there is a colorful path exactly consisting of colors in $C$ and ending in $u$.
$P[C, u] = 0$ otherwise.

At each $i$-th iteration over $i = 1, \ldots, k$, for all $u \in V(G)$ we set the value of $P[C, u]$ for $C \subseteq [k]$ with $|C| = i$ using dynamic programming. At $i = 1$, $P[C, u] = 1$ if and only if $C = \{c(u)\}$. At $i + 1$-th iteration, for each $u \in V(G)$ and $C \subseteq \{1, \ldots, k\}$ of size $i + 1$, we compute $P[C, u]$ as:

- $P[C, u] := 1$ if $c(u) \in C$ and there is $v \in N(u)$ such that $P[C \setminus c(u), v] = 1$.
- $P[C, u] := 0$ otherwise.

This recurrence computes $P[C, u]$ correctly indeed: if there is a colorful $i + 1$-path $Q$ using colors in $C$ and ending at $u$, then for a neighbor $v$ which is a neighbor of $u$ in $Q$, $Q - u$ is a colorful $i$-path using colors in $C \setminus \{c(u)\}$. Conversely, if for some neighbor $v$ of $u$ there is a colorful $i$-path using colors in $C \setminus \{c(u)\}$, such a path can be extended to a colorful $i + 1$-path by adding $u$. The new path uses colors in $C$ and ends at $u$. As the base case when $i = 1$ trivially holds, the correctness of the above recurrence follows.

After finishing $k$-th iteration, there is a vertex $u$ such that $P[\{1, \ldots, k\}, u] = 1$ if and only if there is a colorful $k$-path. This dynamic programming algorithm runs in time

$$O(\sum_{i=1}^{k} \binom{k}{i} \cdot |E|) = O(2^k \cdot |E|).$$

**Lemma 5.2.1.** *One can detect a colorful $k$-path in time $O(2^k \cdot |E|)$, if one exists.*

The following lemma summarizes the above analysis of Step A. and B.

**Lemma 5.2.2.** *One can detect a simple $k$-path in $O((2e)^k \cdot |E|)$ expected running time, if one exists.*

**Lemma 5.2.3.** *One can detect a simple $k$-path with probability at least $e^{-1}$ in time $O((2e)^k \cdot |E|)$, if one exists.*

**Proof:** The probability that a coloring fails to turn a $k$-path $P$ colorful is at most $1 - e^{-k}$. Therefore, the probability that all $e^k$ colorings (each, independent at random) reports no colorful $k$-path is at most

$$(1 - \frac{1}{e^k})^{e^k} \approx e^{-1}.$$

Together with Lemma 5.2.1, the running time follows. □

## 5.3 Random Separation: SUBGRAPH ISOMORPHISM

Another useful technique for designing a randomized algorithm is a random separation technique. Like color-coding, it is useful to design an algorithm to detect a small-sized substructure in a graph.

We exemplify this technique with the problem SUBGRAPH ISOMORPHISM: given an input graph $G$ and a pattern graph $H$ on $k$ vertices, the task is to find a copy of $H$ in $G$ or correctly decide that $G$ does not have $H$ as a subgraph. We take the parameter $k + d$, where $d$ is the maximum degree of $G$ and present a randomized algorithm that runs in time $2^{dk+O(k \log k)} \cdot n^{O(1)}$ which detect $H$ as a subgraph in $G$ with high probability, if exists one[3].

The intuitive idea is to color the edges of $G$ in blue or red so that the edges of $H$ are 'isolate' in this coloring, and thus this isolated copy of $H$ is easy to detect. Fix a subgraph $\tilde{H}$ of G which is isomorphic to $H$. A coloring $c : V(G) \to \{\text{red, blue}\}$ is successful if the next two conditions are satisfied:

1. all edges of $\tilde{H}$ is colored blue, and
2. all edges of $E(G) \setminus E(\tilde{H})$ incident with a vertex of $V(\tilde{H})$ is colored blue.

On how many edges does a successful coloring requests a specific color? All edges incident with a vertex of $\tilde{H}$ are requested to be either blue or red, depending on whether it belongs to $E(\tilde{H})$ or not. As there are at most $d \cdot V(\tilde{H}) = dk$ such edges, a random coloring $c$ is successful with probability at least $2^{-dk}$.

Now assume that the current coloring $c$ is successful. Consider the subgraph $G'$ of $G$ consisting of the blue edges. Let us call a connected component in $G'$ a blue component, and let $\mathcal{B}$ be the set of all blue components. Now we have narrow down which part of $G$ we need to match $H$. To begin with, any blue component having more than $k$ vertices has no chance of being $\tilde{H}$ (under a successful coloring).

Let $H_1, \ldots, H_p$ be the connected components of $H$ (possibly $p = 1$). We make a bipartite graph $W$ in which one part of the vertex bipartition is $\mathcal{H} = \{H_1, \ldots, H_p\}$ and the other part is $\mathcal{B}$, the set of all blue components. The bipartite graph $W$ has an edge between $H \in \mathcal{H}$ and $B \in \mathcal{B}$ if $H$ is isomorphic to $B$. As the isomorphism between $H$ and $B$ (on at most $k$ vertices each) can be tested in time $k! \cdot k^{O(1)} = 2^{O(k \log k)}$, the construction of $W$ can be done in time $2^{O(k \log k)} \cdot n$. It remains to observe that if $\tilde{H}$ exists and the current color is successful for $\tilde{H}$, this copy must be a disjoint union of $p$ blue components each of which is isomorphic to $H_1, \ldots, H_p$ respectively. We can decide whether such $p$ blue components exist by examining whether a maximum matching on $W$ exists saturating all vertices in $\mathcal{H}$. The latter problem is polynomial-time solvable.

[3]Mind that if you parameterize by $k$ only, then we cannot expect to have an fpt-algorithm: when $H$ is a clique on $k$ vertices, it is known that deciding if $G$ contains $H$ as a subgraph or not is known to be W[1]-hard (you will learn this notion in the next class) parameterized by $k$. This means that under a widely-accepted complexity assumption that $W[1] \neq FPT$, SUBGRAPH ISOMORPHISM is unlikely to be fixed-parameter tractable with respect to $k$ only.

To summarize, if $G$ contains a copy of $H$ (say $\tilde{H}$), then with probability at least $2^{-dk}$ a random coloring is successful for $\tilde{H}$. Given a successful coloring, $\tilde{H}$ can be correctly retrieved from $G$ in time $2^{O(k \log k)} \cdot n^{O(1)}$. Let us call this procedure $\mathcal{A}$. The probability[4] that no copy of $H$ is detected after $2^{dk}$ repetitions of $\mathcal{A}$ while $G$ contains a copy of $H$ is at most

$$(1 - 2^{-dk})^{2^{dk}} = (1 - 2^{-dk})^{(-2^{dk}) \cdot (-1)} \approx e^{-1}$$

that is, with constant probability a copy of $H$ is detected after $2^{dk}$ runs of $\mathcal{A}$. This constant success probability can be boosted to an arbitrarily high constant probability by repetitions.

## 5.4 Derandomization

The randomization technique of color-coding and random separation can be derandomized. For derandomizing color-coding, we use a family of functions called an $(n, k)$-perfect hash family. A family $\mathcal{F}$ of functions $f : [n] \to [k]$ is an $(n, k)$-perfect hash family if for every subset $S \subseteq [n]$ of size $k$, there exists $f \in \mathcal{F}$ such that $f$ assigns pairwise distinct values to the elements of $S$. Note that if $S$ is the fixed object that we want to find, then such $f$ will make $S$ colorful. A repetition of random colorings in color-coding technique can be replaced by an $(n, k)$-perfect hash family with almost negligible computational overhead, due to the following theorem.

**Theorem 5.4.1** (Naor, Schulman, Srinivasan 1995)**.** *For every $n, k \geq 1$, an $(n, k)$-perfect hash family of size $e^{k+o(k)} \cdot \log n$ can be constructed in time $e^{k+o(k)} \cdot n \log n$.*

For random separation, we use a different method. An $(n, k)$-universal set $\mathcal{U}$ is a family of subsets of $[n]$ such that for any $S \subseteq [n]$ of size $k$, all possible subsets of $S$ appear in the projection of $\mathcal{U}$ on $S$, that is, $\{S \cap A : A \in \mathcal{U}\} = 2^S$. In our application to SUBGRAPH ISOMORPHISM on graphs with maximum degree at most $d$, $S$ will correspond to the set of edges incident with a vertex of $V(\tilde{H})$, whose size is at most $2^{dk}$, and with a successful coloring we are looking for a partition of these edges into edges in $\tilde{H}$ and the rest. Now repeated random colorings can be replaced by trying the colorings in $\mathcal{U}$, interpreting a set $A \in \mathcal{U}$ as blue edges. This strategy works with little overhead because of the following theorem

**Theorem 5.4.2** (Naor, Schulman, Srinivasan 1995)**.** *For every $n, k \geq 1$, an $(n, k)$-universal set of size $2^{k+o(k)} \cdot \log n$ can be constructed in time $2^{k+o(k)} \cdot n \log n$.*

[4]We use the fact the natural log base $e$ equals $\lim_{x \to 0}(1 + x)^{\frac{1}{x}}$.

# Chapter 6

# Tree Decomposition

## 6.1 Tree decomposition: definition and basic properties

**Definition 6.1.1.** *Let $G$ be a graph. A tree decomposition of $G$ is a pair $(T, \chi)$, where $T$ is a tree and $\chi : V(T) \to 2^{V(G)}$ is a mapping associating each tree node $t$ to a vertex subset $\chi(t) \subseteq V(G)$ called a bag, which satisfies the following conditions.*

- **Vertex Coverage.** $\bigcup_{t \in V(T)} \chi(t) = V(G)$.
- **Edge Coverage.** *For every edge $e = uv$ of $G$, there is a tree node $t$ of $T$ such that $u, v \in \chi(t)$.*
- **Connectivity.** *For every vertex $v$ of $G$, the nodes of $T$ whose bags contain $v$ is connected in $T$. In other words, the set $\{t \in V(T) : v \in \chi(t)\}$ is a subtree of $T$ for each $v$ of $G$.*

*The width of a tree decomposition $(T, \chi)$, written as $\mathsf{width}(T, \chi)$, equals $\max_{t \in V(T)} |\chi(t)| - 1$. The treewidth of $G$, denoted as $\mathsf{tw}(G)$ is*

$$\mathsf{tw}(G) := \min_{(T,\chi)} \mathsf{width}(T, \chi),$$

*where $(T, \chi)$ is taken over all tree decompositions of $G$.*

**Some graph terminology.** We set up some terminology to be used later. We follow the standard graph terminology, e.g. as in Diestel [**?** ].

A tree decomposition $(T, \chi)$ of $G$ is said to be redundant if there are two adjacent nodes $t, t'$ of $T$ such that $\chi(t) \subseteq \chi(t')$ or $\chi(t') \subseteq \chi(t)$ holds, non-redundant otherwise. A rooted tree decomposition is a tree decomposition $(T, \chi)$ where $T$ is a rooted tree. When $T$ is rooted, one can talk about a child/parent/ancestor/descendant of a tree node.

Let $A, B \subseteq V(G)$ be vertex subsets of $G$; note that they may intersect. An $(A, B)$-path in $G$ is a path with one endpoint in $A$ and another endpoint in $B$. If there is a vertex $v$ in the intersection $A \cap B \neq \emptyset$, then $v$ itself forms a trivial $(A, B)$-path. We say that a vertex set $X \subseteq V(G)$ separates $A$ and $B$, or say that $X$ is an $(A, B)$-separator in $G$ if every $(A, B)$-path contains a vertex of $X$. Note that any $(A, B)$-separator must contain $A \cap B$ fully. In the special case when $A = \{a\}$ and $B = \{b\}$ for some $a, b \in V(G)$, we write $(a, b)$-separator instead of (rather cumbersome) $(\{a\}, \{b\})$-separator. A vertex set $X$ is called a separator of $G$ if there $a, b \in V(G) \setminus X$ such that $X$ is an $(a, b)$-separator.

A contraction of a graph $G$ by an edge $e = uv$ of $G$ is an operation on $G$ which (i) deletes $u$ and $v$ from $G$, (ii) adding a new vertex $z_e$, and (iii) connecting $z_e$ to each vertex of $N_G(u) \cup N_G(v)$ (by an edge). The graph obtained from $G$ by contracting an edge $e$ (of $G$) is written as $G/e$.

Here are some basic properties of a tree decomposition and treewidth of a graph.

**Lemma 6.1.1.** *Let $(T, \chi)$ be a tree decomposition of $G$. The following holds.*

1. *One can turn $(T, \chi)$ into a non-redundant tree decomposition in linear time without increasing the width.*
2. *For any clique $K$ of $G$, there is a tree node $t$ of $T$ such that $K \subseteq \chi(t)$.*
3. *Let $xy$ be an edge of $T$ and let $T_x$ (respectively $T_y$) be the subtree of $T - xy$ containing $x$ (respectively, with $y$). Then $\chi(x) \cap \chi(y)$ is a separator of $\bigcup_{t \in V(T_x)} \chi(t)$ and $\bigcup_{t \in V(T_y)} \chi(t)$. In particular, there is no edge between $\bigcup_{t \in V(T_x)} \chi(t) \setminus (\chi(x) \cap \chi(y))$ and $\bigcup_{t \in V(T_y)} \chi(t) \setminus (\chi(x) \cap \chi(y))$.*
4. *If $(T, \chi)$ is non-redundant, $T$ has at most $n$ nodes.*

**Proof:** We prove the last property[1]. Consider a mapping $\mathsf{top} : V(G) \to V(T)$ defined as

$$\mathsf{top}(v) := \text{the topmost node of } T \text{ containing } v.$$

In other words, top chooses the node of $T$ whose distance to root is the minimum among all nodes whose bags contain $v$.

We first argue that top is well-defined, i.e. there is a unique such node achieving the minimum distance to root. Suppose the contrary, i.e. there are two distinct nodes $x, y$ of $T$ with $v \in \chi(x)$ and $v \in \chi(y)$ neither of which is an ancestor of the other. By the connectivity property of tree decomposition, all the nodes on the $(x, y)$-path $P$ of $T$ contains $v$ in their bags. In particular, the least common ancestor of $x$ and $y$, which must be distinct from both $x$ and $y$, lies on $P$ and contains $v$. This contradicts the choice of $x$ and $y$.

Next, we claim that top is surjective, i.e. for every tree node $t$ there is a vertex $w$ of $G$ such that $\mathsf{top}(w) = t$. Indeed, the non-redundancy of $(T, \chi)$ subsumes $\chi(\mathsf{root}) \neq \emptyset$ and for every vertex $w \in \chi(\mathsf{root})$, $\mathsf{top}(w) = \mathsf{root}$. If $t$ is an internal node with a parent node $p(t)$, the set $\chi(t) \setminus \chi(p(t)) \neq \emptyset$ due to the non-redundancy of the given tree decomposition. For each $w \in \chi(t) \setminus \chi(p(t))$, the mapping top maps $w$ to $t$. This proves that top is surjective, implying $|V(T)| \leq |V(G)|$.

□

**Lemma 6.1.2.** *For any graph $G$, he following holds.*

1. $\mathsf{tw}(G) - 1 \leq \mathsf{tw}(G - v) \leq \mathsf{tw}(G)$ *for any vertex $v$ of $G$.*
2. $\mathsf{tw}(G) - 1 \leq \mathsf{tw}(G - e) \leq \mathsf{tw}(G)$ *for any edge $e$ of $G$.*
3. $\mathsf{tw}(G) - 1 \leq \mathsf{tw}(G/e) \leq \mathsf{tw}(G)$ *for any edge $e$ of $G$.*
4. *The number of edges in $G$ is at most $\mathsf{tw}(G) \cdot n$, where $n := |V(G)|$.*

**Lemma 6.1.3.** *A graph $G$ is a forest if and only if $\mathsf{tw}(G) \leq 1$.*

[1] Other properties were proved in the class.

**Nice tree decomposition.** A nice tree decomposition is a tree decomposition which is tailored to ease the design and description of a dynamic programming algorithm over a tree decomposition.

**Definition 6.1.2.** *Let $G$ be a graph. A nice tree decomposition $(T, \chi)$ of $G$ is a rooted tree decomposition such that each tree node $t$ of $T$ falls into one of the next four types and satisfies the corresponding property.*

- **Leaf node.** *$t$ is a leaf of $T$.*
- **Introduce node.** *$t$ has exactly one child, say $t'$, in $T$ and it holds that $\chi(t) = \chi(t') \cup \{v\}$ for some vertex $v$ of $G$.*
- **Forget node.** *$t$ has exactly one child, say $t'$, in $T$ and it holds that $\chi(t) = \chi(t') \setminus \{v\}$ for some vertex $v$ of $G$.*
- **Join node.** *$t$ has exactly two children, say $t_1$ and $t_2$, in $T$ and it holds that $\chi(t) = \chi(t_1) = \chi(t_2)$.*

**Lemma 6.1.4.** *Let $(T, \chi)$ be a tree decomposition of $G$ of width $w$. In linear time, one can obtain a nice tree decomposition $(T', \chi')$ of $G$ of width $w$. Moreover,*

- *one can further impose the condition that all leaf nodes and the root of $T'$ has empty bags, and*
- *the number of nodes in $T'$ is at most $4wn$, where $n := |V(G)|$.*

Let us fix some notations for a rooted tree decomposition $(T, \chi)$ of $G$. The root of $T$ is written as $\mathsf{root}$. For a tree node $t$ of $T$, $T_t$ denotes the subtree of $T$ rooted at $t$; that is, the set of nodes in $T_t$ is precisely the set of all tree nodes $x$ such that the (unique) path between $x$ and $\mathsf{root}$ in $T$ intersects $t$. We define

$$V_t := \bigcup_{b \in V(T_t)} \chi(b), \qquad G_t := G[V_t].$$

A useful property of a nice tree decomposition for designing DP algorithm is that the vertex $v$ introduced in an introduce node has all its neighbors in $G_t$ in the bag containing $v$. This property is a simple corollary of (iii) in Lemma 6.1.1 applied for the specific setting of a nice tree decomposition.

**Lemma 6.1.5.** *Let $(T, \chi)$ be a nice tree decomposition of $G$. Let $t$ be an introduce node of $T$ with a child $t'$ and $v$ be the vertex of $G$ such that $\chi(t) = \chi(t') \cup \{v\}$. Then $N_{G_t}(v) \subseteq \chi(t)$.*

A similar consequence of Lemma 6.1.1 to join node is stated in the next lemma.

**Lemma 6.1.6.** *Let $(T, \chi)$ be a nice tree decomposition of $G$. Let $t$ be a join node of $T$ with two children $t_1$ and $t_2$. Then $V_{t_1} \cap V_{t_2} = \chi(t)$.*

## 6.2 Dynamic programming over tree decomposition

Recall that $I \subseteq V(G)$ is an independent set of $G$ if no two vertices of $I$ are adjacent.

MAXIMUM INDEPENDENT SET
**Instance:** a graph $G = (V, E)$.
**Goal:** Find a maximum size independent set of $G$.

MAXIMUM INDEPENDENT SET is one of the exemplary NP-hard problems. On the other hand, it can be solved in polynomial time (in fact, $O(n)$-time) via a bottom-up dynamic programming algorithm when the input graphs are restricted to trees. Even more generally, when a graph is 'close to being a tree', a dynamic programming algorithm can be designed in a similar fashion. 'Close to being a tree' can be defined in various ways. Arguably the most prominent way is by means of treewidth. We often talk about 'graphs of small (constant, bounded) treewidth'. This is an informal way of referring to a graph class $\mathcal{C}$, i.e. a collection of graphs, with a universal constant $w$ such that $\mathsf{tw}(G) \leq w$ for all graph $G \in \mathcal{C}$. Recall that trees have tree width at most 1. Graphs of small treewidth, a.k.a. a class of graphs all of which have treewidth at most $w$ for some fixed $w$, has many powerful properties that trees have. Admitting a linear[2] time algorithm for well-known NP-hard problems such as MAXIMUM INDEPENDENT SET is one of them.

We show that MAXIMUM INDEPENDENT SET can be solved in time $O(2^w \cdot n)$ when the input is additionally given with a tree decomposition $(T, \chi)$ of $G$ of width at most $w$.

**Theorem 6.2.1.** *There is an algorithm which, given as input $G$ and a (non-redundant) tree decomposition $(T, \chi)$ of width at most $w$, finds a maximum size independent set of $G$ in time $O(w2^w \cdot n)$.*

To begin with, we assume that the given tree decomposition $(T, \chi)$ is a nice tree decomposition of width at most $w$ and all leaf bags and the root bag are $\emptyset$. This assumption can be achieved in time $O(n)$ by applying the algorithm of Lemma 6.1.4.

We define $\mathsf{IND}_t[S]$, for each tree node $t$ and each subset $S \subseteq \chi(t)$, as the following value:

$$(\star) \quad \mathsf{IND}_t[S] = \begin{cases} \text{the size of a maximum independent set } I \text{ of } G_t \text{ such that } I \cap \chi(t) = S, \text{ if one exists} \\ \bot \qquad \text{if no such } I \text{ exists.} \end{cases}$$

Note that $\mathsf{IND}_{\mathsf{root}}[\emptyset]$ is precisely the size of a maximum independent set of $G = G_{\mathsf{root}}$. Therefore, if we can compute the values of the table $\mathsf{IND}_t$ (over all subsets $S \subseteq \chi(t)$) for each tree node $t$, we have solved MAXIMUM INDEPENDENT SET. The plan is the following.

A. (Initialization and base case of induction) At each leaf $t$, compute a DP table $\mathsf{IND}_t$.

B. (Recursive formula) For a forget/introduce/join node $t$, define a recursive formula to compute $\mathsf{IND}_t$ from the tables of its children

C. (Runtime analysis) Argue that computing the value of $\mathsf{IND}_t(S)$ takes constant time for each $S \subseteq \chi(t)$.

Notice that the current DP algorithm only computes the size of a maximum independent set, and does not return an actual set which achieves the maximum size. However, once the value of $\mathsf{IND}_{\mathsf{root}}(\emptyset)$ has been computed, one can backtrack the entries which leads to the size of a maximum independent set along the tree decomposition in a top-to-bottom manner. This will be discussed at the end of the section.

**Initialization.** Let $t$ be a leaf node of $T$. For every $S \subseteq \chi(t)$, we claim

$$\mathsf{IND}_t(S) = \begin{cases} |S| & \text{if } S \text{ is an independent set,} \\ \bot & \text{otherwise.} \end{cases}$$

[2]The running time of a typical DP algorithms is $f(w) \cdot n$. When $w$ is deemed as a fixed constant, such an algorithm runs in linear time (albeit there is quite a large hidden constant in the runtime function).

Since $V_t = \chi(t)$, a max independent $I$ of $G_t$ such that $I \cap \chi(t) = S$ is precisely $S$ itself, if $S$ is independent, and no such $I$ exists if $S$ is not independent. Therefore the above equation holds.

**Forget node** $t$**.** Let $t'$ be a child of $t$ and $\chi(t) = \chi(t') \setminus \{v\}$. We claim that the next recursive formula holds for each $S \subseteq \chi(t)$.

$$\mathsf{IND}_t(S) = \max\{\mathsf{IND}_{t'}(S), \mathsf{IND}_{t'}(S \cup \{v\})\},$$

Here, we define $\perp < 0$.

To see $\mathsf{IND}_t(S) \leq \max\{\mathsf{IND}_{t'}(S), \mathsf{IND}_{t'}(S \cup \{v\})\}$ observe that for any (maximum) independent set $I$ of $G_t$ with $I \cap \chi(t) = S$, either $I \cap \chi(t') = S$ or $I \cap \chi(t') = S \cup \{v\}$ holds. In particular, this means at least one of the inequalities $\mathsf{IND}(S) \leq \mathsf{IND}_{t'}(S)$ and $\mathsf{IND}(S) \leq \mathsf{IND}_{t'}(S \cup \{v\}$ holds. This proves the claimed inequality.

To establish the inequality $\mathsf{IND}_t(S) \geq \max\{\mathsf{IND}_{t'}(S), \mathsf{IND}_{t'}(S \cup \{v\})\}$, suppose that there is an independent set $I$ in $G_t$ with $I \cap \chi(t') = S$ and take $I$ as such set of maximum size. Note that $I \cap \chi(t) = S$, and hence $\mathsf{IND}_t(S) \geq |I| = \mathsf{IND}_{t'}(S)$. Similarly, if there is an independent set $J$ in $G_t$ with $J \cap \chi(t') = S \cup \{v\}$, take $J$ as such set of maximum size. Note that $J \cap \chi(t) = J \cap (\chi(t') \setminus \{v\}) = J \cap \chi(t') \setminus \{v\} = S$, which implies $\mathsf{IND}_t(S) \geq |J| = \mathsf{IND}_{t'}(S \cup \{v\})$. Therefore, we have $\mathsf{IND}_t(S) \geq \max\{\mathsf{IND}_{t'}(S), \mathsf{IND}_{t'}(S \cup \{v\})\}$.

**Introduce node** $t$**.** Let $t'$ be a child of $t$ and $\chi(t) = \chi(t') \cup \{v\}$. We claim that the next recursive formula holds for each $S \subseteq \chi(t)$.

$$\mathsf{IND}_t(S) = \begin{cases} \mathsf{IND}_{t'}(S) & \text{if } v \notin S, \\ \perp & \text{if } v \in S \text{ and } S \text{ is not an independent set,} \\ \mathsf{IND}_{t'}(S \setminus \{v\}) + 1 & \text{if } v \in S \text{ and } S \text{ is an independent set.} \end{cases}$$

We prove the equation for each case of $S$. Note that $G_t = G_{t'}$.

If $S$ does not contain the newly introduced vertex $v$, then we have $S \subseteq \chi(t')$. From $G_t = G_{t'}$, the equation immediately holds. If $S$ is not an independent set, then it is clear that there is no independent set $I$ satisfying $I \cap \chi(t) = S$ and the equation correctly decides the value of $\mathsf{IND}_t(S)$.

Assume $v \in S$ and $S$ is an independent set and we want to settle the equation $\mathsf{IND}_t(S) = \mathsf{IND}_{t'}(S \setminus \{v\}) + 1$. Note that $\mathsf{IND}_t(S) \neq \perp$ in this case as $S$ itself is an independent set. For $\mathsf{IND}_t(S) \leq \mathsf{IND}_{t'}(S \setminus \{v\}) + 1$, for any maximum independent set $I$ of $G_t$ satisfying $I \cap \chi(t) = S$, $I \setminus \{v\}$ is an independent set of $G_{t'}$ satisfying $I \cap \chi(t') = S \setminus \{v\}$. Hence, we deduce

$$\mathsf{IND}_t(S) = |I| = |I \setminus \{v\}| + |\{v\}| \leq \mathsf{IND}_{t'}(S \setminus \{v\}) + 1.$$

Conversely, consider a maximum independent set $J$ of $G_{t'}$ satisfying $J \cap \chi(t') = S \setminus \{v\}$. The key observation here is that $J \cup \{v\}$ is an independent set of $G_t$. If this is not the case, i.e. $J \cup \{v\}$ has an edge (in $G_t$), then it must be incident with $v$ because $J$ is an independent set of $G_{t'}$, thus of $G_t$. Due to Lemma 6.1.5, it

follows that $N_{G_t}(v) \subseteq \chi(t)$, and $N_{G_t}(v) \cap J \subseteq \chi(t) \cap J = S \setminus \{v\}$. This contradicts the assumption that $S$ is an independent set, and we conclude that $J \cup \{v\}$ is an independent set of $G_t$. Now,

$$\mathsf{IND}_t(S) \geq |J \cup \{v\}| = |J| + 1 \geq \mathsf{IND}_{t'}(S \setminus \{v\}) + 1,$$

where the first inequality is due to $J \cup \{v\}$ is an independent set of $G_t$ satisfying $(J \cup \{v\}) \cap \chi(t) = S$.

**Join node** $t$**.** Let $t_1, t_2$ be the two children of $t$ with $\chi(t) = \chi(t_1) = \chi(t_2)$. We claim that the next recursive formula holds for each $S \subseteq \chi(t)$.

$$\mathsf{IND}_t(S) = \mathsf{IND}_{t_1}(S) + \mathsf{IND}_{t_2}(S) - |S|,$$

where $\perp + z$ is defined as $\perp$ for any $z \in \{\perp\} \cup \mathbb{N}$.

Let $I$ be a maximum independent set of $G_t$ satisfying $I \cap \chi(t) = S$. Note that $I_i := I \cap V_{t_i}$ is an independent set of $G_{t_i}$ satisfying $I \cap \chi(t_i) = S$ for both $i = 1, 2$. Because $V_{t_1} \cup V_{t_2} = V_t$, it holds that $I = I \cap V_t = I \cap (V_{t_1} \cup V_{t_2}) = (I \cap V_{t_1}) \cup (I \cap V_{t_2})$[3]. Therefore we can rewrite

$$\mathsf{IND}_t(S) = |I| = |I_1 \cup I_2| = |I_1| + |I_2| - |S| \leq \mathsf{IND}_{t_1}(S) + \mathsf{IND}_{t_2}(S) - |S|.$$

Here, the third equation holds because $I_1 \cap I_2 = (I \cap V_{t_1}) \cap (I \cap V_{t_2}) = I \cap (V_{t_1} \cap V_{t_2}) = I \cap \chi(t) = S$ by Lemma 6.1.6.

To see the opposite direction of the inequality, consider a maximum independent set $J_i$ of $G_{t_i}$ satisfying $J_i \cap \chi(t_i) = S$ for $i = 1, 2$. We claim that $J_1 \cup J_2$ is an independent set of $G_t$. Indeed, if this is not the case, then there is a vertex $v_i \in J_i \setminus S$ for $i = 1, 2$ such that $v_1 v_2$ is an edge of $G_t$. However, the existence of such an edge contradicts the property 3 of Lemma 6.1.1. This means that $J_1 \cup J_2$ is an independent set of $G_t$ which furthermore satisfies $(J_1 \cup J_2) \cap \chi(t) = S$. Therefore,

$$\mathsf{IND}_t(S) \geq |J_1 \cup J_2| = |J_1| + |J_2| - |J_1 \cap J_2| = \mathsf{IND}_{t_1}(S) + \mathsf{IND}_{t_2}(S) - |S|.$$

Here, the last equation is justified because $J_1 \cap J_2 \subseteq V_{t_1} \cap V_{t_2} = \chi(t)$ due to Lemma 6.1.6 and thus $J_1 \cap J_2 = (J_1 \cap J_2) \cap \chi(t) = (J_1 \cap \chi(t)) \cap (J_2 \cap \chi(t)) = S$.

**Recap of the algorithm, correctness and runtime.** Using the recursive formula described above, the algorithm simply computes the tables $\mathsf{IND}_t$ in a bottom-to-top manner. Computing the table $\mathsf{IND}_t$ for each leaf is straightforward from the equation as one can enumerate all subsets of $\chi(t)$ and check if there is an edge in the set under consideration. This takes $2^{w+1} \cdot poly(w)$-time. The correctness of this step, i.e. the computed table $\mathsf{IND}_t$ matches the definition of $\mathsf{IND}_t$ in $(\star)$, is trivial.

The algorithm then selects a lowermost internal (i.e. non-leaf) node $t$ for which the table $\mathsf{IND}_t$ is not computed yet, and compute $\mathsf{IND}_t$ using the recursive formula depending on the type of $t$. The computed table $\mathsf{IND}_t$ is correct, i.e. is as defined in $(\star)$, is shown above for each node type. The running time of computing $\mathsf{IND}_t$ for each $t$ is $O(2^{\chi(t)} \cdot poly(w))$. As the number of nodes in $T$ is $4w \cdot n$ by Lemma 6.1.4, the total running time of the algorithm is $O^*(2^w \cdot n)$, where the $O^*()$ notation hides the poly-logarithmic factor.

[3]Here we use the fact that set intersection is distributive over set union.

## 6.3 Courcelle's Theorem

The dynamic programming algorithm presented in Section 6.2 can be generalized to a much wider family of problems due to the celebrated theorem by Bruno Courcelle.

**Theorem 6.3.1** (Courcelle's Theorem [**?** ])**.** *There exists an algorithm which, upon an input consisting of a graph $G$, a tree-decomposition $(T, \chi)$ of $G$ of width at most $\omega$, and an $\mathrm{MSO}_2$-sentence $\varphi$, decides if $G \models \varphi$ in time $f(\omega, |\varphi|) \cdot n$.*

## 6.4 Algorithm for approximating treewidth

Reed [**?** ] proposed an elegant algorithm for approximating computing treewidth of a graph, which is the subject of this section.

**Theorem 6.4.1.** *There is an algorithm which, given a graph $G$ and a positive integer $w$, either outputs a tree decomposition of width at most $4w + 4$ or correctly concludes that $\mathsf{tw}(G) > w$ in time $O(27^w \cdot wnm)$.*

Let $G$ be a graph and $S \subseteq V(G)$ be a vertex subset. A vertex subset $X \subseteq V(G)$ is called a $\underline{\alpha\text{-balanced } S\text{-separator}}$ if for every connected component $C$ of $G - X$, we have $|C \cap S| \leq \alpha \cdot |S|$. When $S = V(G)$, an $\alpha$-balanced $S$-separator is simply called an $\alpha$-balanced separator. We will take $\alpha := \frac{1}{2}$ and simply refer to a balanced $S$-separator without mentioning the coefficient $\alpha$.

A $\underline{\text{separation}}$ of $G$ is a pair $(A, B)$ with $A \cup B = V(G)$ such that there is no edge between $A \setminus B$ and $B \setminus A$. The $\underline{\text{order}}$ of a separation $(A, B)$ is the size $|A \cap B|$. A $\underline{\alpha\text{-balanced } S\text{-separation}}$ is a separation $(A, B)$ which additionally satisfies $|(A \setminus B) \cap S| \leq \alpha \cdot |S|$ and $|(B \setminus A) \cap S| \leq \alpha \cdot |S|$.

We use three key lemmas. The proofs of Lemmata 6.4.1 and 6.4.2 can be found in the textbook by Cygan et al. [**?** ] as Lemma 7.19 and Lemma 7.20

**Lemma 6.4.1.** *Let $G$ be a graph of treewidth at most $w$ and let $S$ be an arbitrary vertex subset of $G$. Then there is a $\frac{1}{2}$-balanced $S$-separator of $G$ consisting of at most $w + 1$ vertices.*

**Lemma 6.4.2.** *Let $G$ be a graph and let $S \subseteq V(G)$. If $X \subseteq V(G)$ is $\frac{1}{2}$-balanced $S$-separator of $G$, then there is a $\frac{2}{3}$-balanced $S$-separation $(A, B)$ such that $A \cap B = X$.*

**Lemma 6.4.3.** *There is an algorithm which, given a graph $G$, a vertex set $S \subseteq V(G)$ and an integer $w \geq 1$, either finds a $\frac{2}{3}$-balanced $S$-separation of order $w+1$ or correctly concludes that there is no such separation of $G$ in time $O(3^{|S|} \cdot wm)$.*

**Proof:** Consider a tripartition of $S$ into $(S_A, S_X, S_B)$ and pick an arbitrary $(S_A, S_B)$-separator $Y$ of $G - S_X$. Then for a connected component $C$ of $G - (S_X \cup Y)$, observe that $C$ intersects at most one of $S_A$ and $S_B$ because $S_X \cup Y$ is a $(S_A, S_B)$-separator in $G$. Now, let $(A', B')$ be a bipartition of the connected components of $G - (S_X \cup Y)$ so that $\mathcal{A}'$ contains all connected components of $G - (S_X \cup Y)$ intersecting $S_A$ and $\mathcal{B}'$ contains the remaining components. Those connected components intersecting neither of $S_A$ and $S_B$ are put arbitrarily. Define $A := \bigcup_{C \in \mathcal{A}'} C \cup (S_X \cup Y)$ and $B := \bigcup_{C \in \mathcal{B}'} C \cup (S_X \cup Y)$, and note that $(A, B)$ is a separation with $A \cap B = (S_X \cup Y)$ such that $S \cap (A \setminus B) = S_A$, $S \cap (B \setminus A) = S_B$ and $S \cap (S_X \cup Y) = S_X$.

We summarize the above in the next claim.

**Claim 6.4.1.** *Let $(S_A, S_X, S_B)$ be a tripartition of $S$ and $Y$ be a $(S_A, S_B)$-separator of $G - S_X$. Then there is a separation $(A, B)$ of $G$ such that $A \cap B = S_X \cup Y$, $S \cap (A \setminus B) = S_A$, $S \cap (B \setminus A) = S_B$ and $S \cap (A \cap B) = S_X$. Moreover, one can construct such a separation in polynomial time.*

The algorithm for checking if $\frac{2}{3}$-balanced $S$-separation of order $w+1$ works as follows. We guess all possible tripartitions of $S$ into $(S_A, S_X, S_B)$ such that $|S_A| \leq \frac{2}{3} \cdot |S|$ and $|S_B| \leq \frac{2}{3} \cdot |S|$. For each guess $(S_A, S_X, S_B)$, we invoke max-flow min-cut algorithm to find a $(S_A, S_B)$-separator of $Y$ minimum size in $G - S_X$. If $|S_X \cup Y| \leq w+1$, then by Claim 6.4.1 we can obtain a $\frac{2}{3}$-balanced $S$-separation of order at most $w+1$. We output such a separation $(A, B)$.

Suppose for every guess $(S_A, S_X, S_B)$ and for every minimum-size $(S_A, S_B)$-separator of $Y$ in $G - S_X$ found with max-flow min-cut algorithm, we have $|S_X \cup Y| > w+1$. We argue that $G$ has no $\frac{2}{3}$-balanced $S$-separation of order at most $w+1$.

Indeed, if there is a $\frac{2}{3}$-balanced $S$-separation $(A, B)$ of order $w+1$, then let $X = A \cap B$. Let $S_A := S \cap (A \setminus B)$, $S_X := S \cap X$ and $S_B := S \cap (B \setminus A)$, and observe that $(S_A, S_X, S_B)$ forms a tripartition of $S$ with $|S_A| \leq \frac{2}{3} \cdot |S|$ and $|S_B| \leq \frac{2}{3} \cdot |S|$. Moreover, $X$ separates $S_A$ and $S_B$ in $G$, and thus $X \setminus S_X$ is an $(S_A, S_B)$-separator in $G - S_X$ of size at most $w + 1 - |S_X|$. Now, another $(S_A, S_B)$-separator $Z$ in $G - S_X$ could have been returned by the max-flow min-cut algorithm and notice that $|S_X \cup Z| \leq |S_X| + |Z| \leq |S_X| + (w + 1 - |S_X|) \leq w + 1$. This contradicts the assumption that no such separator was found for any considered tripartition of $S$.

The factor $3^{|S|}$ in the running time comes for the upper bound on the number of tripartitions on $S$. The polynomial part comes from the running time $O(f \cdot |E(G)|)$ of Ford-Fulkerson algorithm for solving max-flow min-cut when the capacities are integral and the flow value is at most $f$. □

The algorithm for treewidth is described below. Let $\mathcal{A}(G, S, w)$ be an algorithm as depicted in Lemma 6.4.3. To compute a tree decomposition of $G$, we execute the procedure **compute-tw** on $(G, \emptyset, w)$.

**Algorithm 8** Algorithm for computing treewidth

**procedure compute-tw**$(G, S, w)$
  **if** $|V(G)| \leq 4w + 5$ **then return** $(\{z\}, \{(z, V(G)\})$. ▷ Tree on a single node $z$ with $\chi(z) = V(G)$
  Expand $S$ up to size $3w + 4$ by adding arbitrary vertices of $V(G) \setminus S$.
  ▷ $|V(G)| > 4w + 5$ and $|S| = 3w + 4$.
  **if** $\mathcal{A}(G, S, w) =$ NO **then return** "tw $> w$."
  **else**
    Let $(A, B)$ be the separation returned by $\mathcal{A}(G, S, w)$ and let $X := A \cap B$.
    **if compute-tw**$(G[A], (A \cap S) \cup X, w)$="tw $> w$" **then return** "tw $> w$."
    **if compute-tw**$(G[B], (B \cap S) \cup X, w)$="tw $> w$" **then return** "tw $> w$."
    **else**
      Let $(T_A, \chi_A) =$ **compute-tw**$(G[A], (A \cap S) \cup X, w)$
      Let $(T_B, \chi_B) =$ **compute-tw**$(G[B], (B \cap S) \cup X, w)$
      Create a bag $Z := S \cup X$.
      $T$ is the tree obtained by creating a node $z$ and connecting $z$ with $\mathsf{root}(T_A)$ and $\mathsf{root}(T_B)$.
      Let $\chi$ be the 'canonical' mapping from $V(T)$ to $2^{V(G)}$ with $\chi(z) = Z$.
      **return** $(T, \chi)$

**Correctness.** To begin with, we argue that $|S|$ when **compute-tw**$(G, S, w)$ is called has size at most $3w + 3$

and the execution at line 3 is valid. We prove it by induction on the length of the path from the root call to the current recursive call. For the root call, this is trivially true because $S = \emptyset$. Assume that at the point **compute-tw**$(G, S, w)$ is called, it holds that $|S| \leq 3w + 3$. When $\mathcal{A}$ returns a separation $(A, B)$ on the instance $(G, S, w)$ after expanding $S$ up to size $3w + 4$, that $(A, B)$ is a $\frac{2}{3}$-balanced $S$-separation implies $|(A \cap S) \cup X| \leq |((A \setminus B) \cap S) \cup X| \leq (2w+2)+(w+1) \leq 3w+3$, as desired. Therefore, at line 11-12, the calls are made with the second argument having size at most $3w + 3$.

If **compute-tw**$(G, S, w)$ returns "tw $> w$", there is a triple $(G', S', w)$ for some induced subgraph $G'$ of $G$ and a vertex subset $S'$ of $G'$ such that $\mathcal{A}(G', S', w) =$ NO. In this case $\mathsf{tw}(G') > w$ since otherwise, by Lemmas 6.4.1 and 6.4.2, the procedure $\mathcal{A}$ on $(G', S', w)$ would have detected a balanced $S'$-separation of order at most $w + 1$. From $\mathsf{tw}(G) \geq \mathsf{tw}(G') > w$, the output of **compute-tw**$(G, S, w)$ is correct.

It remains to show that when **compute-tw**$(G, S, w)$ does not return "tw $> w$", that is, output some pair $(T, \chi)$, then it is a tree-decomposition of $G$ of width at most $4w+4$. We prove a stronger claim by induction on $|V(G)|$.

($\star$) If **compute-tw**$(G, S, w)$ does not return "tw $> w$", then it outputs a tree-decomposition of $G$ such that the width is at most $4w + 4$ and the root bag contains $S$.

When $|V(G)| \leq 4w + 5$, the output pair $(T, \chi)$, where $T$ consist of a single node, trivially satisfies ($\star$). Henceforth, we assume that $|V(G)| > 4w + 5$ and that **compute-tw**$(G, S, w)$ does not return "tw $> w$". Let $(A, B)$ be the output of $\mathcal{A}(G, S, w)$.

Note that $|(A \setminus B) \cap S| \leq 2w + 2$ and $|(B \setminus A) \cap S| \leq 2w + 2$ hold as $(A, B)$ is a $\frac{2}{3}$-balanced $S$-separation and $|S| = 3w + 4$ in line 4. Note that

$$\begin{aligned}|(B \setminus A) \cap S| &= |S| - |S \cap A| = |S| - |S \cap (A \setminus B)| - |S \cap (A \cap B)| \\ &\geq 3w + 4 - (2w + 2) - (w + 1) = 1.\end{aligned}$$

This implies $|A| < |V(G)|$ and $|B| < |V(G)|$. Assume that $(T_A, \chi_A)$ and $(T_B, \chi_B)$ be tree decompositions of $G[A]$ and $G[B]$ respectively satisfying ($\star$). Let $z_A$ and $z_B$ be the roots of the former and the latter tree decomposition respectively.

Let us check that $(T, \chi)$ is a tree decomposition of $G$. For any vertex $v$ or an edge $e$ of $G$ the vertex and edge coverage property hods for $v$ or $e$ by induction hypothesis; that is, for $V(G) = A \cup B$ without loss of generality we have $v \in A$ and the vertex coverage of the (sub)tree decomposition $(T_{z_A}, \chi|_{V(T_{z_A})}) = (T_A, \chi_A)$ for $v$ implies the vertex coverage of $(T, \chi)$ for $v$. For an edge $e$ of $G$, notice that either both endpoints of $e$ are in $A$ or both are in $B$ because there is no edge between $(A \setminus B, B \setminus A)$ and the same argument works to show the edge coverage of $e$ in $(T, \chi)$.

To verify the connectivity condition, we consider a few cases. When $w \notin S \cup X$, without loss of generality we may assume $w \in A \setminus B \setminus S$. Then the bags containing $w$ are only contained in the subtree decomposition of $(T, \chi)$ rooted at $z_A$ and the induction hypothesis ensure the connectivity of $w$. For $v \in A \cap B$, note that $A \cap B = X$ are contained in the root bags of both $(T_A, \chi_A)$ and $(T_B, \chi_B)$ as well as the bag $Z = S \cup X$, thus the bags of $T$ containing $v$ are connected. For $v \in A \cap S$ (the case when $v \in B \cap S$ is symmetric), note that $v$ must be contained in the bag of $z_A$ and in the bag of $z$. By induction hypothesis, the nodes of $T_A$ whose bags containing $v$ form a subtree of $T_A$ rooted at $z_A$, and thus the nodes of $T$ whose bags containing $v$ form a subtree of $T$.

That the root bag of $(T, \chi)$ contains $S$ is valid by construction. It remains to verify that $(T, \chi)$ has width at most $4w + 4$. This is trivial from the induction hypothesis ($(T_A, \chi_A)$ and $(T_B, \chi_B)$ have width at most $4w + 4$ respectively) and that $|S \cup X| \leq 3w + 4 + w + 1 \leq 4w + 5$.

**Runtime.** At each execution of **compute-tw**$(G, S, w)$, the most expensive part is the execution of the procedure $\mathcal{A}$ in line 5, which takes $O(3^{|S|} \cdot wm) = O(27^w \cdot wm)$ by Lemma 6.4.3. Therefore, if we prove that the constructed tree decomposition has at most $n$ nodes, the claimed running time of the main theorem is settled. The key observation is that at line 3, we add at least one vertex to $S$ (otherwise, we terminated the current call at line 2) because the previous recursive call sets $|S| \leq 3w + 3$ as discussed above. Moreover, any newly added vertex at line 3 does not appear in the parent node (why?). Therefore, with the tree decomposition $(T, \chi)$ returned at the end of the full execution of the algorithm, one can define the mapping $\mathsf{top} : V(G) \to V(T)$ so that $\mathsf{top}(v)$ is the topmost node whose bag contains $v$. Now that this mapping is surjective, it holds that $V(T) \leq V(G) = n$. This settles the claimed running time.

# Chapter 7

# Dynamic programming over Subsets

If a problem can be optimally solved by combining the solutions to a smaller problem, then dynamic programming approach can be used. We give two dynamic programming algorithm, one for TRAVELLING SALESMAN PERSON and another for STEINER TREE. Both runs in time $2^n \cdot n^{O(1)}$ and requires exponential space.

TRAVELLING SALESMAN PERSON
**Instance:** a complete graph $G = (V, E)$ with distance $d : \binom{V}{2} \to \mathbb{R}_{\geq 0}$
**Question:** find a closed tour of minimum total distance visiting every vertex precisely once.

For a graph $G = (V, E)$, and a vertex subset $K \subseteq V$, a steiner subgraph for $K$ is a connected subgraph $H$ of $G$ which contains all vertices of $K$. Intuitively, a steiner subgraph for $K$ is an essential structure in $G$ that pairwise connect the vertices of $K$ The vertices of $K$ are called terminals. For a subgraph $H$ of an edge-weighted graph $G$ with weight function $\omega : E \to \mathbb{R}_{\geq 0}$, the weight of $H$ is the sum $\sum_{e \in E(H)} \omega(e)$ over all $H$'s edges and will be denoted by $\omega(H)$. In this vein, we are interested in finding a steiner subgraph of minimum weight. With non-negative weights, a steiner subgraph of minimum edge count/weight sum can be assumed to be a tree and we call a steiner subgraph which is a tree a steiner tree, or $K$-steiner tree to emphasize the terminal set that the tree covers. This leads to the following fundamental problem.

STEINER TREE
**Instance:** an edge-weighted graph $G = (V, E)$ with weight function $\omega : E \to \mathbb{R}_{\geq 0}$, and a set of vertices $K \subseteq V$ (terminals)
**Question:** find a $K$-steiner tree of minimum weight, if one exists.

## 7.1 DP for TRAVELLING SALESMAN PERSON

Fix a vertex $s$. For all subsets $s \in S \subseteq V$ and a vertex $v \in S$, we compute the value $P[S, v]$ of a minimum distance $(s, v)$-path in $G[S]$ visiting every vertex exactly once. Note that the value of a minimum distance closed tour in $G$ visiting every vertex once equals

$$\min\{P[S, v] + d(v, s) : v \in V\}.$$

The base case is when $S = \{s\}$ and $v = s$, and we have $P[S, s] = 0$ trivially. For sets $S$ containing $s$ with $|S| \geq 2$, the next recursion for $P[S, v]$ is easy to see.

$$P[S, v] = \begin{cases} 0 & \text{if } v = s \\ \min\{P[S \setminus v, w] + d(w, v) : w \in S \setminus v\} & \text{if } v \neq s. \end{cases}$$

Each computation of $P[S, v]$ requires $O(|S|)$ look-ups of the table $P$ constructed for sets of size $|S| - 1$. As there are $2^{n-1} \cdot n$ entries in the table, the algorithm takes $O(2^n \cdot n^2)$-time.

The above recursive formula and the resulting algorithm computes the value of an optimal TSP tour. How can we find a tour whose total distance equals the determined value?

## 7.2 DP for STEINER TREE

**Assumption.** If $|K| \leq 2$, then STEINER TREE has a trivial solution; if $|K| = 1$, a trivial steiner tree consisting of a singleton is an optimal solution and a steiner tree which is a shortest path between two terminals is an optimal solution for the case when $|K| = 2$. Therefore, we assume $|K| \geq 3$. Moreover, $G$ can be assumed to be connected; if $K$ resides in more than one connected components of $G$, there is no $K$-steiner tree and we report so. If this is not the case, we can take as the input graph the unique connected component of $G$ containing the entire set $K$.

We may also assume that each terminal vertex has degree 1 in $G$, and thus is a leaf in any $K$-steiner tree in $G$. Indeed, one can modify the graph $G$ to $G'$ by appending $v'$, as a leaf, to each $v \in K$ and declaring the new appended vertices as the set $K'$ of terminals. The edge $vv'$ has weight 1 in the new graph $G'$. Observe that there is a one-to-one correspondence between $K$-steiner trees in $G$ and $K'$-steiner trees in $G'$. Given a $K$-steiner tree $T$, a non-terminal vertex of $T$ is called a steiner vertex of $T$. Notice that with the aforementioned modification, any $K$-steiner tree contains a steiner vertex.

**Notations.** For all subsets $\emptyset \neq D \subseteq K$ and $v \in V(G) \setminus K$, let $t(D, v)$ be the weight of an optimal $(D \cup v)$-steiner tree~~which takes $v$ as a leaf~~. Note that $v \notin D$ as the second entry $v$ is taken from $V(G) \setminus K$. Recall that we modified the input graph so that any $K$-steiner tree contains at least one steiner vertex. Therefore, our goal is to compute $\min\{t(K, s) : s \in V(G) \setminus K\}$. Figuring out how to construct an optimal steiner tree achieving this minimum weight is left to the readers as an exercise. We denote by $\mathsf{dist}(u, v)$ the total weight of a shortest (with respect to $\omega$) $(u, v)$-path.

Notice that $|D \cup v| \geq 2$. For $|D \cup v| = 2$, we have $t(D, v) = \mathsf{dist}_G(u, v)$ where $D \cup v = \{u, v\}$. Therefore, we consider the case when $|D \cup v| \geq 3$ and compute the table entry $t(D, v)$.

**Idea sketch.** Let $T$ be a $K \dot{\cup} \{v\}$-steiner tree, where $v \in V(G) \setminus K$. We view $T$ as rooted at $v$. Let $z$ be a vertex of $T$ closest to $v$ on $T$ and has at least two children in $T$. As all terminals of $K$ are leaves in $T$ and $|K| \geq 3$ (see the assumption above), there exists at least one descendant of $v$ with at least two children, possibly with $z = v$. Now we decompose (the edge set of) $T$ into three parts; one is the $(v, z)$-path $P$ (possibly with empty set of edges if $z = v$), and partition $E(T) - E(P)$ into two subtrees $T_1$ and $T_2$. As $z$ is chosen to have at least two children in $T$, one can build such a tripartition of $E(T)$ so that each of $T_1$ and $T_2$ contain at least one leaf of $T$ and each leaf of $T$ belongs to $K$. This means both $T_1$ and $T_2$ contain at least one terminal of $K$ each.

Now we want to view $T_i$ as a steiner tree for $(K \cap V(T_i)) \cup \{z\}$ for both $i = 1, 2$. As $z$ has degree at least two in $T$ and thus in $G$, it does not belong to $K$. This leads us to the recursive formula below,.

**Recursive formula.** We prove that the following recursive formula holds for all subsets $\emptyset \neq D \subseteq K$ and $v \in V \setminus K$ with $|D \cup v| \geq 3$.

$$(\star) \qquad t(D,v) = \min_{\substack{z \in V(G)\setminus K \\ D^1 \uplus D^2 = D \\ D^i \neq \emptyset, i=1,2}} t(D^1,z) + t(D^2,z) + \mathsf{dist}_G(z,v)$$

**Lemma 7.2.1.** $t(D,v) \leq \min_{\substack{z \in V(G)\setminus K \\ D^1 \uplus D^2 = D \\ D^i \neq \emptyset, i=1,2}} t(D^1,z) + t(D^2,z) + \mathit{dist}_G(z,v)$

**Proof:** Let $T_i$ be an optimal $(D^i \cup z)$-steiner tree for $i = 1, 2$, and note that $\omega(T_i) = t(D^i, z)$. Let $P$ be a shortest $(z,v)$-path of $G$. Then the graph $H$ on the vertex set $V(H) := V(T_1) \cup V(T_2) \cup V(P)$ and the edge set $E(T_1) \cup E(T_2) \cup E(P)$ is a steiner subgraph for $D^1 \cup D^2 \cup \{z, v\}$, connected via $z$, and thus a steiner subgraph connecting $D \cup \{v\}$. It suffices to observe that $\omega(H) = t(D^1, z) + t(D^2, z) + \mathsf{dist}_G(z, v)$ and $H$ contains a $(D \cup v)$-steiner tree $T$ of weight at most $\omega(H) = t(D^1, z) + t(D^2, z) + \mathsf{dist}_G(z, v)$. □

**Lemma 7.2.2.** $t(D,v) \geq \min_{\substack{z \in V(G)\setminus K \\ D^1 \uplus D^2 = D \\ D^i \neq \emptyset, i=1,2}} t(D^1,z) + t(D^2,z) + \mathit{dist}_G(z,v)$

**Proof:** Let $T$ be an optimal $(D \cup v)$-steiner tree, thus satisfying $\omega(T) = t(D \cup v)$. We construe $T$ as rooted at $v$. Note that $T$ has at least three leaves due to the assumption $|K| \geq 3$, meaning that $v$ has a descendant with at least two children in $T$. Choose such a descendant $z$ so that the $(v, z)$-path $P$ is as short as possible; if $v$ has two children itself, we take $z = v$ and $P$ is the trivial path consisting of $v$ itself. By the choice of $z$, we know that $z \notin K$. Let $T_1, \ldots, T_\ell$ be the subtress of $T$ obtained from $T$ by removing all vertices of $P$. We observe that $\ell \geq 2$ and each subtrees $T_i$ contains at least one terminal. Let $T^1$ be the subtree of $T$ induced by the vertex set $V(T_1) \cup \{z\}$ and $T^2$ be the subtree of $T$ induced by the vertex set $\bigcup_{i=2}^{\ell} V(T_i) \cup \{z\}$. Then for both $i = 1, 2$, $T^i$ is a $(D^i \cup z)$-steiner tree, where $D^i = D \cap V(T^i)$. Note that $D^1$ and $D^2$ are disjoint as the only common vertex of $T^1$ and $T^2$ is $z$, which does not belong to $K$ (thus not in $D$). Moreover, we have $D^i \neq \emptyset$ for $i = 1, 2$ and $(D^1, D^2)$ forms a bipartition of $D$. Clearly, $P$ is a $(z, v)$-path of $G$. Therefore

$$\begin{aligned} t(D,v) = \omega(T) &= \omega(T^1 \dot\cup T^2 \dot\cup P) = \omega(T^1) + \omega(T^2) + \omega(P) \\ &\geq t(D^1,z) + t(D^2,z) + \mathsf{dist}_G(z,v) \\ &\geq \min_{\substack{w \in V(G)\setminus K \\ D^1 \uplus D^2 = D \\ D^i \neq \emptyset, i=1,2}} t(D^1,w) + t(D^2,w) + \mathsf{dist}_G(w,v) \end{aligned}$$

which settles the inequality. □

**Algorithm and Runtime.** Assuming that $t(D, v)$ have been computed for all $\emptyset \neq D \subseteq K$ with $|D| \leq i$ and for all $v \in V(G)\setminus K$, one can compute $t(D, v)$ for all $\emptyset \neq D \subseteq K$ with $|D| = i+1$ and for all $v \in V(G)\setminus K$. This is because the computation of $t(D, v)$ requires access only to those entries of the form $t(D', z)$ with $\emptyset \neq D' \subsetneq D$ and $z \in V(G) \setminus K$, meaning $|D'| < |D|$. Therefore we can compute the full table $t(\cdot, \cdot)$, and in particular determine the weight of an optimal $K$-steiner tree by computing $\min\{t(K, v) : v \in V(G) \setminus K\}$.

To see the running time, observe that determining the value of $t(D, v)$ requires to inspect $n$ different choices for $z$ and at most $2^{|D|}$ different bipartitions of $D$ into $D^1$ and $D^2$. Therefore, computing $t(D, v)$ takes at

most $n \cdot 2^{|D|}$ arithmetic operations. In total, it takes

$$(\text{Running time of all-pairs shortest paths problem}) + \sum_{i=2}^{|K|} n \cdot \binom{|K|}{i} \cdot 2^i = O(n^3) + n3^{|K|},$$

that is, $O(n^3 + n \cdot 3^{|K|})$-time.

# Chapter 8

# Inclusion-Exclusion Technique

## 8.1 Inclusion-Exclusion formula

**Theorem 8.1.1** (Inclusion-Exclusion, union version)**.** *Let $A_i$ for $i = 1, \ldots, n$ be finite sets. Then,*

$$|\bigcup_{i\in[n]} A_i| = \sum_{\emptyset\neq X\subseteq[n]} (-1)^{|X|+1} |\bigcap_{i\in X} A_i|.$$

**Proof:** Notice that an element not in $\bigcup_{i\in[n]} A_i$ contributes neither to any term of the right-hand side, nor to the left-hand side. For an element $x \in \bigcup_{i\in[n]} A_i$, its contribution to the left-hand side is 1. It remains to show that the sum of contribution of $x$ to the right-hand side is precisely 1. Let $Y \subseteq [n]$ be the set of indices $i$ such that $x \in A_i$. Then for every $\emptyset \neq X \subseteq Y$, $\bigcap_{i\in X} A_i$ contains $x$. Conversely, for every $\emptyset \neq X \nsubseteq Y$ we have $x \notin \bigcap_{i\in X} A_i$. Therefore, $x$ creates the following terms of the right-hand side:

$$\begin{aligned}
\sum_{\emptyset\neq X\subseteq Y} (-1)^{|X|+1} \cdot 1 &= (-1) \sum_{\emptyset\neq X\subseteq Y} (-1)^{|X|} \\
&= -\sum_{i=1}^{|Y|} \sum_{X\subseteq Y, |X|=i} (-1)^i \\
&= -\sum_{i=1}^{|Y|} \binom{|Y|}{i} (-1)^i 1^{|Y|-i} \\
&= -\Big(\sum_{i=0}^{|Y|} \binom{|Y|}{i} (-1)^i 1^{|Y|-i} - 1\Big) \\
&= 1 - (-1+1)^{|Y|} = 1.
\end{aligned}$$

□

**Theorem 8.1.2** (Inclusion-Exclusion, intersection version)**.** *Let $A_i$ for $i = 1, \ldots, n$ be sets of a finite universe $U$. Then,*

$$|\bigcap_{i\in[n]} A_i| = \sum_{X\subseteq[n]} (-1)^{|X|} |\bigcap_{i\in X} (U \setminus A_i)|.$$

**Proof:** First, we note that for finite sets $B_i$, $i \in [n]$,

$$U \setminus \bigcup_{i\in[n]} B_i = \bigcap_{i\in[n]} (U \setminus B_i). \tag{8.1}$$

Therefore, by Theorem 8.1.1 it holds that

$$\begin{aligned} |U \setminus \bigcup_{i\in[n]} B_i| &= |U| + \sum_{\emptyset\neq X\subseteq[n]} (-1)^{|X|} |\bigcap_{i\in X} B_i| \\ &= \sum_{X\subseteq[n]} (-1)^{|X|} |\bigcap_{i\in X} B_i|. \end{aligned} \tag{8.2}$$

Set $A_i = U \setminus B_i$ and combine the equations (8.1)-(8.2). Now,

$$\begin{aligned} |\bigcap_{i\in[n]} A_i| = |\bigcap_{i\in[n]} (U \setminus B_i)| &= |U \setminus \bigcup_{i\in[n]} B_i| \\ &= |U| - \sum_{\emptyset\subsetneq X\subseteq[n]} (-1)^{|X|+1} |\bigcap_{i\in X} B_i| \\ &= \sum_{X\subseteq[n]} (-1)^{|X|} |\bigcap_{i\in X} (U \setminus A_i)|, \end{aligned}$$

where the last equation follows from the convention of writing $U = \bigcap_{i\in\emptyset} B_i$. □

## 8.2 IE-based algorithm for HAMILTONIAN CYCLE

Using the Inclusion-exclusion formula we can compute HAMILTONIAN CYCLE in $2^n \cdot n^{O(1)}$-time. In fact we can count the number of Hamiltonian cycles in the same running time.

Let $G = (V, E)$ be on $n$ vertices $v_1, \ldots, v_n$, and let $v_0 = v_n$. A closed walk is a sequence of vertices of $G$ whose start and end vertices are identical, and any two consecutive vertices are adjacent in $G$. Notice that a vertex or an edge might appear in a walk multiple times. The length of a closed walk is the length of vertex sequence minus one. By $v_0$-walk, we mean a closed walk that begins and ends with $v_0$. To apply the (intersection version) of inclusion-exclusion formula, we define the ground set $U$ as follows:

$$U = \{\text{all } v_0\text{-walks of length } n\}.$$

Now we can view a Hamiltonian cycle (with an orientation) as a $v_0$-walk of length $n$ which visits every $v \in V$. Notice that each Hamiltonian cycle yields two $v_0$-walks of length $n$ visiting every vertex $v$. Therefore with $A_i$ defined as

$$A_i = \{\text{all } v_0\text{-walks of length } n \text{ visiting } v_i\},$$

the Hamiltonian cycles, the $v_0$-walks of length $n$ visiting all $v \in V$ to be precise, are captured by $\bigcap_{i\in[n]} A_i$. Its cardinality can be computed by computing $|\bigcap_{i\in X}(U \setminus A_i)|$ for every $X \subseteq [n]$ thanks to Theorem 8.1.2.

So, what kind objects constitute $\bigcap_{i\in X}(U \setminus A_i)$? Observe that $U \setminus A_i$ are precisely the $v_0$-walks of length $n$ which avoid $v_i$, and thus $\bigcap_{i\in X}(U \setminus A_i)$ are $v_0$-walks of length $n$ which avoid all vertices corresponding to $X$. In other words, $\bigcap_{i\in X}(U \setminus A_i)$ are the set of all $v_0$-walks of length $n$ in $G - X$ (formally $G - \{v_i : i \in X\}$).

Finally, the number of $(v_i, v_j)$-walks of length $\ell$ in a graph $H$ can be computed in polynomial time by computing $\ell$-th power of the adjacency matrix of $H$ and reading off the $(i, j)$-entry of the resulting matrix. This completes the algorithm and it is straightforward to see that after $2^n$ steps all the terms of $\sum_{X\subseteq[n]}(-1)^{|X|}|\bigcap_{i\in X}(U \setminus A_i)|$ have summed up. We remark that this algorithm works both for directed and undirected graphs.

## 8.3 IE-based algorithm for $k$-COLORING

To apply the intersection version of inclusion-exclusion formula, we view a $k$-coloring as a $k$-tuple of independent sets of $G$. Namely, we define

$$U = \{(I_1, \ldots, I_k) : I_i \text{ is an independent set of } G\}.$$

Notice that two independent sets in a tuple may intersect and even coincide. Observe that there is a (proper) $k$-coloring if and only if there is $k$-tuple of independent sets covering all vertices of $G$. Therefore let

$$A_i = \{(I_1, \ldots, I_k) \in U : v_i \in I_1 \cup \cdots \cup I_k\},$$

and $G$ admits a proper $k$-coloring if and only if $\bigcap_{i\in[n]} A_i \neq \emptyset$. Due to Theorem 8.1.2, we can decide this via computing the value $\sum_{\emptyset\neq X\subset[n]}(-1)^{|X|+1}|\bigcap_{i\in X}(U \setminus A_i)|$.

Again, $\bigcap_{i\in X}(U \setminus A_i)$ is the set of all $k$-tuples of independent sets avoiding the vertices in $X$ altogether. In other words, it is the set of all $k$-tuples of independent sets of $G-X$. Let $i(G)$ be the number of independent sets of $G$ and observe

$$|\bigcap_{i\in X}(U \setminus A_i)| = i(G - X)^k.$$

Now $i(G)$ can be computed with dynamic programming. Choose an arbitrary vertex $v \in G$ and note that

$$i(G) = i(G - v) + i(G - N[v])$$

where the first term in r.h.s counts the independent sets of $G$ *not* containing $v$ and the second term counts the independent sets of $G$ containing $v$, thus excluding $N(v)$. The base case is $i(\emptyset)$ and $i(K_1)$, i.e. an empty graph and a graph on one vertex. The number of independent sets in each case is 1 and 2 respectively. This recursion indicates that $i(G[Z])$ over all subsets $Z$ of $V$ can be tabulated, and this can be done in time $2^n \cdot n^{O(1)}$.

With the above table containing values for $i(G - X)$ for all $X \subseteq [n]$, we can compute

$$\sum_{X\subseteq[n]}(-1)^{|X|+1}|\bigcap_{i\in X}(U \setminus A_i)| = \sum_{X\subseteq[n]}(-1)^{|X|+1}i^k(G - X)$$

in time $2^n \cdot n^{O(1)}$.

## 8.4 Matrix Tree Theorem

Let $G = (V, E)$ be a loopless (undirected) graph on $n$ vertices with $V = \{v_1, \ldots, v_n\}$. We identify the vertex set $V$ with $[n]$, referring to vertex $v_i$ by $i$. Fix an arbitrary ordering $\sigma$ on the vertex set of a graph $G$. Let $A_G$ be the adjacency matrix of $G$ and $D_G$ be the $n$ by $n$ diagonal matrix such that $i$-th diagonal entry of

$D_G$ equals $deg(i)$. In both $A_G$ and $D_G$, the rows and columns are ordered with respect to the fixed ordering $\sigma$.

The Laplacian $L_G$ of $G$ is defined as $D_G - A_G$. We prove the next theorem, called Matrix Tree Theorem (or often called Kirchhoff's theorem), which relates the number of spanning trees of $G$ with the determinant of $L_{G-v_n}$. Here the choice of $v_n$ is arbitrary and any choice of a vertex $G$ works.

**Theorem 8.4.1.** *Let $G$ be a connected graph and $v_n$ be a vertex of $G$. Then the number of spanning trees of $G$ equals the determinant of $L_{G-v_n}$, the Laplacian of $G - v_n$.*

We begin with the observation that instead of a spanning tree, we can equivalently count the number of trees rooted at a fixed vertex, say $v_n$. A function $f : [n-1] \to [n]$ is a pseudo-forest mapping (with respect to $G$ and a fixed root $v_n$) if for every $i \in [n-1]$, the image $f(i)$ is a neighbor of vertex $i$ in $G$. A cyclic sequence (with respect to $G$) is a sequence $i_1, \ldots, i_k$ of distinct elements of $[n-1]$ such that $i_2 = N(i_1)$, $i_3 = N(i_2), \cdots, i_k = N(i_{k-1})$ and $i_1 = N(i_k)$. Two cyclic sequences are considered identical if one is obtained from the other by cyclic shift of the other. An easy way to understand cyclic sequence is that it either corresponds to an edge, or it corresponds to a cycle of $G - v_n$ with a consistent orientation on the edges of the cycle. A pseudo-forest mapping is said to produce a cyclic sequence $C = i_1, \ldots, i_k$ if $f(i_j) = i_{j+1}$ for all $j \in [k-1]$ and $i_1 = f(i_k)$. A pseudo-forest mapping is acyclic if it does not produce any cyclic sequence.

An $f$-graph for a pseudo-forest mapping $f$ is the directed graph on $[n]$ whose arc set is $\{(i, f(i) : i \in [n-1]\}$. Note that a cycle of pseudo-forest mapping $f$ corresponds to a directed cycle in the $f$-graph, and any cycle of pseudo-forest mapping $f$ has length at least two because $G$ is loopless.

**Observation 8.4.1.** *There is one-to-one correspondence between the set of all spanning trees of $G$ rooted at $v_n$ and the set of all acyclic pseudo-forest mappings.*

**Proof:** For each acyclic pseudo-forest mapping $f$, the graph on the vertex set $[n]$ and with the edge set $\{(i, f(i) : i \in [n-1]\}$ is a spanning tree of $G$ rooted at $v_n$. Conversely, for any spanning tree of $G$ rooted at $v_n$, one can associate a pseudo-forest mapping $f$ by setting $f(i)$ to be the parent of $i$ in the rooted tree. □

Let $\mathcal{C} = \{C_1, \ldots, C_M\}$ be the collection of all cyclic sequences with respect to $G$. We define the universe $U$ and the set $A_\ell$ for $\ell \in [M]$ as follows.

- $U$ is the set of all pseudo-forest mappings.
- $A_\ell$ is the set of all pseudo-forest mappings each of which produces the cyclic sequence $C_\ell$.

By Observation 8.4.1, the number of spanning trees rooted at $v_n$ equals the number of all acyclic pseudo-forest mappings. As the set of all acyclic pseudo-forest mappings is $\bigcap_{\ell \in [M]} (U \setminus A_\ell)$, by the intersection version of Inclusion-Exclusion formula, the cardinality of it can be expressed as

$$(\star) \qquad |\bigcap_{\ell \in [M]} (U \setminus A_\ell)| = \sum_{X \subseteq [M]} (-1)^{|X|} |\bigcap_{\ell \in X} A_\ell|.$$

For $X \subseteq [M]$, let $\mathcal{C}_X \subseteq \mathcal{C}$ be the subcollection $\{C_i : i \in X\}$ of cyclic sequences. The next claim is essential.

**Lemma 8.4.1.** *For $X \subseteq [M]$, the following holds.*

$$|\bigcap_{\ell \in X} A_\ell| = \begin{cases} \prod_{i \notin \cup_{x \in X} C_x} deg(i) & \text{if the cyclic sequences in } \mathcal{C}_X \text{ are pairwise disjoint} \\ 0 & \text{otherwise.} \end{cases}$$

**Proof:** Notice that the collection of cyclic sequences produced by a pseudo-forest mapping $f$ are pairwise disjoint, which implies the equality in the second case. Suppose the cyclic sequences in $\mathcal{C}_X$ are pairwise disjoint. Observe that pseudo-forest mapping $f$ produces $\mathcal{C}_X$ if and only if only if $f$ satisfies the following.

- if $i$ appears in some cyclic sequence $C \in \mathcal{C}_X$, then $f(i)$ is the succeeding element of $i$ on $C$, and
- $i$ does not appear in any cyclic sequence of $\mathcal{C}_X$, then $f(i) \in N(i)$.

This implies that the number of pseudo-forest mappings producing $\mathcal{C}_X$ is exactly $\prod_{i \notin \cup \mathcal{C}_X} deg(i)$, where $\cup \mathcal{C}_X$ denotes the set $\bigcup_{C \in \mathcal{C}_X} C$. This completes the proof. □

Let $X \subseteq [M]$ be (the indices of) a pairwise disjoint cyclic sequences of $\mathcal{C}$. We define a function $\pi_X$ on $[n-1]$ as follows.

$$\pi_X(i) = \begin{cases} \text{the succeeding element of } i \text{ on } C & \text{if } i \in C \in \mathcal{C}_X \\ i & \text{otherwise.} \end{cases}$$

It is clear that $\pi_X$ is a permutation on $[n-1]$ for every $X \subseteq [M]$ such that the cyclic sequences in $\mathcal{C}_X$ are pairwise disjoint. Conversely, every permutation $\pi$ on $[n-1]$ produces a (possibly empty) set of pairwise disjoint cyclic sequences. This is summarized in the next observation.

**Observation 8.4.2.**

$$\{\pi_X : X \subseteq [M] \text{ and the cyclic sequences in } \mathcal{C}_X \text{ are pairwise disjoint}\} = S_{n-1}.$$

Note that for $X \subseteq [M]$ such that the cyclic sequences in $\mathcal{C}_X$ are pairwise disjoint, $\pi_X$ is precisely the permutation whose cycle representation is $\mathcal{C}_X$. The next statement is a folklore.

**Observation 8.4.3.** *Let $X \subseteq [M]$ be such that the cyclic sequences in $\mathcal{C}_X$ are pairwise disjoint. We have* $\mathsf{sgn}(\pi_X) = (-1)^{\sum_{x \in X}(|C_x|-1)}$.

We are ready to rewrite $(\star)$:

$$
\begin{aligned}
|\bigcap_{\ell\in[M]}(U\setminus A_\ell)| &= \sum_{X\subseteq[M]}(-1)^{|X|}|\bigcap_{\ell\in X}A_\ell| \\
&= \sum_{\substack{X\subseteq[M]\\ \mathcal{C}_X \text{ pairwise disjoint}}}(-1)^{|X|}\prod_{i\notin\cup\mathcal{C}_X} deg(i) \qquad \because \text{Lemma 8.4.1} \\
&= \sum_{\substack{X\subseteq[M]\\ \mathcal{C}_X \text{ pairwise disjoint}}}(-1)^{\sum_{x\in X}(|C_x|-1)}\cdot(-1)^{\sum_{x\in X}|C_x|}\prod_{i\notin\cup\mathcal{C}_X} deg(i) \\
&= \sum_{\substack{X\subseteq[M]\\ \mathcal{C}_X \text{ pairwise disjoint}}}\mathsf{sgn}(\pi_X)\cdot\prod_{i\in\cup\mathcal{C}_X}(-1)\cdot\prod_{i\notin\cup\mathcal{C}_X} deg(i) \qquad \because \text{Observation 8.4.3} \\
&= \sum_{\pi\in S_{n-1}}\mathsf{sgn}(\pi)\cdot\prod_{i:\pi(i)\neq i}(-1)\cdot\prod_{i:\pi(i)=i} deg(i) \qquad \because \text{Observation 8.4.2} \\
&= \sum_{\pi\in S_{n-1}}\mathsf{sgn}(\pi)\cdot\prod_{i\in[n-1]} L_{i,\pi(i)},
\end{aligned}
$$

where $L$ is the Laplacian of $G - v_n$.

# Chapter 9

# Algorithms for $k$-SAT

# Chapter 10

# Exponential Time Hypothesis

Throughout the lecture note, $n$ denotes the number of vertices of an input graph or the number of variables of an input CNF formula. The number of edges or clauses is denoted by $m$.

In the previous lecture, we learnt Schöning's random walk-based algorithm for $k$-SAT, which works in time $O^*(2^{(1-\Theta(\frac{1}{k}))n})$. This means that one can do better than the $O^*(2^n)$-time brute-force algorithm, enumerating all possible truth assignment on $n$ variables and checking whether one is a satisfying assignment, when there is a bound $k$ on the size of a clause. Two natural questions arise.

- What is the best (i.e. smallest) constant $c \in \mathbb{R}$ such that $k$-SAT can be solved in time $O^*(2^{cn})$? Is there a limit on how small $c$ can get? or can it get arbitrarily close to zero? Formally, let $\delta_k$ to be the infimum of the constants $c$ such that there is an $O(2^{cn} \cdot poly(n))$-time algorithm for $k$-SAT. Clearly $0 \leq \delta_k \leq 1 - \Theta(\frac{1}{k})$.
- When $k$ grows, $k$-SAT encompasses more instances; an instance of $k$-SAT is also an instance of $(k+1)$-SAT, so we have $0 \leq \delta_3 \leq \delta_4 \leq \delta_5 \leq \cdots \leq 1$. On the other hand $\delta_k \leq 1$ holds due to the $O^*(2^n)$-time brute-force algorithm. What is the limit of this sequence? Does $\delta_k$ converges [1] to 1 or the supremum is strictly below 1?

We do not know have firm answers to these question. However, despite much work over the last decades we see little indication that an algorithm for 3-SAT with subexponential running time will be attainable or $(2-\epsilon)^n$-time algorithm achievable for CNF-SAT. Impagliazzo and Paturi [**?** ] formulated the following hypothesis, known as `Exponential Time Hypothesis` and `ETH` in short.

**Conjecture 10.0.1** (`Exponential Time Hypothesis`)**.** $\delta_3 > 0$.

Regarding the second question above, Impagliazzon and Paturi proposed `Strong Exponential Time Hypothesis` in [**?** ].

**Conjecture 10.0.2** (`Strong Exponential Time Hypothesis`)**.** $\lim_{k\to\infty} \delta_k = 1$.

**Lemma 10.0.1.** *Unless `ETH` fails,* 3-SAT *does not admit an $O^*(2^{o(n)})$-time algorithm.*

**Proof:** Suppose 3-SAT admits an algorithm $A$ running in time $O^*(2^{o(n)})$. Recall that $f(n) = o(g(n))$ for two functions $f$ and $g$ if for every $\epsilon > 0$, there exists a number $n_\epsilon$ such that $f(n) < \epsilon \cdot g(n)$ for every $n \geq n_\epsilon$.

[1] For a bounded non-decreasing sequence in the reals, there is a limit and it coincides with the supremum of the sequence.

Therefore, for every $c > 0$ there is an algorithm that runs in time $O^*(2^{cn})$. Notice that we can pick the same algorithm, namely $A$, for the sequence of algorithms corresponding to the choice of $c$. As the sequence get arbitrarily zero (and lower bounded by zero), $\delta_3 = 0$. □

Notice that $\delta_3 = 0$ does not necessarily mean that there is an algorithm for 3-SAT running in time $O^*(2^{\delta_3 n})$ (that will be a polynomial-time algorithm! The world is not that extreme to make a choice between polynomial versus $2^n$). It merely points out the unlikelihood of a sequence of algorithms whose running time is $O^*(2^{cn})$ for $c$ getting arbitrarily close to zero (but no $c$ is actually zero unless $P = NP$). Notice as well that a subexponential-time algorithm as in Lemma 10.0.1 is stronger than a sequence of such algorithms. In the latter, the choice of algorithm for each $c$ can be different whereas in the existence of a subexponential-time algorithm guarantees that a single algorithm work for such a sequence. To summarize, the existence of an $O^*(2^{o(n)})$-time algorithm implies the collapse of ETH but the opposite implication does not necessarily hold; within our current knowledge, a universe is possible where ETH is valid and there is no subexponential-time algorithm, i.e. there is a sequence of $O^*(2^{cn})$-algorithms for $c$ getting arbitrarily close to zero but only due to distinct choices of algorithm for each $c$.

The next lemma is a useful consequence of SETH which is often used as a starting point of an SETH-based lower bound. The easy proof is left to the readers.

**Lemma 10.0.2.** *Unless SETH fails,* CNF-SAT *does not admit an $O^*((2-\epsilon)^n)$-time algorithm for any $\epsilon > 0$.*

What makes ETH and SETH a robust ground for proving algorithmic lower bounds is so-called *Sparsification Lemma* thanks to [**?** ]. Thanks to the *Sparsification Lemma*, the following can be shown.

**Lemma 10.0.3.** *Unless ETH fails,* 3-SAT *does not admit an $O^*(2^{o(n+m)})$-time algorithm.*

**Lemma 10.0.4.** *SETH implies ETH.*

## 10.1 Some consequence of ETH

We have seen $O^*(2^k)$-time algorithm for VERTEX COVER and there is an $O^*(1.2738^k)$-time algorithm. On the other hand, is it ever possible to achieve an algorithm of running time in which the exponent is $o(k)$ instead of $O(k)$? One can ask similar questions for many other problems, i.e. whether the current algorithm is the asymptotically optimal (i.e. up to some constant factor) running time for a problem.

- For FEEDBACK VERTEX SET we studied an $O^*(5^k)$-time algorithm, which was recently improved to a (randomized) $O^*(2.7^k)$-time algorithm by [**?** ]. Is there a hope to design an algorithm whose asymptotical behavior on the exponent can be significantly better, e.g. $O^*(100^{\sqrt{k}})$, or even $O^*(1000^{k/\log k})$?
- For HAMILTONIAN CYCLE, we learnt algorithms (one using dynamic programming and another one using Inclusion-Exclusion formula) running in time $O^*(2^n)$ and there is an $O^*(1.657^n)$-time algorithm by [**?** ]. Can we radically depart from a running time of the form $O^*(c^n)$ for some constant $c$?
- For the problem CYCLE PACKING[2], there is a standard dynamic programming algorithm over tree-decomposition that runs in time $O^*(2^{O(t \log t)})$, where $t$ is the treewidth of $G$. Can we aim for an algorithm for CYCLE PACKING which runs in time $O^*(2^{O(t)})$, or even $O^*(2^{O(t\sqrt{\log t})})$?

[2]Given a graph $G$ and a positive integer $k$, one wants to decide if there is a set of $k$ pairwise vertex-disjoint cycles in $G$.

The answer to these questions is NO assuming ETH. And as one can readily expect, there are many more problems whose asymptotic running time can be bounded from below under ETH, which often asymptotically matches the upper bound given by known algorithms.

**Lemma 10.1.1.** *There is a polynomial transformation which, given a 3-CNF formula $\Phi$ on $n$ variables and with $m$ clauses, outputs an instance of* MAXIMUM INDEPENDENT SET *on $N = 2n + 3m$ vertices.*

**Lemma 10.1.2.** *There is a polynomial transformation which, given a 3-CNF formula $\varphi$ on $n$ variables and with $m$ clauses, outputs an instance of* MAXIMUM INDEPENDENT SET *on $N = 2n + 3m$ vertices.*

An incidence graph $G_\varphi$ of a CNF formula $\varphi$ is a bipartite graph on the $V(\varphi) \dot\cup C(\varphi)$, where $V(\varphi)$ and $C(\varphi)$ are the sets of variables and clauses of $\varphi$ respectively, and $G_\varphi$ has an edge $xC$ between $x \in V(\varphi)$ and $C \in C(\varphi)$ whenever the variable $x$ appears in the clause $C$ (either positively or negatively) in $\varphi$.

The problem PLANAR 3-SAT is the problem 3-SAT with the additional restriction that the incidence graph of an input 3-CNF formulas is planar.

PLANAR 3-SAT
**Instance:** a 3-CNF formula $\varphi$ (on $n$ variables) such that $G_\varphi$ is planar.
**Question:** Is $\varphi$ satisfiable?

PLANAR 3-SAT WITH VARIABLE CYCLE
**Instance:** a 3-CNF formula $\varphi$ and a cyclic ordering $x_1, \ldots, x_n$ on its variables such that $G_\varphi + \{x_i x_{i+1} : i \in [n]\}$ (i.e. the graph obtained from $G_\varphi$ by adding the edges of the form $x_i x_{i+1}$) is planar.
**Question:** Is $\varphi$ satisfiable?

For a fixed cyclic ordering $x_1 \prec x_2 \prec x_3 \prec \cdots \prec x_n \prec x_1$ on the variable set of $\varphi$, note that the added set of edges $\{x_i x_{i+1} : i \in [n]\}$ forms a cycle, which we call variable cycle.

The following is known.

**Lemma 10.1.3.** *There is a polynomial transformation which, given a 3-CNF formula $\Phi$ on $n$ variables and with $m$ clauses, outputs a 3-CNF $\varphi$ on $O((n+m)^2)$ variables and clauses and a cyclic ordering $\prec$ on the variables of $\varphi$ such that*

- *$G_\varphi$ is planar, and moreover*
- *$G_\varphi$ remains planar after adding a 'variable cycle' following the cyclic order $\prec$,*
- *$\Phi$ is satisfiable if and only if $\varphi$ is satisfiable.*

*In particular,* PLANAR 3-SAT *and* PLANAR 3-SAT WITH VARIABLE CYCLE *are NP-complete.*

**Lemma 10.1.4.** *There is a polynomial transformation which, given a 3-CNF formula $\Phi$ on $n$ variables and with $m$ clauses, outputs a 3-CNF $\varphi$ on $O(n+m)$ variables and clauses such that*

- *each variable of $\varphi$ occurs in at most three clauses, and*
- *$\Phi$ is satisfiable if and only if $\varphi$ is satisfiable.*

**Lemma 10.1.5.** *There is a polynomial transformation which, given a 3-CNF formula $\Phi$ on $n$ variables and with $m$ clauses, outputs a 3-CNF $\varphi$ on $O((n+m)^2)$ variables and clauses and a cyclic ordering $\prec$ on the variables of $\varphi$ such that*

- *each variable of $\varphi$ occurs in at most three clauses, and*
- *$G_\varphi$ is planar, and moreover*
- *$G_\varphi$ remains planar after adding a 'variable cycle' following the cyclic order $\prec$,*
- *$\Phi$ is satisfiable if and only if $\varphi$ is satisfiable.*

## 10.2 Some consequence of `SETH`

# Chapter 11

# Parameterized Intractability

# Chapter 12

# Approximation Algorithms: Basics

## 12.1 Basics of Approximation Algorithm

From now on, as long as approximation algorithms are concerned, we consider Optimization problems instead of decision problems. Regarding the formal definition of optimization problems, see the first lecture note.

The optimization version of VERTEX COVER called MINIMUM VERTEX COVER, takes a graph $G$ as an input, and our goal is to find a vertex cover of maximum size. We cannot say that MINIMUM VERTEX COVER is NP-complete because NP-completeness is a term for decision problems only. However, we can consider the canonical decision version of an optimization problem. If the decision version is NP-hard, by convention we say the optimization problem NP-hard. Clearly, the decision variant can be solved by solving the optimization problem, so unless the decision variant is in P, we cannot expect to solve the optimization problem in polynomial time.

In the next couple of week, we learn how to design an algorithm which runs in polynomial time and which produces a solution as close to an optimal solution as possible. Such an algorithm is called an polynomial-time approximation algorithm or simply approximation. Unlike in FPT algorithms or fast exact algorithms, our running time budget is polynomial function and our main concern is to design an algorithm which produces a solution as close to an optimal solution as possible. Before formally defining this performance measure, let us consider an approximation algorithm for MINIMUM VERTEX COVER.

**Algorithm 9** Algorithm for MINIMUM VERTEX COVER

```
1: procedure minVC(G)
2:     S ← ∅.
3:     Mark every edge of G as uncovered
4:     while G has an uncovered edge do
5:         Pick an edge uv.
6:         S ← S ∪ {u, v}
7:         Mark every edge incident with u or v covered
8:     return S.
```

Note that no two edges chosen in line 5 share a vertex. Indeed, once an edge $e = uv$ is chosen in line 5

all edges sharing a vertex with $e$ will be marked as `covered` and will never be chosen later during the algorithm. That is, the edge set $M$ chosen during the course of the algorithm 9 is a matching of $G$. Let `opt` be the size of an optimal vertex cover. Because any vertex cover of $G$ must cover $M$ as well, we have $\texttt{opt} \geq |M|$. Now,

$$|S| = 2 \cdot |M| \leq 2 \cdot \texttt{opt},$$

that is, the returned solution of Algorithm 9 does not exceed twice the size of an optimal solution.

In general, for a minimization problem $\Pi$, a (polynomial-time) algorithm is said to be $\alpha$-approximation if it always (i.e. for any input instance to $\Pi$) outputs a solution whose objective value is at most $\alpha \cdot \texttt{opt}$, where `opt` is the objective value of an optimal solution. For a maximization problem, an $\alpha$-approximation outputs a solution whose objective value is at least $\frac{1}{\alpha} \cdot \texttt{opt}$, where `opt` is the objective value of an optimal solution. Clearly, we have $\alpha \geq 1$ and $\alpha$ is called the approximation ratio of the said algorithm. Sometimes $\alpha$ can be a function of the input size, e.g. the number of vertices.

## 12.2 Approximation for TRAVELLING SALESMAN PERSON

We formulate TRAVELLING SALESMAN PERSON as a graph problem.

> TRAVELLING SALESMAN PERSON
> **Instance:** an undirected graph $G$ and a distance function $d : E(G) \to \mathbb{Q}$.
> **Goal:** Find a cycle of minimum total distance visiting every vertex exactly once.

TRAVELLING SALESMAN PERSON does not admit an $\rho$-approximation for any constant $\rho$.

**Lemma 12.2.1.** TRAVELLING SALESMAN PERSON *does not admit an $\alpha$-approximation for any constant $\alpha$ unless $P = NP$.*

**Proof:** We reduce from HAMILTONIAN CYCLE. Given an input graph $G = (V, E)$ of HAMILTONIAN CYCLE, we create an instance $(G', d)$ of TRAVELLING SALESMAN PERSON as follows.

- $G'$ is a complete graph on $V$.
- $d(u, v) = 1$ if $uv \in E$ and $d(u, v) = \rho \cdot n + 1$

Notice that

(a) if $G$ has a hamiltonian cycle $C$, the corresponding tour along $C$ has total distance $n$. Conversely, any TSP tour of total distance $n$ in $G'$ forms a hamiltonian cycle of $G$.

(b) any TSP tour of $G'$ of total distance $> n$ has distance at least $(\rho + 1) \cdot n > \rho \cdot n$

Suppose that there is a $\rho$-approximation $\mathcal{A}$ for TRAVELLING SALESMAN PERSON. We argue that HAMILTONIAN CYCLE can be solved in polynomial-time using $\mathcal{A}$, a contradiction. To see this, consider the following algorithm. First, we run the above polynomial-time reduction from HAMILTONIAN CYCLE to TRAVELLING SALESMAN PERSON on the input $G$ of HAMILTONIAN CYCLE, and apply the presumed $\rho$-approximation $\mathcal{A}$ on the constructed instance $(G', d)$ to TRAVELLING SALESMAN PERSON. Finally,

(I) If $\mathcal{A}$ outputs a TSP tour $W$ of distance at most $\rho \cdot n$, we declare that $G$ has a hamiltonian cycle.

(II) If $\mathcal{A}$ outputs a TSP tour $W$ of distance strictly larger than $\rho \cdot n$, we declare that $G$ does not have a hamiltonian cycle.

To complete the proof, it suffices to show that the output at Case I and II are correct respectively. For Case I, note that (b) implies that $W$ actually has distance $n$ and thus by the second part of (a), $G$ has a hamiltonian cycle. For Case II, note that if $G$ contains a hamiltonian cycle, then by the first part of (a) and that $\mathcal{A}$ is a $\rho$-approximation, the TSP tour $W$ must have distance at most $\rho \cdot n$. Therefore, the output of II is correct. □

One of the essential variants of TRAVELLING SALESMAN PERSON studied extensively in the literature is so-called METRIC TSP; here, the distance function satisfies the triangle inequality. That is, for any three vertices $u, v, w$ of $G$ it holds that

$$d(u, v) \leq d(u, w) + d(w, v).$$

METRIC TSP
**Instance:** an undirected complete graph $G = (V, E)$ and a distance function $\omega : E \to \mathbb{Q}$.
**Goal:** Find a cycle of minimum total distance visiting every vertex exactly once.

For METRIC TSP, there is a simple 2-approximation (works of independent groups of authors) and an 1.5-approximation attributed to Christofides. The latter remained as an algorithm achieving the best approximation ratio for the general[1] METRIC TSP until a randomized $(1.5 - 10^{-36})$-approximation was proposed by Karlin, Klein, and Gharan (STOC 2021). As a deterministic one, it is not known if Christofides' algorithm can be improved.

### 12.2.1 2-Approximation for METRIC TSP

Let $T$ be a minimum spanning tree $G$. Clearly $T$ can be found in polynomial time using any of the polynomial-time algorithm for constructing a minimum spanning tree of a graph. Let $C^*$ be an optimal TSP tour of $(G, \omega)$. The first observation is

$$\omega(T) \leq \omega(C^*)$$

because $C^*$ minus an edge of $C^*$ is a spanning tree of $G$, therefore at least as large as the weight of a minimum spanning tree $T$.

Consider the auxiliary graph $H := (V, T + T)$, the graph on $V$ obtained by doubling the edges of $T$. As all vertices of $H$ has even degree (in $H$), it has an eulerian (closed) walk $W$. Let $W = w_0, w_1, \ldots, w_t = w_0$ be an eulerian walk of $H$ and note that $\omega(W) = 2 \cdot \omega(T)$. Now we extract a hamiltonian cycle of $G$ using the vertex sequence along $W$. We take the subsequence (not necessarily contiguous) of $W$

$$w_{i_0}, w_{i_1}, \ldots, w_{i_{n-2}}, w_{i_{n-1}}$$

defined as follows:

- $i_0 = 0$, i.e. $w_{i_0} = w_0$ and
- for every $1 \leq j \leq n - 1$, $i_j$ is the least index $k$ such that $k \geq i_{j-1} + 1$ and $w_k$ is distinct from all vertices preceding it on $W$.

[1] A nicely compiled list of TRAVELLING SALESMAN PERSON variants, albeit outdated, is found on the following thread of Stack Exchange; https://cstheory.stackexchange.com/questions/9241/approximation-algorithms-for-metric-tsp.

Because all $n$ vertices of $G$ appear in the sequence $W$, such a subsequence exists. Let us add the edge $(w_{i_{n-1}}, w_{i_n})$ to this sequence with $w_{i_n} = w_{i_0}$ and let $\tilde{W} := w_{i_0}, w_{i_1}, \ldots, w_{i_{n-2}}, w_{i_{n-1}}, w_{i_n} = w_{i_0}$ be the resulting cycle of $G$. Clearly $\tilde{W}$ is a TSP tour. Now,

$$\begin{aligned}
\omega(\tilde{W}) &= \sum_{j=0}^{n-1} \omega(w_{i_j}, w_{i_{j+1}}) \\
&\leq \sum_{j=0}^{n-1} \big(\omega(w_{i_j}, w_{i_j+1}) + \cdots + \omega(w_{i_{j+1}-1}, w_{i_{j+1}})\big) \\
&= \sum_{k=0}^{t-1} \omega(w_k, w_{k+1}) \\
&= \omega(W) \\
&= 2 \cdot \omega(T) \\
&\leq 2 \cdot \omega(C^*)
\end{aligned}$$

where the second inequality is due to the triangle inequality.

### 12.2.2 1.5-Approximation for METRIC TSP

We modify the above 2-approximation algorithm as follows. At the stage when you create an auxiliary graph $H$, we do not double the edges of $T$, i.e a minimum spanning tree, and instead you take $H := (V, T + M)$, where $M$ is a minimum weight (with respect to $\omega$) perfect matching on the vertex set $O$ consisting of those vertices having odd degree in $T$. Clearly $|O|$ is even[2] and there is a perfect matching in the graph $G[O]$.

We claim that $\omega(M) \leq 0.5 \cdot \omega(C^*)$ holds, which yields the claimed approximation ratio immediately:

$$\begin{aligned}
\omega(\tilde{W}) &\leq \omega(T) + \omega(M) \\
&\leq \omega(C^*) + 0.5 \cdot \omega(C^*) \\
&\leq 1.5 \cdot \omega(C^*).
\end{aligned}$$

To establish the inequality $\omega(M) \leq \omega(C^*)$, consider the TSP tour $C^*$ and how it traverses the vertex set $O$. Let $o_0, o_1, o_2, \ldots, o_{2p-1}, o_{2p} = o_0$ be the sequence of vertices of $O$ in the order of appearance on $C^*$ (the first vertex $o_0 = o_{2p}$ is chosen arbitrarily). Let $P_i$ be the subwalk of $C^*$ from $o_{i-1}$ to $o_i$. Observe

$$\begin{aligned}
(\star) \qquad \sum_{i=1}^{2p} \omega(o_{i-1}, o_i) &\leq \sum_{i=1}^{2p} \omega(P_i) \\
&= \omega(C^*)
\end{aligned}$$

where the first inequality holds due to the triangle inequality. Note that the edge set along the cycle $o_0, o_1, o_2, \ldots, o_{2p-1}, o_{2p} = o_0$ can be partitioned into two matchings, each covering all vertices of $O$;

[2]The number of odd degree vertices in any graph $G$ is even thanks to so-called the Handshaking Lemma asserting $|\sum_{v \in V(G)} d_G(v)| = |E(G)|$.

one matching takes all edges of the form $(o_{2i}, o_{2i+1})$ and the other matching takes all edges of the form $(o_{2i-1}, o_{2i})$. This partition and the inequality $(\star)$ together imply that there is a perfect matching of $G[O]$ of weight at most $0.5 \cdot \omega(C^*)$. As $M$ is a minimum weight perfect matching of $G[O]$, we conclude that $\omega(M) \leq 0.5 \cdot \omega(C^*)$, as claimed.

## 12.3 Approximation for STEINER TREE and METRIC STEINER TREE

STEINER TREE
**Instance:** a graph $G = (V, E)$, a set $K$ of terminals, a weight function $\omega : E \to \mathbb{Q}$.
**Goal:** Find a steiner tree for $K$ of minimum weight.

METRIC STEINER TREE
**Instance:** a complete graph $G = (V, E)$, a set $K$ of terminals, a weight function $\omega : E \to \mathbb{Q}$ satisfying the triangle inequality.
**Goal:** Find a steiner tree for $K$ of minimum weight.

**Approximation-factor Preserving Reduction.** Let us construct an instance $(G', K, \omega')$ to METRIC STEINER TREE from an arbitrary instance $(G, K, \omega)$ to STEINER TREE as follows.

- $G'$ is a complete graph on the vertex set $V(G)$ (and $K$ is the set of terminals for the new instance as well).
- $\omega'(u, v)$ is the weight of a shortest (w.r.t the weight $\omega$) $(u, v)$-path in $G$. When there is no $(u, v)$-path in $G$, then we set $\omega'(u, v) = \infty$.

The proof of the next statement is left to the readers.

**Lemma 12.3.1.** *The following holds.*

1. $\omega'(T) \leq \omega(T)$ *for any tree* $T$ *of* $G$.
2. *For any* $K$*-steiner tree* $T'$ *of* $G'$*, there exists a* $K$*-steiner tree* $T$ *of* $G$ *such that* $\omega(T) \leq \omega'(T')$*. Moreover, such* $T$ *can be obtained in polynomial time.*

Using Lemma 12.3.1, one can readily see that an $\alpha$-approximation for METRIC STEINER TREE implies an $\alpha$-approximation for STEINER TREE, and therefore the above reduction from STEINER TREE to METRIC STEINER TREE is an approximation-factor preserving reduction. Given an input $(G, K, \omega)$ to STEINER TREE, use the above reduction to create an instance $(G', K, \omega')$ to METRIC STEINER TREE. Let $T'$ be a $K$-steiner tree of $G'$ which achieves

$$\omega'(T') \leq \alpha \cdot \omega'(T^{**}),$$

where $T^{**}$ is an optimal $K$-steiner tree of $G'$. Let $T^*$ be an optimal $K$-steiner tree of $G$. By (1) of Lemma 12.3.1, we have

$$\omega'(T^{**}) \leq \omega'(T^*) \leq \omega(T^*).$$

On the other hand, (2) of Lemma 12.3.1 implies that there is a $K$-steiner tree $T$ of $G$ such that $\omega(T) \leq \omega'(T')$. Combining the three inequalities, we deduce

$$\omega(T) \leq \alpha \cdot \omega'(T^{**}) \leq \alpha \cdot \omega(T^*),$$

as claimed.

**2-Approximation for METRIC STEINER TREE.** Henceforth, we see how the 2-approximation for METRIC STEINER TREE works. The algorithm itself is simple: given an instance $(G, K, \omega)$ to METRIC STEINER TREE, find a minimum weight spanning tree $T$ of $G[K]$ and return $T$ as a $K$-steiner tree of $G$.

Let $T^*$ be an optimal $K$-steiner tree of $G$. We want to show that $\omega(T) \leq 2 \cdot \omega(T^*)$. The main idea for proving this bound is similar to what we already saw for METRIC TSP and we sketch the proof here.

Consider the graph $H = (V(T), T^* + T^*)$, i.e. the graph obtained from $T^*$ by doubling the edges. Let $W$ be an eulerian walk of $H$. From the eulerian walk $W$, we extract a hamiltonian cycle $C$ on $K$ by shortcutting non-terminal vertices of $W$. Notice that $\omega(C) \leq \omega(W) = 2 \cdot \omega(T^*)$, where the first inequality is due to the triangle inequality. Choose an arbitrary edge $e$ of $C$. Because the path $C - e$ is a spanning tree of $G[K]$ and $T$ is a minimum weight spanning tree of $G[K]$, we know that $\omega(T) \leq \omega(C - e)$. We conclude $\omega(T) \leq 2 \cdot \omega(T^*)$.

## 12.4 Greedy algorithm for SET COVER and MAX COVERAGE

The following inequality will be often used for analysis.

$$1 + x \leq e^x \qquad \text{for all } x \in \mathbb{R}.$$

We consider the problems SET COVER and MAX COVERAGE and analyze a greedy algorithm for them.

SET COVER
**Instance:** a universe $U$, a collection $\mathcal{S}$ of subsets of $U$.
**Goal:** Find a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ of minimum size (i.e. containing minimum possible sets of $\mathcal{S}$) such that $\bigcup_{S \in \mathcal{S}'} S = U$.

MAX COVERAGE
**Instance:** a universe $U$, a collection $\mathcal{S}$ of subsets of $U$. a positive integer $k$.
**Goal:** Find a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ consisting of $k$ sets which maximizes $|\bigcup_{S \in \mathcal{S}'} S|$

Consider the greedy algorithm which chooses a set following a simple, obvious criteria: choose a set in $\mathcal{S}$ which newly covers the maximum number of elements. For SET COVER, we do this greedy selection until all elements are covered and for MAX COVERAGE, we stop after choosing $k$ sets.

We analyze the greedy algorithm. Notice that the only difference between Algorithms 21 and 12 is the stopping criterion of WHILE-loop. We develop some arguments which are common to both of them.

- Let $\mathcal{S}^*$ be an optimal set cover and let $U^* := \bigcup_{S \in \mathcal{S}^*} S$.
- For SET COVER, it holds that $U^* = U$ and for MAX COVERAGE, we have $|\mathcal{S}^*| = k$.
- For notational convenience, we also set $k := |\mathcal{S}^*|$ for SET COVER.
- Let $X_1, \ldots, X_i, \ldots, X_k$ be the chosen sets by the greedy algorithm.

**Algorithm 10** Algorithm for SET COVER

```
procedure SetCover(U, S)
    Mark every element of U as uncovered.
    S' ← ∅.
    while U has an uncovered element do
        Select X ∈ S \ S' containing the maximum number of uncovered elements of U.
        S' ← S' ∪ {X}
        Mark every element of X covered (unless it is already covered)
    return S'.
```

**Algorithm 11** Algorithm for MAX COVERAGE

```
procedure MaxCoverage(U, S, k)
    Mark every element of U as uncovered.
    S' ← ∅.
    while U has an uncovered element and |S'| < k do
        Select X ∈ S \ S' containing the maximum number of uncovered elements of U.
        S' ← S' ∪ {X}
        Mark every element of X covered (unless it is already covered)
    return S'.
```

- Let $g_t := |U^*| - |\bigcup_{i=1}^{t} X_i|$, i.e. the gap between the number of elements that an optimal solution can cover and what the greedy solution covers at $t$.

The key observation is that for every $t+1$ before the termination of the WHILE-loop, the elements in $U^* - \bigcup_{i=1}^{t} X_i$ can be covered by $k$ sets (e.g. by an optimal set cover), so there exists a set $Y \in \mathcal{S}^*$ such that

$$|Y \cap (U^* \setminus C)| \geq \frac{1}{k} \cdot |U^* \setminus C| \geq \frac{1}{k} \cdot (|U^*| - |C|) = \frac{g_t}{k},$$

where $C = \bigcup_{i=1}^{t} X_i$. As $X_{t+1}$ chosen so as to maximize $|X_{t+1} \setminus C|$, we have

$$g_t - g_{t+1} = |X_{t+1} \setminus C| \geq |Y \setminus C| = |Y \cap (U^* \setminus C)| \geq \frac{g_t}{k},$$

and

$$g_{t+1} \leq (1 - \frac{1}{k}) \cdot g_t \leq (1 - \frac{1}{k})^{t+1} \cdot g_0.$$

Here, notice that $g_0 = |U^*|$.

**Concluding the analysis for MAX COVERAGE.** The output set cover $\{X_1, \ldots, X_k\}$ covers as many as $|\bigcup_{i=1}^{k} X_i| = g_0 - g_k$ elements, which is at least

$$(1 - (1 - \frac{1}{k})^k) \cdot g_0 = (1 - (1 - \frac{1}{k})^k) \cdot |U^*| \geq (1 - \frac{1}{e}) \cdot |U^*|.$$

Therefore, the greedy algorithm is a $(1 - e^{-1})$-approximation for MAX COVERAGE.

**Concluding the analysis for SET COVER.** Let us fix a minimum value $d$ such that $g_d \leq k$, i.e. the number of uncovered elements after $d$-th iteration is at most $k$. Then one can choose at most $k$ additional sets to

cover the remaining $\leq k$ elements. Let $n := |U|$, and note that $g_0 = n$. In order to achieve $g_d \leq k$, it suffices that $d$ satisfies

$$(1 - \frac{1}{k})^d \cdot g_0 = (1 - \frac{1}{k})^d \cdot n \leq k,$$

or equivalently,

$$d \geq -\frac{\ln \frac{n}{k}}{\ln (1 - \frac{1}{k})}.$$

From

$$1 - \frac{1}{k} \leq e^{-1/k},$$

we know that setting $d := k \ln n/k$ satisfies the desired condition. Therefore, the output set cover of the greedy algorithm has size at most

$$k \ln \frac{n}{k} + k = (1 + \ln \frac{n}{\texttt{opt}}) \cdot \texttt{opt} = \alpha(n) \cdot \texttt{opt}$$

for $\alpha(n) = O(\log n)$. To summarize, the algorithm is an $O(\log n)$-approximation for SET COVER.

## 12.5 Greedy algorithm for weighted SET COVER and MAX COVERAGE

We discuss how to handle weights for SET COVER and MAX COVERAGE.

### 12.5.1 The case of WEIGHTED MAX COVERAGE

Notice: the content of this subsection is verbatim the same as the one for unweighted MAX COVERAGE. The content is repeated here to verify and emphasize that the same greedy procedure transfers to the weighted setting with minimal adaptation.

WEIGHTED MAX COVERAGE
**Instance:** a universe $U$ with a weight function $\omega : U \to Q^+$, a collection $\mathcal{S}$ of subsets of $U$, a positive integer $k$.
**Goal:** Find a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ consisting of $k$ sets which maximizes $\sum_{e \in \bigcup_{S \in \mathcal{S}'} S} \omega(e)$

For WEIGHTED MAX COVERAGE, almost the same greedy procedure for the unweighted works with minor modification. Instead of choosing a set which maximizes the number of freshly covered elements, we choose a set which maximizes the weight sum of freshly covered elements. For any subset $X \subseteq U$ of elements, we write $\omega(X)$ to denote $\sum_{e \in X} \omega(e)$.

- Let $\mathcal{S}^*$ be an optimal set cover and let $U^* := \bigcup_{S \in \mathcal{S}^*} S$.
- Let $X_1, \ldots, X_i, \ldots, X_k$ be the chosen sets by the greedy algorithm.
- Let $g_t := \omega(U^*) - \omega(\bigcup_{i=1}^t X_i)$, i.e. the gap between the weight sum of elements that an optimal solution can cover and what the greedy solution covers at $t$.

The key observation is that for every $t + 1$ before the termination of the WHILE-loop, the elements in $U_t^*$ can be covered by $k$ sets (e.g. by an optimal set cover), so there exists a set $Y \in \mathcal{S}^*$ such that

**Algorithm 12** Algorithm for WEIGHTED MAX COVERAGE

```
procedure MaxCoverage(U, ω, S, k)
    Mark every element of U as uncovered.
    S' ← ∅ and C ← ⋃_{S∈S'} S
    while U has an uncovered element and |S'| < k do
        Select X ∈ S \ S' such that ω(X \ C) is as large as possible.
        S' ← S' ∪ {X}
        Mark every element of X covered; C ← C ∪ X.
    return S'.
```

$$\omega(Y \cap (U^* \setminus C)) \geq \frac{1}{k} \cdot \omega(U^* \setminus C) \geq \frac{1}{k} \cdot (\omega(U^*) - \omega(C)) = \frac{g_t}{k},$$

where $C = \bigcup_{i=1}^{t} X_i$. As $X_{t+1}$ chosen so as to maximize $\omega(X_{t+1} \setminus C)$, we have

$$g_t - g_{t+1} = \omega(X_{t+1} \setminus C) \geq \omega(Y \setminus C) = \omega(Y \cap (U^* \setminus C)) \geq \frac{g_t}{k},$$

and

$$g_{t+1} \leq (1 - \frac{1}{k}) \cdot g_t \leq (1 - \frac{1}{k})^{t+1} \cdot g_0.$$

Here, notice that $g_0 = \omega(U^*)$.

**The analysis of Greedy Procedure for MAX COVERAGE.** The output set cover $\{X_1, \ldots, X_k\}$ covers a set of elements of weight sum $g_0 - g_k$, which is at least

$$g_0 - (1 - \frac{1}{k})^k \cdot g_0 = (1 - (1 - \frac{1}{k})^k) \cdot \omega(U^*) \geq (1 - \frac{1}{e}) \cdot \omega(U^*).$$

Therefore, the greedy algorithm is a $(1 - e^{-1})$-approximation for MAX COVERAGE.

### 12.5.2 The case of WEIGHTED SET COVER

WEIGHTED SET COVER
**Instance:** a universe $U$, a collection $\mathcal{S}$ of subsets of $U$ and a cost function $c : \mathcal{S} \to Q^+$.
**Goal:** Find a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ of minimum total cost such that $\bigcup_{S \in \mathcal{S}'} S = U$.

We use a similar greedy procedure that we used for SET COVER. That is, we choose a set $S$ so that the ratio of "the number of newly covered elements" and the cost of the set is maximized. For unweighted SET COVER, the cost was 1 per set. For WEIGHTED SET COVER, the cost will be $c(S)$. For the purpose of the approximation ratio analysis, it turns out that using the reciprocal of this ratio is useful.

Consider a sequence $X_1, \ldots, X_\ell$ of sets in $\mathcal{S}$ such that each set $X_i$ covers at least one new element, i.e. $\tilde{X}_i := X_i \setminus \bigcup_{j<i} X_j$ is non-empty. Let us define the price of an element $e$ covered in this sequence as

$$\mathsf{price}(e) := \frac{c(X_i)}{|\tilde{X}_i|}$$

**Algorithm 13** Algorithm for WEIGHTED SET COVER

```
1: procedure SetCover(U, S, c)
2:     Mark every element of U as uncovered.
3:     S' ← ∅ and C ← ⋃_{S∈S'} S
4:     while U has an uncovered element do
5:         Select X ∈ S \ S' which minimizes c(X)/|X \ C|.
6:         price(e) ← c(X)/|X \ C| for every element e ∈ X \ C.
7:         S' ← S' ∪ {X}
8:         Mark every element of X covered; C ← C ∪ X.
9:     return S'.
```

if $e$ is covered first time by $X_i$. Then the total cost of these sets equals

$$\begin{aligned}\sum_{i=1}^{\ell} c(X_i) &= \sum_{i=1}^{\ell} |\tilde{X}_i| \cdot \frac{c(X_i)}{|\tilde{X}_i|} \\ &= \sum_{i=1}^{\ell} |\tilde{X}_i| \cdot \mathsf{price}(\text{an element of } \tilde{X}_i) \\ &= \sum_{i=1}^{\ell} \sum_{\substack{e \text{ is covered} \\ \text{firstly by } X_i}} \mathsf{price}(e)\end{aligned}$$

**The analysis of Greedy Procedure for WEIGHTED SET COVER.**

Let $X_1, \ldots, X_\ell$ be the sequence produced by the greedy procedure, and consider a total ordering

$$e_1 \prec e_2, \prec \cdots \prec e_n$$

of $U$ such that $\tilde{X}_i \prec \tilde{X}_j$ whenever $i < j$. In other words, $\prec$ follows the ordering of the time when an element is first covered by some set in the sequence, where ties are broken arbitrarily. The key fact for analysis is the following.

**Lemma 12.5.1.** *When the greedy procedure terminates, it holds that*

$$\mathsf{price}(e_k) \leq \frac{c(\mathcal{S}^*)}{n-k+1},$$

*where $\mathcal{S}^*$ is an optimal set cover.*

**Proof:** Let $k'$ be the smallest index such that $\{e_{k'}, \ldots, e_k\}$ are all freshly covered by the set $Z$ chosen in line 6. Note that the subset $\{e_{k'}, e_{k'+1}, \ldots, e_n\}$ is covered by the collection $\mathcal{S}^*$. That is, there is a sequence $S_1, \ldots, S_p$ of sets in a sub-collection $\mathcal{S}^{**}$ of $\mathcal{S}^*$ which minimally covers $\{e_{k'}, e_{k'+1}, \ldots, e_n\}$. Because

$$c(S^{**}) = \sum_{i=1}^{p} c(S_i) = \sum_{i=1}^{p} |\tilde{S}_i| \cdot \frac{c(S_i)}{|\tilde{S}_i|}$$

where $\tilde{S}_i = S_i \cap \{e_{k'}, \ldots, e_n\} \setminus \bigcup_{j<i} S_j$. By Pigeonhole principle, there exists $i^* \in [p]$ such that $c(S_{i^*})/|\tilde{S}_{i^*}| \leq c(S^{**})/(n-k'+1)$. Indeed if not,

$$\sum_{i=1}^{p} |\tilde{S}_i| \cdot \frac{c(S_i)}{|\tilde{S}_i|} > \sum_{i=1}^{p} |\tilde{S}_i| \cdot \frac{c(S^{**})}{n-(k'-1)} = \frac{c(S^{**})}{n-k'+1} \cdot |\bigcup_{i=1}^{p} \tilde{S}_i| = \frac{c(S^{**})}{n-k'+1} \cdot |\{e_{k'}, \ldots, e_n\}| = c(S^{**}),$$

a contradiction.

By the way we choose $Z$ in line 6, $c(S^{**}) \leq c(S^*)$ and $k \geq k'$, we derive

$$\frac{c(Z)}{|Z \setminus C|} = \frac{c(Z)}{|Z \cap \{e_{k'}, \ldots, e_n\}|} \leq \frac{c(S_{i^*})}{|\tilde{S}_{i^*}|} \leq \frac{c(S^{**})}{n-k'+1} \leq \frac{c(S^*)}{n-k+1}.$$

□

With Lemma 12.5.1, we can rewrite the total cost of the set sequence $X_1, \ldots, X_\ell$ produced by the greedy algorithm as

$$\sum_{i=1}^{\ell} c(X_i) = \sum_{i=1}^{n} \mathsf{price}(e_i) \leq \sum_{i=1}^{n} \frac{c(S^*)}{n-k+1} \leq (\ln n + c) \cdot c(S^*)$$

for a small constant $c \leq 1$. That is, the produced set cover is $2 \ln n$-approximation of an optimal set cover.

# Chapter 13

# Layering Technique

## 13.1 2-Approximation for WEIGHTED VERTEX COVER

There are multiple 2-approximation algorithm for WEIGHTED VERTEX COVER, and we see one of them using what's called a layering technique.

WEIGHTED VERTEX COVER
**Instance:** a graph $G = (V, E)$ with a weight function $\omega : V \to Q^+$.
**Goal:** Find a vertex cover of $G$ with the minimum weight.

The essential idea of the layering technique is to decompose the weight function into layers of simple weight function so that when you consider the input with each layer of weight function, there is an easy approximation algorithm. For WEIGHTED VERTEX COVER, a 'simple' weight function is a degree-proportional one. We start with the following observation.

**Lemma 13.1.1.** *For any vertex cover $X$ of $G$, it holds that $\sum_{v \in X} deg_G(v) \geq |E|$. In particular, $\omega(X) \leq 2 \cdot \omega(\texttt{opt})$ for any optimal vertex cover* `opt` *whenever the weight function $\omega : V(G) \to Q^+$ satisfies $\omega(v) = c \cdot deg(v)$ for all $v \in V(G)$ for some constant $c > 0$.*

**Proof:** As every edge $e$ of $E$ is incident with a vertex of $X$, $e$ contributes at least one to the degree sum $\sum_{v \in X} deg_G(v)$ and the inequality follows. □

**Algorithm 14** Algorithm for WEIGHTED VERTEX COVER

```
1: procedure wVC(G, ω)
2:     if G is an edgeless graph then return ∅.
3:     D ← vertices of degree 0.
4:     c ← min_{v∈V(G)\D}{ω(v)/deg(v)}.
5:     ω'(v) ← ω(v) − c · deg(v).
6:     S ← {v ∈ V(G) : ω'(v) = 0}.
7:     return S ∪ wVC(G − S − D, ω').
```

Let us analyze the approximation ratio of the procedure **wVC** on the input $(G, \omega)$. Because $G_{i+1}$ has strictly

smaller number of vertices than $G_i$, the algorithm ends up in an empty graph or an edgeless graph after at most $n$ iterations.

Let $G_0 := G$ for all $i \geq 0$,

- let $G_i = (V_i, E_i)$ be the graph which form the input to the $i$-th recursive call of **wVC** at line 7,
- let $\omega_i$ be the weight function $\omega_i(v) = c_i \cdot deg_{G_i}(v)$ where $c_i$ the constant fixed at line 4 during the execution of the call $\mathbf{wVC}(G_i, \omega_i')$,
- let $S_i$ be the vertex subset of $G_i$ identified at line 8 during the execution of the call $\mathbf{wVC}(G_i, \omega_i')$, and
- $k$ is the index of the last recursive call; $G_k$ edgeless.

Although we defined $\omega_k$, let us set $\omega_k(v) = 0$ for every vertex $v \in V(G_k)$ for notational convenience. As $G_k$ is edgeless, $\omega_k$ is also a degree-proportional weight function on $G_k$ like other weight functions $\omega_i$ for $i < k$.

What is the final output of the procedure **wVC** on $(G, \omega)$? The algorithm will unionize all sets $S_i$ at line 7, so the output is $S := \bigcup_{i=0}^{k} S_i$. Hereinafter, we prove that $S$ is indeed a vertex cover of $G$ and $\omega(S) \leq 2 \cdot \omega(S^*)$, where $S^*$ is an optimal vertex cover of $G$.

• $S$ **is a vertex cover:** Notice that $V = S \cup \bigcup_{i=0}^{k} D_i$. Therefore, if $S$ is not a vertex cover and $e = uv$ is an uncovered edge of $G$, then there exist $i \leq j \leq k$ such that $u \in D_i$ and $v \in D_j$. Note that in this case $v$ is present in $G_i$ and $u$ has a neighbor, so should not have been deleted at line 3.

• $\omega(S) \leq 2 \cdot \omega(S^*)$**:**

- Let $S^*$ be an optimal vertex cover of $(G, \omega)$. Note that $S^* = \mathtt{opt}_0'$.
- Let $\mathtt{opt}_i$ be an optimal vertex cover of $(G_i, \omega_i)$.

We will use the fact for any vertex cover $C$ of $G$ and a vertex subset $Z \subseteq V(G)$, $C \cap Z$ is a vertex cover of $G[Z]$ (why is this true?).

For any vertex $v \in V_0 = V(G)$, we observe

$$(1) \qquad \omega(v) = \sum_{i=0}^{p} \omega_i(v),$$

where $p$ is the largest index such that $v$ appears in $G_p$. Therefore,

$$(2) \qquad \omega(S) = \omega_0(S \cap V_0) + \omega_1(S \cap V_1) + \cdots + \omega_k(S \cap V_k).$$

Due to Lemma 13.1.1 and the fact that both $S \cap V_i$ and $S^* \cap V_i$ are vertex covers of $G_i$, the following holds.

$$\omega_i(S \cap V_i) \leq 2 \cdot \omega_i(\mathtt{opt}_i) \leq 2 \cdot \omega_i(S^* \cap V_i).$$

From the inequalities (1) and (2),

$$\begin{aligned} \omega(S) &\leq 2 \cdot \omega_0(S^* \cap V_0) + 2 \cdot \omega_1(S^* \cap V_1) + \cdots + 2 \cdot \omega_k(S^* \cap V_k) \\ &= 2 \cdot \omega(S^*) \end{aligned}$$

as claimed.

## 13.2 3-Approximation for WEIGHTED FEEDBACK VERTEX SET

Recall that for WEIGHTED VERTEX COVER, the layering technique proposes the following strategy for approximation algorithm: (i) we devise an appropriate 'simple' weight function (degree-proportional for WEIGHTED VERTEX COVER), (ii) prove that a greedy solution (such as any vertex cover in Lemma 13.1.1) is a good approximation w.r.t the proposed weight function, and (iii) design a polynomial-time procedure which decomposes an arbitrary weight function into simple ones and produces a feasible solution in a greedy manner. We follow the same template to design an approximation algorithm for WEIGHTED FEEDBACK VERTEX SET, although the analysis of approximation ratio is slightly more involved.

**Lemma 13.2.1.** *Let $G$ be a graph with $n$ vertices, $m$ edges and $k$ connected components. For any minimal feedback vertex set $X$ of a graph $G$, the following holds.*

1. $|X| \leq m - n + k$.
2. $m - n + k \leq \sum_{v \in X}(deg(v) - 1) \leq 2(m - n) + |X|$ .

**Proof:** Let $F$ be the forest $G - X$. Observe that any forest of $G$, in particular $F$, can be extended into a maximal spanning forest $T$ of $G$ and we have $E(F) \subseteq E(T)$. Moreover, $T$ can be obtained from a forest $F'$ which extends $F$ by adding one edge connecting $v \in X$ to $F$ for each $v \in X$. Therefore, it holds that $|E(T)| \geq |E(F)| + |X|$.

♦ $|X| \leq m - n + 1$ : By $E(F) \subseteq E(T)$, for any $e = uv \in E(G) - E(T)$, at least one of $u$ and $v$ is in $X$. Orient the edges of $E(G) - E(T)$ arbitrarily so that the head of each oriented edge is in $X$. Then every vertex of $X$ must be the head of at least one oriented edge. Indeed, if $v \in X$ is not the head of any oriented edge, then $X \setminus v$ covers all the edges of $E(G) - E(T)$. In particular, deleting $X \setminus v$ from $G$ deletes all edges of $E(G) - E(T)$ (and possibly more) and the remaining set of edges is a subset of $T$, a forest. This contradicts the minimality of $X$. Therefore, every vertex of $X$ is the head of at least one oriented edge, implying that there is a surgection from $E(G) - E(T)$ to $X$. We conclude $|X| \leq |E(G) - E(T)| = m - (n - k)$.

♦ $m - n + 1 \leq \sum_{v \in X}(deg(v) - 1)$**:** Let $Z$ be the set of edges incident with $X$ and note that $|Z| \leq \sum_{v \in X} deg(v)$. Note that $E(G)$ is a disjoint union of $Z$ and $E(F)$, from which we obtain

$$\begin{aligned} |Z| &= |E(G)| - |E(F)| && \because E(G) = Z \dot{\cup} E(F) \\ &= |E(G)| + |X| - (|E(F)| + |X|) \\ &\leq m + |X| - |E(T)| && \because |E(T)| \geq |E(F)| + |X| \\ &= m + |X| - (n - k). \end{aligned}$$

As $|Z| \leq \sum_{v \in X} deg(v)$, it follows

$$\sum_{v \in X}(deg(v) - 1) \geq |Z| - |X| \geq m - (n - k).$$

♦ $\sum_{v \in X}(deg(v) - 1) \leq 2(m - n) + |X|$**:** To see this, we use the equation

$$\sum_{v \in X}(deg(v) - 1) = 2m - \sum_{v \in V \setminus X} deg(v) - |X|$$

and have a good lower bound for the value of $\sum_{v \in V \setminus X} deg(v)$. To the value of $\sum_{v \in V \setminus X} deg(v)$, every edge in the forest $G - X$ contributes 2 and every edge crossing between $X$ and $V \setminus X$ contributes exactly one; the latter type of edges are referred to as crossing edges in the rest of the proof. The value contributed by the forest edges of $G$ is $2 \cdot (n - |X| - \#\text{connected components of } G - X)$ (recall that number of edges in an $n$-vertex forest with exactly $p$ components is $n - p$).

To have a good lower bound on the number of crossing edges, we use the assumption that the min degree of $G$ is at least 2. Under this assumption, the key observation is that each connected component of the forest $G - X$ is incident with at least two crossing edges. Indeed, a component (tree) $T$ of $G - X$ has one crossing edge incident with its vertex set, then as any tree has at least two leaves, at least one leaf of $T$ has degree 1 in $G$. This contradicts the min degree assumption. To sum up, we have $\sum_{v \in V \setminus X} deg(v) \geq 2 \cdot (n - |X| - p) + 2p$, where $p$ is the number of connected components of $G - X$ and

$$\begin{aligned}\sum_{v \in X} (deg(v) - 1) &= 2m - \sum_{v \in V \setminus X} deg(v) - |X| \\ &\leq 2m - 2 \cdot (n - |X|) - |X| \\ &= 2(m - n) + |X|.\end{aligned}$$

□

**Corollary 13.2.1.** *Let $G$ be a graph of minimum degree at least 2 and $\omega : V(G) \to Q^+$ be a weight function which satisfies $\omega(v) = c \cdot (deg(v) - 1)$ for all $v \in V(G)$ for some constant $c > 0$. Then any minimal feedback vertex set $X$ is a 3-approximation of $(G, \omega)$, i.e. $\omega(X) \leq 3 \cdot \omega(\texttt{opt})$, where $\texttt{opt}$ is an optimal feedback vertex set of $(G, \omega)$.*

**Proof:** Let $k$ be the number of connected components of $G$. Observe

$$\begin{aligned}\omega(X) &= \sum_{v \in X} c \cdot (deg(v) - 1) \\ &\leq c \cdot (2m - 2n + |X|) && \because \text{ 2 of Lemma 13.2.1} \\ &\leq c \cdot (2m - 2n + m - n + k) && \because \text{ 1 of Lemma 13.2.1} \\ &\leq 3c \cdot (m - n + k) \\ &\leq 3c \cdot \sum_{v \in \texttt{opt}} (deg(v) - 1) && \because \text{ 2 of Lemma 13.2.1} \\ &= 3 \cdot \omega(\texttt{opt}).\end{aligned}$$

□

**Induction step.** We analyze a single execution of the procedure **wFVS**. Let $G$ be the input graph at the beginning of the procedure, i.e. before deleting vertices at line 2. Let $D$ be the deleted vertices at line 2. The following statement is the key to inductively prove the approximation ratio.

**Lemma 13.2.2.** *Assume that $S'$ returned by $\mathbf{wFVS}(G' = G - S - D, \omega')$ is a feedback vertex set of $G'$ with $\omega'(S') \leq 3 \cdot \omega'(\texttt{opt}')$, where $\texttt{opt}'$ is an optimal feedback vertex set of $(G', \omega')$. Then*

**Algorithm 15** Algorithm for WEIGHTED FEEDBACK VERTEX SET

1: **procedure wFVS**$(G, \omega)$
2: Repeatedly delete vertices of degree at most 1.
3: **if** $G$ is an empty graph **then return** $\emptyset$.
▷ Now every vertex satisfies $deg(v) - 1 \geq 1$.
4: $c \leftarrow \min_{v \in V(G)}\{\omega(v)/(deg(v) - 1)\}$.
5: $\omega'(v) \leftarrow \omega(v) - c \cdot (deg(v) - 1)$.
6: $S \leftarrow \{v \in V(G) : \omega'(v) = 0\}$.
7: Find a minimal fvs $S^o$ of $G$ contained in $S \cup$ **wFVS**$(G - S, \omega')$.
8: **return** $S^o$

1. *there exists a feedback vertex set of $G$ contained in $S \cup S'$, and in particular the output $S^o$ at line 7 is indeed a minimal feedback vertex set of $G$, and*
2. $\omega(S^o) \leq 3 \cdot \omega(\texttt{opt})$*, where* $\texttt{opt}$ *is an optimal feedback vertex set of* $(G, \omega)$.

**Proof:** To show the first statement, it suffices to show that $S \cup S'$ is a feedback vertex set of $G$. If not, then there is a cycle $C$ in $G - (S \cup S')$. As no vertex of $D$ participates in a cycle of $G$, $C$ must be fully contained in $G' - S'$. This contradicts the assumption that $S'$ is a feedback vertex set of $G'$.

To prove the second statement, let $\texttt{opt}$ be an optimal feedback vertex set of $(G, \omega)$ and let $\omega^o(v) := c \cdot (deg_G(v) - 1)$ for all vertices of $G - D$. Notice $\omega(v) = \omega^o(v) + \omega'(v)$ holds for all vertices $v \in V(G) \setminus D$. Moreover, no vertex of $D$ participates in a cycle of $G$ and thus, no vertex of $D$ is contained in a minimal feedback vertex set of $G$. Now

$$
\begin{aligned}
\omega(S^o) &= \omega^o(S^o) + \omega'(S^o) && \because \omega = \omega^o + \omega' \text{ on } S^o \subseteq V(G) \setminus D \\
&= \omega^o(S^o) + \omega'(S^o \cap S) + \omega'(S^o \setminus S) \\
&= \omega^o(S^o) + \omega'(S^o \setminus S) && \because \omega^o(S^o \cap S) = 0 \text{ due to lines 5 and 8} \\
&\leq 3 \cdot \omega^o(\texttt{opt}) + \omega'(S') && \because \text{Corollary 13.2.1 and } S^o \setminus S \subseteq S' \\
&\leq 3 \cdot \omega^o(\texttt{opt}) + 3 \cdot \omega'(\texttt{opt}') && \because \text{Induction hypothesis} \\
&\leq 3 \cdot \omega^o(\texttt{opt}) + 3 \cdot \omega'(\texttt{opt} \cap V(G')) && \because \texttt{opt} \cap V(G') \text{ is a fvs of } G' \\
&\leq 3 \cdot \omega^o(\texttt{opt}) + 3 \cdot \omega'(\texttt{opt}) && \because \texttt{opt} \cap D = \emptyset \\
&= 3 \cdot \omega(\texttt{opt}).
\end{aligned}
$$

□

**Completing the analysis of wFVS.** When $G$ is an empty graph at line 3, it means that before executing line 2 the input graph is acyclic. So the output $\emptyset$ is a valid feedback vertex set. Therefore, the output of $\mathbf{wFVS}(G, \omega)$ is trivially a feedback vertex set of $G$ within triple of an optimal solution value.

Now we apply Lemma 13.2.2 inductively on the depth of the recursive calls, with the induction hypothesis that the precondition ($S'$ returned by ...) of Lemma 13.2.2 is satisfied. By Lemma 13.2.2, the output of the current call $\mathbf{wFVS}(G, \omega)$ is a 3-approximation of an optimal solution. This completes the proof.

## 13.3 Remarks

The textbook and lecture note below make a great presentation on approximation algorithms.

- The design of Approximation Algorithms, David P. Williamson, David B. Shmoys, Cambridge University Press. Can be downloaded at https://www.designofapproxalgs.com/.
- Chandra Chekuri's lecture note on Approximation Algorithms is available here.

# Chapter 14

# Approximation for Cut Problems using Combinatorial Properties

## 14.1 Greedy $(2 - \frac{2}{k})$-approximation for EDGE MULTIWAY CUT

Given an undirected graph $G$ and a set of terminals $T \subseteq V(G)$, an edge multiway cut is a set of edges which pairwise separates $T$. In other words, in the graph $G - X$, there is no $(x, y)$-path for any pair of terminals $x, y \in T$.

EDGE MULTIWAY CUT
**Instance:** an undirected graph $G = (V, E)$ and a set $T \subseteq V$ of $k$ terminals, an edge weight $\omega : E \to Q^+$.
**Goal:** Find a minimum weight edge set $X \subseteq E$ such that any two vertices of $T$ are in distinct connected components of $G - X$.

When $|T| = 2$, the problem is precisely finding a minimum weight $(s, t)$-cut. For $|T| \geq 3$, the (decision version of) EDGE MULTIWAY CUT is NP-complete. While finding a small multiway cut is hard, finding a small weighted cut between a fixed terminal $t$ and the rest of the terminals $T \setminus t$ can be done efficiently using any of the polynomial-time algorithm for MINIMUM $(s, t)$-CUT. Let us call such a cut separating $t \in T$ and $T \setminus t$ an isolating cut for $t$. Clearly, the union of isolating cuts for all terminals is a multiway cut. In fact, you can skip an isolating cut for some arbitrary $t^o \in T$ in the union and the resulting set is still a multiway cut. It turns out that this greedy approach gives $2 - 2/k$-approximation algorithm for EDGE MULTIWAY CUT already.

**Algorithm 16** Algorithm for EDGE MULTIWAY CUT

```
1: procedure edgeMWC(G, T, ω)
2:     for all t ∈ T do
3:         W_t ←minimum weight (t, T \ t)-cut of G.        ▷ W_t can be found in polynomial time.
4:     Let t^o ∈ T be a terminal such that W_{t^o} is maximum.
5:     return ⋃_{t∈T\t^o} W_t.
```

**Analysis of the procedure edgeMWC($G, T, \omega$).** Let $T = \{t_1, \ldots, t_k\}$. Let $S^* \subseteq E$ be an optimal multiway cut and let $S_i^* \subseteq S^*$ be the minimal $(t_i, T \setminus t_i)$-cut contained in $S^*$. Why is a minimal $(t_i, T \setminus t_i)$-cut contained in $S^*$ unique? This is because $G - S^*$ consists of $k$ connected components $V_1, \ldots, V_k$ with $t_i \in V_i$ for each $i \in [k]$ and an edge $e = uv$ is contained in $S^*$ if and only if $u \in V_i$ and $v \in V_j$ for some $i \neq j$. Note that $e$ appears in $S_i^*$ and $S_j^*$, and in no other isolating cut. This implies that

$$\sum_{i=1}^{k} \omega(S_i^*) = 2 \cdot \omega(S^*)$$

because each edge $e$ of $S^*$ appears in precise two isolating cuts $S_i^*$ and $S_j^*$, $i \neq j$.

Let $W_i$ be the isolating cut for $t_i$ found in line 3. Observe

$$\omega(\bigcup_{i=1}^{k} W_i) \leq \sum_{i=1}^{k} \omega(W_i) \leq \sum_{i=1}^{k} \omega(S_i^*) = 2 \cdot \omega(S^*).$$

As we omitted the heaviest isolating set, say $W_k$, in the output solution and $\omega(W_k) \geq \frac{1}{k} \cdot \left( \sum_{i=1}^{k} \omega(W_i) \right)$,

$$\begin{aligned} \omega(\bigcup_{i=1}^{k-1} W_i) &\leq \sum_{i=1}^{k-1} \omega(W_i) \\ &\leq (1 - \frac{1}{k}) \cdot \sum_{i=1}^{k} \omega(W_i) \\ &\leq (1 - \frac{1}{k}) \cdot \sum_{i=1}^{k} \omega(S_i^*) \\ &\leq 2(1 - \frac{1}{k}) \cdot \omega(S^*). \end{aligned}$$

## 14.2 Gomory-Hu Tree

**Alternative views on cuts and submodularity.** For vertex sets $X, Y \subseteq V$, the set of edges with one endpoint in $X$ and another endpoint in $Y$ is denoted by $E(X, Y)$. Note that $E(X, Y)$ also contains all edges whose both endpoints are in $X \cap Y$. When $Y = \bar{X} := V \setminus X$, we write $E(X, \bar{X})$ as $\delta(X)$. Note that $\delta(X) = \delta(\bar{X})$.

A pair of vertex sets $(X, Y)$ is a cut of $G$ if $X \cup Y = V$ and $X \cap Y = \emptyset$. The order of a cut $(X, \bar{X})$ is the cardinality of $\delta(X)$. When the graph $G$ at hand comes with an edge-weight $\omega : E \to \mathbb{R}$, the weight of the cut $(X, \bar{X})$ is defined as the value of $\omega(\delta(X))$. We say that a cut $(X, \bar{X})$ is an $(x, y)$-cut if $|X \cap \{x, y\}| = 1$, or equivalently the edge set $\delta(X)$ is an $(x, y)$-cut of $G$.

We say that two cuts $(X, \bar{X})$ and $(A, \bar{A})$ are crossing if all four sets $X \cap A$, $X \cap \bar{A}$, $\bar{X} \cap A$ and $\bar{X} \cap \bar{A}$ are non-empty. Two cuts are non-crossing if they are not crossing. The following observation is immediate from definition.

**Observation 14.2.1.** *If two cuts $(A_1, A_2)$ and $(B_1, B_2)$ are non-crossing, for some $i, j \in [2]$ it holds $A_i \subseteq B_j$.*

Note that a cut $(X, \bar{X})$ as a vertex bipartition can be uniquely determined by considering the vertex set $X$ or $V \setminus X$. From this perspective, an essential property of cuts called submodularity of cut functions can be observed.

A set function $f : 2^E \to \mathbb{R}$ is submodular if for every sets $A, B \subseteq E$

$$f(A) + f(B) \geq f(A \cap B) + f(A \cup B)$$

holds. For an edge-weighted graph $(G = (V, E), \omega)$, the function $\delta^\omega : 2^V \to \mathbb{R}$ defined as

$$\delta^\omega(S) := \omega(\delta(S)) \qquad \text{for all } S \subseteq V,$$

often known as a cut function, is submodular.

### Gomory-Hu tree.

Let $\alpha : \binom{V}{2} \to Q^+$ be the function mapping every pair $(x, y)$ of vertices of $V$ to the the weight of a minimum $(x, y)$-cut. That is, $\alpha(x, y)$ is the weight of a min $(x, y)$-cut.

**Definition 14.2.1.** *Let $(G = (V, E), \omega)$ be an edge-weighted graph. Consider a tree $(T, \alpha|_{E(T)})$ on the vertex set $V(T) = V(G)$ with an edge-weight inheriting the weight function $\alpha$ restricted on the vertex pairs corresponding to $E(T)$. We say that $(T, \alpha)$ is a Gomory-Hu Tree of $(G, \omega)$ if for every edge $xy$ of $T$,*

$$\omega(V_x, V_y) = \alpha(xy),$$

*where $V_x$ and $V_y$ are the vertex sets of the two connected components of $T - xy$.*

For a tree edge $xy \in T$, there is a corresponding cut $(V_x, V_y)$ of $G$ displayed by the tree edge $xy$, where $V_x$ and $V_y$ are the vertex sets of the two connected components of $T - xy$. Definition 14.2.1 says that for every edge $xy$ of a Gomory-Hu tree, each cut displayed by a $xy$ is a minimum $(x, y)$-cut of $G$. Especially, the $(n-1)$ mininimum cuts displayed by the tree edges pairwise non-crossing. Moreover, Theorem 14.2.1 says that these pairwise non-crossing $n - 1$ cuts (as vertex bipartition) of Gomory-Hu tree describes minimum $(x, y)$-cuts for all pairs $(x, y) \in \binom{V}{2}$.

To summarize, there are two surprising facts about Gomory-Hu cut. First, if a Gomory-Hu tree exists, you know not only minimum cuts for $n - 1$ vertex pairs, but also all possible $\binom{n}{2}$ min cuts of $G$. This is stated in Theorem 14.2.1. Second, Gomory-Hu tree actually exists, the result of Theorem 14.2.2.

**Theorem 14.2.1.** *Let $(T, \alpha)$ be a Gomory-Hu tree of $(G, \omega)$. Then for every vertex pair $(u, v) \in \binom{V}{2}$,*

$$\alpha(u, v) = \textit{the weight of a min } (u, v)\textit{-cut of } (T, \alpha)$$

**Proof:** We first observe the following.

**Claim 14.2.1.** *For any vertices $x, y, z \in V$, $\alpha(x, z) \geq \min\{\alpha(x, y), \alpha(y, z)\}$.*

PROOF OF THE CLAIM: Consider a min $(x, z)$-cut $(X, Z)$, where $x \in X$ and $z \in Z$. If $y \in X$, then $(X, Z)$ is a $(y, z)$-cut and thus $\alpha(x, z) \geq \alpha(y, z)$ holds. Similarly if $y \in Z$, it holds $\alpha(x, z) \geq \alpha(x, y)$. The claimed inequality follows. ◊

If $uv$ is an edge of $T$, the statement holds by definition of Gomory-Hu tree. Suppose that $u = u_0$ and $v = u_\ell$ are non-adjacent in $T$ and let $u_0, u_1 \ldots, u_\ell$ be the $(u, v)$-path of $T$. By Claim 14.2.1,

$$\alpha(u, v) \geq \min\{\alpha(u_i, u_{i+1}) : i = 0, \ldots, \ell - 1\}.$$

On the other hand, any cut $(U_i, \bar{U}_i)$ is a $(u, v)$-cut by definition of Gomory-Hu cut, where $(U_i, \bar{U}_i)$ the cut displayed by the tree edge $u_i u_{i+1}$. This in particular means

$$\alpha(u, v) \leq \min\{\alpha(u_i, u_{i+1}) : i = 0, \ldots, \ell - 1\},$$

thus settling the claimed equation. □

The next is an immediate consequence of Theorem 14.2.1.

**Corollary 14.2.1.** *Let $(T, \alpha)$ be a Gomory-Hu tree of $(G, \omega)$. Then for every vertex pair $(u, v) \in \binom{V}{2}$, $E(V_x, V_y)$ is a min $(u, v)$-cut, where $xy$ is the lightest edge on the $(u, v)$-path of $T$.*

### Gomory-Hu tree exists.

**Theorem 14.2.2.** *For any edge-weighted graph $(G, \omega)$, a Gomory-Hu tree $(T, \alpha)$ exists. Moreover, a Gomory-Hu tree can be constructed in polynomial time.*

We present a couple of technical lemmas that are essential for proving Theorem 14.2.2.

**Lemma 14.2.1.** *Let $s, t \in V$ be two arbitrary vertices of $G$ and $(S, \bar{S})$ be a minimum $(s, t)$-cut of $G$. Then for any $u, w \in S$, there exists a minimum $(u, w)$-cut $(U, \bar{U})$ which is non-crossing with $(S, \bar{S})$*

**Proof:** Let $(U, \bar{U})$ be a min $(u, w)$-cut of $G$ and without loss of generality we assume $u \in U$,$w \in \bar{U}$, $s \in S$ and $t \in \bar{S}$. If $(U, \bar{U})$ and $(S, \bar{S})$ are non-crossing, the statement vacuously holds. So, suppose they are crossing. Note that $s$ either belongs to $U$ or $\bar{U}$, and without loss of generality we may assume that $s \in U$ (otherwise we exchange the roles of $u$ and $w$ in the subsequent proof). To summarize, the setup is as follows.

- $u \in S \cap U$, $w \in S \setminus U$, and
- $s \in S \cap U$, $t \in \bar{S}$.

There are two possibilities depending on whether $t$ belongs to $U$ or not. Let $\delta^\omega : V \to Q^+$ be defined by $\delta^\omega(W) := \omega(\delta(W))$ and observe that $\delta^\omega$ is submodular.

**Case when $t \notin U$.** By submodularity of $\delta^\omega$,

$$\delta^\omega(S) + \delta^\omega(U) \geq \delta^\omega(S \cap U) + \delta^\omega(S \cup U).$$

Because $\delta(S \cap U)$ and $\delta(S \cap U)$ are a $(u, v)$-cut and an $(s, t)$-cut of $G$ respectively, we have

$$\delta^\omega(S \cap U) \geq \delta^\omega(U)$$

and

$$\delta^\omega(S \cup U) \geq \delta^\omega(S).$$

This implies that $\delta^\omega(S \cap U) = \delta^\omega(U)$ (and $\delta^\omega(S \cup U) = \delta^\omega(S)$ as well), or equivalently, the cut $(S \cap U, S \bar{\cap} U)$ is a minimum $(u, v)$-cut of $G$. It remains to observe that $(S \cap U, S \bar{\cap} U)$ and $(S, \bar{S})$ are non-crossing.

**Case when $t \in U$.** In this case, we apply the submodularity to the sets $S$ and $\bar{U}$. That is,

$$\delta^\omega(S) + \delta^\omega(\bar{U}) \geq \delta^\omega(S \cap \bar{U}) + \delta^\omega(S \cup \bar{U}).$$

Now that $\delta(S \cap \bar{U})$ and $\delta(S \cup \bar{U})$ are a $(u,v)$-cut and an $(s,t)$-cut of $G$ respectively, and consequently $\delta^\omega(S \cap \bar{U}) = \delta^\omega(U)$. That is $(S \setminus U, S \setminus U)$ is a minimum $(u,v)$-cut, and it is non-crossing with $(S, \bar{S})$. □

**Proof of Theorem 14.2.2:**

To demonstrate the existence of a Gomory-Hu tree, we shall construct a sequence of trees $T_i$ on a set $R_i \subseteq V$ of $i$ nodes for $i = 1$ up to $n$ while maintaining the following invariant:

- (†) there is a surjective mapping $\varphi_i : V \to R_i$ from the vertices of $G$ onto $R_i$ such that $\varphi(r) = r$ for every $r \in R_i$,
- ($\star$) for each tree edge $e = xy$ of $T_i$, the cut $(U, \bar{U})$ of $G$ displayed by $e$ is a minimum $(x,y)$-cut of $G$. Here, $U = \bigcup_{r \in K} \varphi^{-1}$ and $K \subseteq R_i$ is a connected component of $T_i - e$.

Clearly, $(T_n, \alpha|_{E(T)})$ is a Gomory-Hu tree of $(G, \omega)$.

Let $R_1 = \{s\}$ where $s$ is an arbitrary vertex of $G$. The trivial tree $T_1$ on $\{s\}$ with $\varphi_1(V) = \{s\}$ satisfies the invariants (†) and ($\star$) trivially. Assume that $T_i$ is a tree on $R_i \in V$ satisfying (†) and ($\star$) with a mapping $\varphi_i$ for $i < n$. Choose a vertex $x \in R_i$ such that $\varphi^{-1}(x)$ contains at least two vertices of $G$ and let $y$ be an arbitrary vertex of $\varphi_i^{-1}(x) \setminus x$. Let $(U, \bar{U})$ be a minimum $(x,y)$-cut of $G$.

We claim that there is a minimum $(x,y)$-cut $(U^*, \bar{U}^*)$ which is non-crossing with each cut of $G$ displayed by a tree edge in $T_i$ incident with $x$. Once such an $(x,y)$-cut $(U^*, \bar{U}^*)$ is found, we refine $T_i$ into $T_{i+1}$ as follows. Without loss of generality, we assume $x \in U^*$.

1. Let $w_1, \ldots, w_\ell \in R_i$ be the neighbors of $x$ in $T_i$.
2. Let $W_j \subseteq V$ be the vertex set $\bigcup_{r \in K_j} \varphi_i^{-1}(r)$, where $K_j$ is the connected component of $T_i - x$ containing $w_j$. If we see $T_i$ as a tree rooted at $x$ and thus $w_j$'s as the children of $x$ in $T_i$, $W_j$ is the set of all vertices of $V$ which are assigned to the subtree rooted at $w_j$.
3. Classify each vertex $w_j$ into one of the two parts depending on whether the corresponding set $W_j$ is fully contained in $U^*$ or in $\bar{U}^*$. Recall that one of the two situations should occur, see Observation 14.2.1.
4. Let $J \subseteq [\ell]$ be the set of children $w_j$ such that $W_j \in U^*$.
5. Let $R_{i+1} = R_i \cup \{y\}$.
6. $T_{i+1}$ is obtained from $T_i$ by substituting the node $x$ by the edge $xy$ and
   - each subtree rooted at $w_j$ with $j \in J$ is a child of $x$, (recall $x \in U^*$ and $y \in \bar{U}^*$)
   - each subtree rooted at $w_j$ with $j \in [\ell] \setminus J$ is a child of $y$.
7. Finally, $\varphi_{i+1}(v) = x$ for all $v \in \varphi_i^{-1}(x) \cap U^*$ and $\varphi_{i+1}(v) = y$ for all $v \in \varphi_i^{-1}(x) \setminus U^*$, and for all other vertices of $G$ assigned to a tree node other than $x$ by $\varphi_i$, we keep the same assignment in the new $\varphi_{i+1}$.

It is tedious to verify that the newly constructed structure $(R_{i+1}, T_{i+1}, \varphi_{i+1})$ meets the invariants $(\dagger)$ and $(\star)$. Moreover, such a cut $(U^*, \bar{U}^*)$ can be computed by a single application of minimum cut algorithm (How?). This complete the proof of the theorem.

Note that the proof is constructive and can be turned into an efficient algorithm, provided that a desired cut $(U^*, \bar{U}^*)$ above can be found in polynomial time. This is left as an exercise for the readers. □

## 14.3 Greedy $(2 - \frac{2}{k})$-approximation for $k$-CUT

$k$-CUT
**Instance:** an undirected graph $G = (V, E)$, an edge weight $\omega : E \to Q^+$ and a positive integer $k$.
**Goal:** Find a minimum weight edge set $X \subseteq E$ such that $G - X$ consists of $k$ connected components.

Using the result of the previous subsection, we present a greedy approximation algorithm for $k$-CUT with approximation ratio $(2 - 2/k)$.

**Algorithm 17** Algorithm for $k$-CUT

```
1: procedure kCUT(G, ω)
2:     Compute a Gomory-Hu Tree T of G.
3:     Let f_1, ..., f_{k-1} be a set of k − 1 lightest edge in T.
4:     Let W_1, ..., W_{k-1} be the edge sets of G displayed by f_1, ..., f_{k-1}.
5:     return ⋃_{i=1}^{k-1} W_i.
```

**Analysis of the procedure $k$CUT$(G, T, \omega)$.** Let $S^* \subseteq E$ be an optimal $k$-cut of $G$ and

- let $V_i$ for $1 \leq i \leq k$ be the connected components of $G - S^*$,
- let $S_i^* := E(V_i, V \setminus V_i)$ for $1 \leq i \leq k$.

Without loss of generality, we assume that $\omega(S_i^*) \leq \omega(S_k^*)$ for every $i \in [k]$. Clearly, it holds that $\omega(S_k^*) \geq (1/k) \cdot \sum_{i=1}^{k} \omega(S_i^*)$. For now, let us assume that $G$ is connected. The case when $G$ is not connected will be discussed at the end of the analysis.

Consider a Gomory-Hu tree $(T, \alpha)$ of $(G, \omega)$. Because $T$ spans the vertex set of $G$, there is a subset $F \subseteq E(T)$ of $k - 1$ edges of $T$ so that the following holds:

- for every $i \in [k-1]$, $F$ has exactly one edge $e_i = (v_i, v_i')$ such that $v_i \in V_i$ and $v_i' \in V \setminus V_i$, and
- the graph $H$ obtained from $(V, F)$ by identifying each $V_i$, $i \in [k]$, into a single vertex is a tree on $\{V_1, \ldots, V_k\}$.

Now, the key fact here is that for each $i \in [k-1]$, $S_i^*$ separates $V_i$ and $V \setminus V_i$. This in particular means that $S_i^*$ is a $(v_i, v_i')$-cut of $G$, and consequently, $\alpha(v_i, v_i') \leq \omega(S_i^*)$. As the edges $e_i$ are all distinct edges of $T$, and $W_1, \ldots, W_{k-1}$ chosen during the the procedure $k$CUT$(G, T, \omega)$ are the $k - 1$ lightest edges of $T$, we have the lower bound

$$\sum_{i=1}^{k-1} \omega(W_i) = \sum_{i=1}^{k-1} \alpha_T(f_i) \leq \sum_{i=1}^{k-1} \alpha_T(e_i) \leq \sum_{i=1}^{k-1} \omega(S_i^*).$$

From $\sum_{i=1}^{k} \omega(S_i^*) = 2 \cdot \omega(S^*)$ and $\omega(S_k^*) \geq (1/k) \cdot \sum_{i=1}^{k} \omega(S_i^*)$, we conclude

$$\omega(\bigcup_{i=1}^{k-1} W_i) \leq \sum_{i=1}^{k-1} \omega(W_i) \leq 2(1 - \frac{1}{k}) \cdot \omega(S^*).$$

Notice that the first equation holds by definition of Gomory-Hu tree.

It remains to see that $\bigcup_{i=1}^{k-1} W_i$ is indeed a $k$-cut of $G$. By definition of Gomory-Hu Tree, with each addition of $W_i$ to the output solution we separate a new vertex pair, thus strictly increasing the number of connected components. Therefore, $G - \bigcup_{i=1}^{k-1} W_i$ has at least $k$ connected components.

**Remark on the case when $G$ is not connected.** Note that Gomory-Hu tree can be constructed regardless of whether $G$ is connected or not, and distinct connected components are displayed by a tree edge with zero weight. The same analysis goes through in the presence of multiple connected components in $G$.

# Chapter 15

# Linear Program, Duality Theorem and LP-based rounding

## 15.1 2-Approximation for WEIGHTED VERTEX COVER via LP rounding

The integer program for WEIGHTED VERTEX COVER is formulated as follows.

$$\begin{aligned} \min \sum_{u \in V(G)} & \omega_u \cdot x_u \\ x_u + x_v &\geq 1 && \forall (u,v) \in E(G) \\ x_u &\in \{0,1\} && \forall u \in V(G) \end{aligned}$$

The linear program relaxation of the above formulation can be obtained simply by replacing the constraint $x_u \in \{0,1\}$ by $x_u \geq 0$. Notice that we dropped the constraint $x_u \leq 1$ as the objective is to minimize $\sum_{u \in V(G)} \omega_u \cdot x_u$ for non-negative coefficients while the constraint will be satisfied once any of the two variables involved in the constraint has value at least 1.

The algorithm is very simple. Solve the LP for WEIGHTED VERTEX COVER, and let $x^*$ be a fractional optimal solution. Output the set $S := \{v \in V : x^*_v \geq 0.5\}$.

**Analysis of the deterministic rounding algorithm.** Let $S$ be the output vertex set.

First, observe that $S$ is indeed a vertex cover of $G$ because if this is not true, there exists an edge $e = uv$ such that $x^*_u + x^*_v < 1$. This is impossible as $x^*$ is a feasible solution to the LP.

Second, we argue that $\omega(S) \leq 2 \cdot \omega(\texttt{opt})$. This is because $2 \cdot x^*_v \geq 1$ for every $v \in S$, and

$$\omega(S) = \sum_{v \in S} \omega_v \cdot 1 \leq \sum_{v \in S} \omega_v \cdot (2x^*_v) \leq 2 \cdot \sum_{v \in V} \omega_v \cdot x^*_v \leq 2 \cdot \omega(\texttt{opt}).$$

## 15.2 Linear Program and Duality

See Chapter 12 of the textbook "Approximation Algorithms" by Vazirani.

## 15.3 $(s,t)$-mincut via LP rounding and Max-Flow Min-Cut Theorem

The material in this subsection is based on Chandra Chekrui's lecture note, Chapter 14. We consider a directed graph $G = (V, E)$ with a fixed pair of vertices, $s$ and $t$ of $G$.

Let $\mathcal{P}_{s,t}$ be the collection of all $(s,t)$-path in $G$. The linear program for MINIMUM $(s,t)$-CUT is formulated as follows.

$$\begin{aligned} \min \sum_{e \in E(G)} \omega_e \cdot x_e & \\ \sum_{e \in P} x_e \geq 1 & \qquad \forall P \in \mathcal{P}_{s,t} \\ x_e \geq 0 & \qquad \forall e \in E(G) \end{aligned}$$

The dual LP of the linear program for MINIMUM $(s,t)$-CUT is:

$$\begin{aligned} \max \sum_{P \in \mathcal{P}_{s,t}} z_P & \\ \sum_{\substack{P \in \mathcal{P}_{s,t}: \\ P \text{ traverses } e}} z_P \leq \omega_e & \qquad \forall e \in E(G) \\ z_P \geq 0 & \qquad \forall P \in \mathcal{P}_{s,t} \end{aligned}$$

The dual LP program seeks to find a maximum flow; fractional assignment of non-negative value on each $(s,t)$-path so that the total flow on each edge $e$ does not exceed its capacity $c_e$. Therefore, by the strong LP duality theorem it holds that

$$\text{max flow value} = \text{weight of min fractional cut} \leq \text{weight of min cut.}$$

In fact, we will show that there is a (integral) minimum $(s,t)$-cut whose weight matches the weight of a fractional minimum $(s,t)$-cut. We show this result using a randomized rounding of an LP solution, which can be easily derandomized. Once this is established, the above inequality implies that there is a minimum $(s,t)$-cut whose weight matches the maximum flow value. This is precisely Max-Flow Min-Cut Theorem.

**Algorithm 18** Randomized Rounding for MINIMUM $(s,t)$-CUT

1: **procedure** $\theta$**-mincut**$(G, \omega, s, t)$
2: Solve LP for MINIMUM $(s,t)$-CUT to obtain an optimal fractional solution $x^*$.
3: Compute the distance $\texttt{dist}_G(s,v)$ for every $v \in V$ with $x^*_e$ interpreted as the length of edge $e$.
4: Choose $\theta \in (0,1)$ uniformly at random.
5: Let $S := B(s,\theta) \subseteq V$ be the set of vertices $v$ with $\texttt{dist}_G(s,v) \leq \theta$.
6: **return** $\delta(S)$.

**Analysis of the procedure $\theta$-mincut$(G, \omega, s, t)$.** We argue that the output edge set at line 17 is indeed an $(s,t)$-cut, and the expected cost of an output $(s,t)$-cut of the procedure is at most $\sum_{e \in E} \omega_e \cdot x^*_e$, namely the optimal objective value of the LP. This implies that there is an $(s,t)$-cut whose weight equals $\sum_{e \in E} \omega_e \cdot x^*_e$.

**Lemma 15.3.1.** *The output set of the procedure $\theta$-**mincut**$(G, \omega, s, t)$ is an $(s,t)$-cut.*

**Proof:** Note that an edge set $\delta(S)$ for a vertex set $S \subseteq V$ is an $(s,t)$-cut if and only if $|S \cap \{s,t\}| = 1$. As $\texttt{dist}_G(s,s) = 0 \leq \theta$ and $B(s,\theta) \subseteq V$ contains $s$ already, it suffices to argue that $t \notin B(s,\theta) \subseteq V$. Indeed, if $t$ is contained in $B(s,\theta) \subseteq V$, then $\texttt{dist}_G(s,t) \leq \theta < 1$. This means that there is an $(s,t)$-path $P$ in $G$ such that $\sum_{e \in E(P)} x^*_e < 1$. Then the constraint of the primal LP corresponding to $P$ is violated by $x^*$, contradicting to the feasibility of $x^*$. □

**Lemma 15.3.2.** *The expected weight of an output set is at most $\sum_{e \in E} \omega_e \cdot x^*_e$.*

**Proof:** The expected weight of an output set is

$$\sum_{e=xy \in E} \omega_e \cdot \texttt{prob}[\texttt{dist}_G(s,x) \leq \theta \text{ and } \texttt{dist}_G(s,y) > \theta]$$

and the event of "$\texttt{dist}_G(s,x) \leq \theta$ and $\texttt{dist}_G(s,y) > \theta$" will happen when $\theta$ is chosen in the interval $[\texttt{dist}_G(s,x), \texttt{dist}_G(s,y))$. The probability of this event is precisely

$$\frac{\texttt{dist}_G(s,y) - \texttt{dist}_G(s,x)}{\text{the length of the interval that } \theta \text{ is chosen from}}.$$

As $\texttt{dist}_G(s,y) \leq \texttt{dist}_G(s,x) + x^*_e$, the probability of the said event is at most $x^*_e$. This yields the desired expectation. □

Lemmas 15.3.1 and 15.4.2 combined implies that there exists an $(s,t)$-cut whose weight is at most $\sum_{e \in E} \omega_e \cdot x^*_e$. As an $(s,t)$-cut, or equivalently an integral feasible solution to the LP, is lower bounded by the optimal fractional objective value which equals $\sum_{e \in E} \omega_e \cdot x^*_e$. Therefore, such an $(s,t)$-cut is a minimum $(s,t)$-cut. By the Strong LP duality Theorem, we conclude that the weight of a minimum $(s,t)$-cut equals the maximum flow value; indeed the dual LP formulates the maximum flow problem.

## 15.4 $(2 - \frac{2}{k})$-approximation for EDGE MULTIWAY CUT via LP rounding

Let $T = \{t_1, \ldots, t_k\}$ be the set of terminals and for $1 \leq i < j \leq k$, let $\mathcal{P}_{i,j}$ be the collection of all $(t_i, t_j)$-path in $G$. The linear program for EDGE MULTIWAY CUT is formulated as follows.

$$\begin{aligned}
\min \sum_{e \in E(G)} & \omega_e \cdot x_e & & \\
\sum_{e \in P} & x_e \geq 1 & \forall P \in \mathcal{P}_{i,j} \quad & \text{for all } 1 \leq i < j \leq k \\
& x_e \geq 0 & & \forall e \in E(G)
\end{aligned}$$

**Lemma 15.4.1.** *The output set of the procedure $\theta$-**mwc**$(G, \omega, T)$ is a multiway cut.*

**Proof:** Clearly, $t_i \in S_i$ for each $i \in [k]$. Moreover, $S_i$ does not contain any terminal $t_j$ for $j \neq i$ because $\texttt{dist}_G(t_i, t_j) \geq 1 > \theta$. Therefore $\delta(S_i)$ is an isolating cut for $t_i$, i.e. an $(t_i, T \setminus t_i)$-cut of $G$. It follows that $\bigcup_{i=1}^k \delta(S_i)$ pairwise separates all terminals of $T$. □

**Lemma 15.4.2.** *The expected weight of an output set is at most $2 \cdot \sum_{e \in E} \omega_e \cdot x^*_e$.*

**Algorithm 19** Randomized Rounding for EDGE MULTIWAY CUT

**procedure** $\theta$-**mwc**$(G, \omega, T)$
  Solve LP for MINIMUM $(s,t)$-CUT to obtain an optimal fractional solution $x^*$.
  For each $i \in [k]$ and $v \in V$, compute $\texttt{dist}_G(t_i, v)$ with $x^*_e$ interpreted as the length of edge $e$.
  Choose $\theta \in (0, \frac{1}{2})$ uniformly at random.
  For each $i \in [k]$, let $S_i := B(t_i, \theta) \subseteq V$ be the set of vertices $v$ with $\texttt{dist}_G(t_i, v) \leq \theta$.
  **return** $\bigcup_{i=1}^{k} \delta(S_i)$.

**Proof:** The expected weight of an output set is at most

$$\sum_{e=xy \in E} \omega_e \cdot \texttt{prob}[\texttt{dist}_G(t,x) \leq \theta \text{ and } \texttt{dist}_G(t,y) > \theta \text{ for some } t \in T],$$

where $x$ is an endpoint of $e$ which not farther from $t_i$ than the other endpoint $y$ is, and this choice can be different for a fixed edge $e$ depending on the terminal $t_i$.

To analyze the probability of the event $[\texttt{dist}_G(t,x) \leq \theta \text{ and } \texttt{dist}_G(t,y) > \theta \text{ for some } t \in T]$, we define $B_i^{0.5}$ as the set of vertices $v$ with $\texttt{dist}_G(t_i, v) < 0.5$.

There are a few cases to consider for the type of an edge $e = xy$. The cases are exclusive, and we assume that none of the preceding cases are applicable for the later case.

1. an endpoint of $e$, say $x$, belongs to at least two balls $B_i^{0.5}$ and $B_j^{0.5}$.
2. (1 is not applicable, and) $x \in B_i^{0.5}$ and $y \in B_j^{0.5}$ for $i \neq j$.
3. (1-2 are not applicable, and) there exists a unique ball $B_i^{0.5}$ such that $B_i^{0.5} \cap \{x, y\} \neq \emptyset$.
4. (1-3 are not applicable, and) there is no ball $B_i^{0.5}$ such that $B_i^{0.5} \cap \{x, y\} \neq \emptyset$.

Type 1 edges do not exist because if $x \in B_i^{0.5} \cap B_j^{0.5}$, then $\texttt{dist}_G(t_i, t_j) < 1$ and $x^*$ is infeasible. Consider a Type 2 edge $e = xy$ and note that $\texttt{dist}_G(t_i, y) > 0.5$ and $\texttt{dist}_G(t_j, x) > 0.5$. Therefore,

$$\begin{aligned}
&\texttt{prob}[\texttt{dist}_G(t,x) \leq \theta \text{ and } \texttt{dist}_G(t,y) > \theta \text{ for some } t \in T] \\
&\leq \texttt{prob}[\texttt{dist}_G(t_i,x) \leq \theta \text{ and } \texttt{dist}_G(t_i,y) > \theta] + \texttt{prob}[\texttt{dist}_G(t_j,y) \leq \theta \text{ and } \texttt{dist}_G(t_j,x) > \theta]\} \\
&\leq \texttt{prob}[\texttt{dist}_G(t_i,x) \leq \theta] + \texttt{prob}[\texttt{dist}_G(t_j,y) \leq \theta]\} \\
&= \frac{0.5 - \texttt{dist}_G(t_i,x)}{\text{the length of the interval that } \theta \text{ is chosen from}} + \frac{0.5 - \texttt{dist}_G(t_i,x)}{\text{the length of the interval that } \theta \text{ is chosen from}} \\
&\leq 2 \cdot (1 - \texttt{dist}_G(t_i,x) - \texttt{dist}_G(t_j,y)) \\
&\leq 2 \cdot x^*_e
\end{aligned}$$

where the last inequality comes from $\texttt{dist}_G(t_i, x) + x^*_e + \texttt{dist}_G(t_j, y) \geq \texttt{dist}_G(t_i, t_j) \geq 1$.

For Type 3 edge $e$, the same analysis for the randomized rounding for MINIMUM $(s,t)$-CUT easily yields the probability bound at most $2x^*_e$. Notice that both endpoints or exactly one endpoint of $e$ may lie in $B_i^{0.5}$. For Type 4 edge $e$, the probability is simply zero.

To sum up,

$$\sum_{e=xy \in E} \omega_e \cdot \texttt{prob}[\texttt{dist}_G(t,x) \leq \theta \text{ and } \texttt{dist}_G(t,y) > \theta \text{ for some } t \in T] \leq 2 \cdot \sum_{e=xy \in E} \omega_e \cdot x^*_e,$$

which completes the proof. □

One can subtract the factor $2/k$ from the approximation ratio by not considering some terminal in both the approximation algorithm and in the analysis.

## 15.5 2-Approximation for EDGE MULTICUT ON TREES via LP rounding

EDGE MULTICUT ON TREES
**Instance:** a tree $T = (V, E)$, a set $k$ terminal pairs $\{(s_1, t_1), \ldots, (s_k, t_k)\}$ and edge weight $\omega : E \to Q^+$.
**Goal:** Find a minimum weight edge set $X \subseteq E$ such that no terminal pair $(s_i, t_i)$ has a path in $T - X$.

EDGE MULTICUT ON TREES is NP-complete even on a tree of height 1 (star) and uniform weight on the edges. In this section, we present a 2-approximation algorithm for EDGE MULTICUT ON TREES based on a simple randomized rounding of LP solution.

Note that any pair of vertices on a tree is connected by a unique path. Let $P_i$ be the unique $(s_i, t_i)$-path on $T$. The linear program for EDGE MULTICUT ON TREES is formulated as follows.

$$\begin{aligned} \min \sum_{e \in E(G)} \omega_e \cdot x_e & \\ \sum_{e \in P_i} x_e \geq 1 & \qquad \forall i \in [k] \\ x_e \geq 0 & \qquad \forall e \in E(G) \end{aligned}$$

Let $x^*$ be an optimal fractional multicut, that is, an optimal solution to the above LP. Again, we interpret $x^*_e$ as the edge length on $e$. We construe $T$ as a rooted tree, and let $r$ be the (arbitrarily chosen) root of $T$. With $x^* : E \to Q^+$ interpreted as the edge length function[1] we define the distance of a vertex $v \in V$ from the root in the usual way:

$$\texttt{dist}_{x^*}(u, v) := \sum_{e \in E(P_{u,v})} x^*_e,$$

where $P_{u,v}$ is the unique $(u, v)$-path on $T$.

Choose $\theta \in (0, \frac{1}{2})$ uniformly at random. Let $B(r, d)$ be the set of all vertices $v$ such that $\texttt{dist}_{x^*}(r, v) \leq d$. We cut all the edges in $\delta(B(r, \theta))$, $\delta(B(r, \theta + \frac{1}{2}))$, $\delta(B(r, \theta + 1))$, $\delta(B(r, \theta + \frac{3}{2}))$. We argue that the set of deleted edges, denoted as $D$, is a multicut within $2 \cdot \omega(\texttt{lp opt})$.

The next observation is immediate from $\texttt{dist}_{x^*}(s_i, \ell_i) + \texttt{dist}_{x^*}(t_i, \ell_i) = \texttt{dist}_{x^*}(s_i, t_i) \geq 1$ and the feasibility of $x^*$.

**Observation 15.5.1.** *Let $\ell_i$ be the least common ancestor of $s_i$ and $t_i$ on $T$. It holds that*

$$\max\{\texttt{dist}_{x^*}(s_i, \ell_i), \texttt{dist}_{x^*}(t_i, \ell_i)\} \geq 0.5.$$

[1] When all coefficients are rational, there is a rational LP optimal solution and the usual polynomial-time algorithm finds such a solution.

**Algorithm 20** Randomized Rounding for EDGE MULTICUT ON TREES

**procedure** $\theta$-**mwctree**$(G, \omega, T)$
  Solve LP for EDGE MULTICUT ON TREES to obtain an optimal fractional solution $x^*$.
  Compute the distance $\texttt{dist}_{x^*}(r, v)$ for each $v \in V$.
  Choose $\theta \in (0, \frac{1}{2})$ uniformly at random.
  For $i \in \mathbb{N}$, let $S_i := B(r, \theta + i \cdot \frac{1}{2}) \subseteq V$ be the set of vertices $v$ with $\texttt{dist}_{x^*}(r, v) \leq \theta + \frac{1}{2} \cdot i$.
  **return** $\bigcup_{i=1}^{\infty} \delta(S_i)$.

**Lemma 15.5.1.** *The edge set $B$ is a multicut.*

**Proof:** It suffices to argue that for each $i \in [k]$, at least one of the $(s_i, \ell_i)$-path and $(t_i, \ell_i)$-path is cut by $D$. By Observation 15.5.1, we may assume that $\texttt{dist}_{x^*}(s_i, \ell_i) \geq 0.5$ without loss of generality. Let $p$ be the least integer such that $\ell_i \in B(r, \theta + p \cdot \frac{1}{2})$.

If $p = 0$, then it holds that $\texttt{dist}_{x^*}(r, s_i) \geq \texttt{dist}_{x^*}(\ell_i, s_i) \geq 0.5 > \theta \geq \texttt{dist}_{x^*}(r, \ell_i)$, implying that $s_i \notin B(r, \theta)$ and the $(\ell_i, s_i)$-path is cut by $D$.

If $p \geq 1$, observe

$$\texttt{dist}_{x^*}(r, s_i) = \texttt{dist}_{x^*}(r, \ell_i) + \texttt{dist}_{x^*}(\ell_i, s_i) > \big(\theta + (p-1) \cdot \frac{1}{2}\big) + \frac{1}{2} = \theta + p \cdot \frac{1}{2}.$$

This again implies that $s_i \notin B(r, \theta + p \cdot \frac{1}{2})$ and the $(\ell_i, s_i)$-path is cut by $D$. □

**Lemma 15.5.2.** *The expected weighted of the chosen edge set is at most $2 \cdot \sum_{e \in E} \omega_e \cdot x^*_e$.*

**Proof:** The probability that each edge is chosen for $D$ is at most $2 \cdot x^* e$ and the statement follows. □

## 15.6 Half-integrality of VERTEX MULTIWAY CUT via Complementary Slackness

VERTEX MULTIWAY CUT
**Instance:** an undirected graph $G = (V, E)$, a set $T \subseteq V$ of $k$ terminals $t_1, \ldots, t_k$, a vertex weight $\omega : V \setminus T \to Q^+$.
**Goal:** Find a minimum weight vertex set $X \subseteq V \setminus T$ such that any two vertices of $T$ are in distinct connected components of $G - X$.

Let $T = \{s_1, \ldots, s_k\}$ be the set of terminals and for $1 \leq i < j \leq k$, let $\mathcal{P}_{i,j}$ be the collection of all $(t_i, t_j)$-path in $G$. The collection $\mathcal{P}$ is the union of $\mathcal{P}_{i,j}$ for all $1 \leq i < j \leq k$. The linear program for VERTEX MULTIWAY CUT is formulated as follows.

$$\begin{aligned}
\min \sum_{v \in V \setminus T} \omega_v \cdot x_v & \\
\sum_{v \in V(P) \setminus T} x_v \geq 1 & \qquad \forall P \in \mathcal{P} \\
x_v \geq 0 & \qquad \forall v \in E(G)
\end{aligned}$$

The dual LP for VERTEX MULTIWAY CUT is:

$$\begin{aligned} \max \sum_{P \in \mathcal{P}} f_P & \\ \sum_{\substack{P \in \mathcal{P}: \\ P \text{ traverses } v}} f_P \leq \omega_v & \qquad \forall v \in V \setminus T \\ f_P \geq 0 & \qquad \forall P \in \mathcal{P} \end{aligned}$$

We prove that the LP for VERTEX MULTIWAY CUT always has a half-integral solution, that is, an optimal solution to the primal LP such that each variable is assigned with a value in $\{0, 0.5, 1\}$.

**Theorem 15.6.1.** *There is an optimal solution $x^{**}$ to LP for* VERTEX MULTIWAY CUT *such that $x^{**}_v \in \{0, 0.5, 1\}$ for every $v \in V \setminus T$. Moreover, such an optimal solution can be constructed in polynomial time.*

**Corollary 15.6.1.** *In polynomial time, one can find a vertex multiway cut $S$ to an input $(G, T, \omega)$ such that $\omega(S) \leq 2 \cdot \sum_{v \in V \setminus T} \omega_v \cdot x^*_v$, where $x^*$ is an optimal solution to LP of* VERTEX MULTIWAY CUT.

**Proof:** We apply Theorem 15.6.1 and find a half-integral optimal LP solution $x^*$. Let $X = \{v \in V \setminus T : x^*_v > 0\}$. It is clear that $X$ is a multiway cut for $(G, T)$. Observe that $\sum_{v \in X} \omega_v \leq \sum_{v \in X} \omega_v \cdot 2x^*_v = 2 \sum_{v \in V \setminus T} \omega_v \cdot x^*_v$, as claimed. □

Henceforth, we prove the half-integrality of VERTEX MULTIWAY CUT.

Let $x^*$ and $f^*$ be optimal solutions to the primal and dual LP of VERTEX MULTIWAY CUT, and beware that $x^*$ and $f^*$ are not necessarily half-integral. As in the case of MINIMUM $(s,t)$-CUT and EDGE MULTIWAY CUT we interpret the assigned value $x^*_v$ on $v$ by the primal optimal fractional solution $x^*$ as the length (or cost) of the vertex $v$. For $v \in V$ and a terminal $s \in T$, the distance of $v$ from $t$ is

$$\texttt{dist}_{x^*}(t, v) := \begin{cases} 0 & \text{for } v = t \\ \min_{P \in \mathcal{P}_{t,v}} \sum_{u \in V(P) \setminus T} x^*_u & \text{for } v \in T \setminus t \end{cases}$$

where $\mathcal{P}_{t,v}$ is the collection of all $(s, v)$-path of $G$. Because $x^*$ is a feasible solution to the primal LP and $\mathcal{P}_{t,t'} \subseteq \mathcal{P}$ for every $t, t' \in T$, we have $\texttt{dist}_{x^*}(t, t') \geq 1$ for every $t, t' \in T$.

**Region, Boundary, Communal and private boundary.** For $i \in [k]$, let $T_i \subseteq V$ be the set $\{v \in V : \texttt{dist}_{x^*}(t_i, v) = 0\}$, called the region of terminal $t_i$, and $B_i := N_G(T_i)$, i.e. the vertices of $V \setminus T_i$ which has a neighbor in $T_i$. Let us call $B_i$ the boundary of the region $T_i$ and $B$ be the union of all boundaries, i.e. $\bigcup_{i=1} B_i$. Note that

$$x^*_v > 0 \qquad \forall v \in B \tag{15.1}$$

since otherwise (i.e. $x^*_v = 0$) $v$ would have been included in $T_i$. Moreover, all vertices of $T_i$ for any $i \in [k]$ is assigned 0 by $x^*_v$ whereas $x^*_v > 0$ for all vertices $v \in B_i$ for any $i \in [k]$, implying

$$B \cap T_i = \emptyset \qquad \text{for all } i \in [k].$$

A vertex $v$ of $B = \bigcup_{i=1}^{k} B_i$ falls into one of the two types:

- there are (at least) two indices $i, j \in [k]$ such that $v \in B_i \cap B_j$; let $B^{com}$ denote the set of these vertices.

- there is a unique index $i \in [k]$ such that $v \in B_i$; let $B^{prv}$ denote the set of such vertices.

**Lemma 15.6.1.** *$x^*_v = 1$ for every $v \in B^{com}$.*

**Complementary Slackness Condition.**

**Lemma 15.6.2.** *Let $P \in \mathcal{P}$ be a $(t_i, t_j)$-path with $f^*_P > 0$. Then either (i) $V(P) \cap B = \{v\}$ for some $v \in B^{com}$, or (ii) $V(P) \cap B = \{u, v\}$ for some $u \in B_i \cap B^{prv}$ and $v \in B_j \cap B^{prv}$.*

**Proof:** Note that Dual complementary slackness condition says:

$$\text{for every } P \in \mathcal{P} \text{ with } f^*_P > 0, \text{ we have } \textstyle\sum_{v \in V(P) \setminus T} x^*_v = 1.$$

Together with Lemma 15.6.1, we know that if such $P$ intersects with $B^{com}$, say at $v$, $P$ cannot contain any other vertex with positive value at $x^*$. Especially, $P$ does not contain any other vertex of $B$ other than $v$ due to Equation 15.1.

Suppose that $V(P) \cap B^{com} = \emptyset$. As $P$ connects two terminals $t^1$ and $t^2$ and $P$ can run between distinct terminals only via some boundary vertex of $B_1$ and $B_2$, we have $P$ traverses at least two vertices $u \in B_i \cap B^{prv}$ and $v \in B_j \cap B^{prv}$. Suppose that $P$ traverses another boundary vertex $z \in B_\ell \cap B^{prv}$ (possibly $\ell = i$ or $\ell = j$). As $i \neq j$, we may assume that $\ell$ is different from at least one of $i$ and $j$. Without loss of generality, assume $\ell \neq i$. Then the $(t_i, t_\ell)$-path $Q$ can be obtained from $P$ by taking the $(t_i, z)$-subpath of $P$ and appending it with the edge $zt_\ell$. On the other hand, $Q$ does not contain $v$ while $x^*_v > 0$, which implies

$$\sum_{w \in V(Q) \setminus T} x^*_w \leq \big( \sum_{w \in V(P) \setminus T} x^*_w \big) - x^*_v = 1 - x^*_v < 1$$

where the second equality holds due to the dual complementary slackness condition. This means that $x^*$ violates the primal constraint corresponding to $Q$, a contradiction. Therefore, we conclude that if $V(P) \cap B^{com} = \emptyset$, then the case (ii) in the statement holds. □

Now we define a half-integral solution $x'$ to LP of VERTEX MULTIWAY CUT and prove that $x'$ is an optimal LP solution.

$$x'_v = \begin{cases} 1 & \text{if } v \in B^{com} \\ \frac{1}{2} & \text{if } v \in B^{prv} \\ 0 & \text{otherwise} \end{cases}$$

It is trivial to verify that $x'$ is a feasible solution to the primal LP. Recall that due to Weak LP duality and the optimality of $f^*$, the following holds for $x'$ and $f^*$.

$$\sum_{P \in \mathcal{P}} f^*_P \leq \sum_{P \in \mathcal{P}} f^*_P \cdot \big( \sum_{v \in V(P) \setminus T} x'_v \big) = \sum_{v \in V \setminus T} x'_v \cdot \big( \sum_{\substack{P \in \mathcal{P}: \\ P \text{ traverses } v}} f^*_P \big) \leq \sum_{v \in V \setminus T} \omega_v \cdot x'_v,$$

and the Complementary Slackness Condition for primal and dual optimal solutions says that $x'$ is an optimal solution to the primal LP if and only if the following holds ($f^*$ is a dual optimal already).

**Primal complementary slackness** for every $v$ with $x'_v > 0$, we have $\sum_{\substack{P \in \mathcal{P}: \\ P \text{ traverses } v}} f^*_P = \omega_v$.

**Dual complementary slackness** for every $P \in \mathcal{P}$ with $f^*_P > 0$, we have $\sum_{v \in V(P) \setminus T} x'_v = 1$.

Due to the optimality of $x^*$, for every $u \in \{v \in V \setminus T : x^*_v > 0\}$ it holds that $\sum_{\substack{P \in \mathcal{P}: \\ P \text{ traverses } u}} f^*_P = \omega_u$. From $x^*_v > 0$ for every $v \in B$ (see Equation 15.1) and

$$\{v \in V \setminus T : x'_v > 0\} = B \subseteq \{v \in V \setminus T : x^*_v > 0\},$$

we know that the primal complementary slackness condition holds for $x'$ and $f^*$. The dual complementary slackness holds due to Lemma 15.6.2 and by the construction of $x'$.

# Chapter 16

# More on LP rounding

## 16.1 Derandomization and probability analysis 101

For the simple randomized rounding where we choose a single value $\theta$, there are only polynomially many points on which $\theta$ makes a difference in the rounding procedure. This approach might be readily available for some problems. In that case, the method of conditional expectation can be applied.

The following is Markov's inequality. For every $t \in \mathbb{R}^+$, it holds that

$$\texttt{prob}[X \geq t] \leq \frac{E[X]}{t}$$

for a non-negative random variable $X$.

## 16.2 Randomized rounding for WEIGHTED SET COVER

WEIGHTED SET COVER
**Instance:** a universe $U$, a collection $\mathcal{S}$ of subsets of $U$ and a weight function $\omega : \mathcal{S} \to Q^+$.
**Goal:** Find a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ of minimum total weight such that $\bigcup_{S \in \mathcal{S}'} S = U$.

## 16.3 MAXIMUM SATISFIABILITY

## 16.4 UNCAPACITATED FACILITY LOCATION

# Chapter 17

# Primal-Dual Method

## 17.1 WEIGHTED SET COVER revisited: a dual feasible solution as a certificate of your solution

Consider the problem WEIGHTED SET COVER. Suppose you're a director of an engineers' team and in charge of a new project, which is to construct a set of transmission tower which collectively covers a given set of points $U$. One can imagine that the set of points correspond to the ground set of the WEIGHTED SET COVER instance. You're given a set of candidate locations for constructing a tower and each candidate location for a transmission tower will cover certain set of points. You already noticed that each candidate, described by the set of points it covers, is a set $S \in \mathcal{S}$. Constructing a transmission tower at a candidate location (corresponding to 'choosing a set' $S \in \mathcal{S}$ in the set cover) comes with a cost $c_S$. Your job as a director of engineering is to choose a set of locations for constructing towers so that the transmission spans the entire points while minimizing the cost incurred at the company.

Your boss is not a mathematician, and he is impatient about the cost. You were somehow able to persuade the boss $O(\log n)$ times the optimal cost is the best one can ever dream to achieve. But without knowing the cost of an optimal solution, how can we persuade the boss that the solution you brought up with your team is actually $O(\log n) \cdot \texttt{opt}$? Mind that your boss is not a mathematician. So explaining the greedy algorithm below (from previous lecture) and showing the proof does not work. It only makes him even more irritated.

**Algorithm 21** Algorithm for WEIGHTED SET COVER

```
1: procedure SetCover(U, S, k)
2:     Mark every element of U as uncovered.
3:     S' ← ∅ and C ← ⋃_{S∈S'} S
4:     while U has an uncovered element do
5:         Select X ∈ S \ S' which minimizes c(X)/|X \ C|.
6:         price(e) ← c(X)/|X \ C| for every element e ∈ X \ C.
7:         S' ← S' ∪ {X}
8:         Mark every element of X covered; C ← C ∪ X.
9:     return S'.
```

We wrote in the previous lecture note:

Consider a sequence $X_1, \ldots, X_\ell$ of sets in $\mathcal{S}$ such that each set $X_i$ covers at least one new element, i.e. $\tilde{X}_i := X_i \setminus \bigcup_{j<i} X_j$ is non-empty. Let us define the price of an element $e$ covered in this sequence as

$$\mathsf{price}(e) := \frac{c(X_i)}{|\tilde{X}_i|}.$$

Recall that the total cost of selected sets by the algorithm equals the sum of prices over all elements.

In fact, the concept of price is extremely useful for convincing your boss that you did your job decently. Imagine the cost of a set $S$ is in fact paid to the land owner. The land owner will put a price on each point $u \in U$ so as to justify being paid $c_S$ for a set of points $S$. The land owner will try to maximize her profit while respecting the budget $c_S$ imposed by the company (if she does not respect the budget constraint $c_S$, her pricing policy will not be taken seriously). This leads to the canonical maximization problem, which is exactly the dual linear program for the linear programming relaxation of SET COVER.

$$\begin{aligned} & \max \sum_{e \in U} y_u & \\ \text{subject to} \quad & \sum_{e \in S} y_e \leq c_S & \forall S \in \mathcal{S} \\ & y_e \geq 0 & \forall e \in U. \end{aligned}$$

As a director, you can use the land owner's pricing policy (a dual feasible solution) as a justification that your proposed solution (a primal integral feasible solution, a set cover in this case) is a reasonable solution; the rationale is "Hey, look the landlady is not a fool and will do her best to maximizer her profit and we need to pay her at least that much. But the solution that our engineers' team proposed is not that far from her best attempt in pricing."

Returning to the greedy heuristic for SET COVER, we want to use the values on $U$ distributed by $\mathsf{price}(\mathsf{e})$. Unfortunately, price is not a feasible dual solution. However,

$$p(e) := \mathsf{price}(e)/H_n$$

is dual feasible. Here $n = |U|$ and $H_n$ is defined as $\sum_{i=1}^{n} 1/i$.

To see that $\{p(e)\}_{e \in U}$ is dual feasible, consider an arbitrary set $S \in \mathcal{S}$. Let $S_1, \ldots, S_\ell$ be the sets chosen by the greedy algorithm and let $W_i = \bigcup_{j \leq i} S_j$. ($W_0 = \emptyset$ by convention.) If an element $e$ is first covered by $S_i$, then

$$\mathsf{price}(e) = \frac{c_{S_i}}{|S_i \setminus W_{i-1}|}$$

and it holds that

$$\mathsf{price}(e) \leq \frac{c_S}{|S \setminus W_{i-1}|}$$

for any $S \in \mathcal{S}$ with $S \setminus W_{i-1} \neq \emptyset$ by the greedy selection of the sets $S_i$'s. Let $k'$ is the least integer such that $S \subseteq \bigcup_{i=1}^{k'}$. Then

$$
\begin{aligned}
\sum_{e \in S} \frac{\mathsf{price}(e)}{H_n} &= \frac{1}{H_n} \cdot \sum_{i=1}^{k'} \sum_{e \in S \cap S_i} \mathsf{price}(e) \\
&= \frac{1}{H_n} \cdot \sum_{i=1}^{k'} |S \cap S_i| \cdot \frac{c_{S_i}}{|S_i \setminus W_{i-1}|} \\
&\leq \frac{1}{H_n} \cdot \sum_{i=1}^{k'} |S \cap S_i| \cdot \frac{c_S}{|S \setminus W_{i-1}|} \\
&\leq \frac{c_S}{H_n} \cdot \Big( \underbrace{\frac{1}{|S|} + \cdots + \frac{1}{|S|}}_{|S \cap S_1| \text{ times}} + \underbrace{\frac{1}{|S \setminus S_1|} + \cdots + \frac{1}{|S \setminus S_1|}}_{|S \cap S_2| \text{ times}} + \cdots + \\
&\qquad\qquad \underbrace{\frac{1}{|S \setminus W_{k'-1}|} + \cdots + \frac{1}{|S \setminus W_{k'-1}|}}_{|S \cap S_{k'}| \text{ times}} \Big) \\
&\leq \frac{c_S}{H_n} \cdot \Big( \frac{1}{|S|} + \frac{1}{|S|-1} + \cdots \frac{1}{2} + \frac{1}{1} \Big) \\
&\leq c_S.
\end{aligned}
$$

Define $p_e := \mathsf{price}(e)/H_n$ and observe that $\{p_e\}_{e \in U}$ is dual feasible. Observe that $\sum_{i=1}^{k} c_{S_i} = \sum_{e \in U} \mathsf{price}(e) = H_n \cdot \sum_{e \in U} p_e \leq H_n \cdot c(\mathtt{opt})$, where $\mathtt{opt}$ is an optimal set cover and $c(\mathtt{opt})$ is the cost of an optimal set cover. Therefore, the produced solution $S_1, \ldots, S_k$ is within $H_n = O(\log n)$ factor of an optimal set cover.

We saw that a dual feasible solution can be used to bound (upper bound for minimization problem and lower bound) Primal-dual method is a way to use a dual feasible solution, which is improved over the course of the algorithm, as a guidance to find a primal feasible solution. The general idea is to maintain some primal and dual conditions while updating a dual feasible solution and a (not necessarily feasible) primal solution so that when we finally get a primal feasible solution, this is bounded by a good function of the dual solution.

## 17.2 2-approximation for VERTEX COVER via primal-dual

**Algorithm 22** Algorithm for VERTEX COVER

**procedure VertexCover**$(G, \omega)$
  $y_e \leftarrow 0$ for all $e \in E$.
  $X \leftarrow \emptyset$.
  ▷ $y$ is dual feasible, $x \leftarrow 0$ (as an indicator vector of $X$) is primal infeasible
  **while** $X$ is primal infeasible (i.e. not a vertex cover) **do**
    Select an arbitrary edge $e$ not covered by $X$.
    Increase $y_e$ to the maximum while preserving dual feasibility.
    Let $u$ be an endpoint of $e$ whose dual constraint is tight. (i.e. $\sum_{e \in \delta(v)} y_e = \omega_u$).
    $X \leftarrow X \cup \{u\}$.
  **return** $X$.

Note that while performing the procedure **VertexCover** the primal solution $x$ (as an indicator vector of $X$ at hand) and the dual solution at hand satisfy the following conditions.

1. **Dual feasibility:** $y$ is a dual feasible solution.
2. **Primal condition:** if $x_u > 0$, then $\sum_{e \in \delta(u)} y_e = \omega_e$.
3. **Dual condition:** if $y_e > 0$ for $e = uv$, then $x_u + x_v \leq 2$.

The dual feasibility condition holds because of line 7. The primal condition is maintained due to the choice of the vertex $u$ at line 8 and the dual condition trivially holds because each edge has exactly two endpoints.

Let $x'$ and $y'$ be the primal and dual solutions when we complete the while-loop, and note that both are feasible solutions. Now

$$\begin{aligned}
\sum_{v \in V} \omega_v \cdot x'_v &= \sum_{v \in V} x'_v \cdot \Big( \sum_{e \in \delta(v)} y'_e \Big) && \because \text{ primal condition} \\
&= \sum_{e=uv \in E} y'_e \cdot \big( x'_u + x'_v \big) && \because \text{ rearranging the terms} \\
&\leq \sum_{e=uv \in E} 2 \cdot y'_e && \because \text{ dual condition} \\
&\leq 2 \sum_{v \in V} \omega_v \cdot x^*_v && \because \text{ feasibility of } y' \text{ and weak duality of LP,}
\end{aligned}$$

where $x^*$ is a optimal fractional solution to the primal LP.

## 17.3 Algorithm for SHORTEST $(s,t)$-PATH via primal-dual

Let $\mathcal{S}$ be the collection of all vertex sets $S \subseteq V$ such that $s \in S$ and $t \notin S$. We express the problem SHORTEST $(s,t)$-PATH by the following integer program:

$$\begin{aligned}
\min & \sum_{e \in E} \omega_v \cdot x_v \\
& \sum_{e \in \delta(S)} x_e \geq 1 && \forall S \in \mathcal{S} \\
& x_e \in \{0,1\} && \forall e \in E
\end{aligned}$$

and the linear programming relaxation is obtained by replacing each constraint $x_e \in \{0,1\}$ by $x_e \geq 0$.

The dual LP is

$$\begin{aligned}
\max & \sum_{S \in \mathcal{S}} y_S \\
& \sum_{\substack{S \in \mathcal{S}: \\ e \in \delta(S)}} y_S \leq \omega_e && \forall e \in E \\
& y_S \geq 0 && \forall S \in \mathcal{S}.
\end{aligned}$$

One can easily verify the following observation, implying that the above integer program is indeed a formulation for SHORTEST $(s,t)$-PATH.

**Observation 17.3.1.** *Let $P \subseteq E$ and let $x^P \in \{0,1\}^E$ be an indicator vector of $P$, that is,*

$$x_e^P = \begin{cases} 1 & \text{if } e \in P \\ 0 & \text{otherwise.} \end{cases}$$

*Then $(V,P)$ contains an $(s,t)$-path if and only if $x^P$ is a feasible solution to the above integer program.*

**Algorithm 23** Algorithm for SHORTEST $(s,t)$-PATH

```
1:  procedure ShortestPath(G, ω)
2:      if s and t are in distinct connected components of G then return primal infeasible.
3:      x_e ← 0 for all e ∈ E.
4:      F ← ∅.
5:                                                          ▷ y is dual feasible.
6:          ▷ We grow F by adding edges one by one till there is an (s,t)-path in the subgraph (V,F).
7:      while there is no (s,t)-path in (V,F) do
8:          Let C be the connected component of (V,F) containing s.
9:          Increase y_C to the maximum while preserving dual feasibility.
10:         Let e be an edge of G whose dual constraint is tight. (i.e. Σ_{S∈S:e∈δ(S)} y_S = ω_e).
11:         F ← F ∪ {e}.
12:     return the unique (s,t)-path in (V,F).
```

By $G[F]$ for $\emptyset \neq F \subseteq E$, we denote the subgraph of $G$ on the vertex set

$$V(F) := \{u \in V : u \text{ is an endpoint of some edge of } F\}$$

with $F$ as the edge set. When $F = \emptyset$, we define $G[F]$ as the graph on the singleton $\{s\}$ (with no edges).

**Observation 17.3.2.** *Throughout the execution of the procedure* **ShortestPath***, the graph $G[F]$ is a tree containing $s$.*

**Proof:** We prove by induction on $|F|$. When $F = \emptyset$, the claim trivially holds. If $G[F]$ is a tree containing $s$ at the beginning of $i$-th execution of the while-loop, the set $C$ chosen at line 8 coincides with $V(F)$ by induction hypothesis. Note that whichever edge $e$ is chose at line 10, $e$ belongs to $\delta(S)$ because changing the value of $y_S$ only affects the dual constraint corresponding to an edge incident with $S$. Therefore we maintain the property that $G[F+e]$ is a tree containing $s$. □

Let us examine the primal and dual conditions that the procedure **ShortestPath** maintains. Consider $C \subseteq V$ found at line 8 and $v \in C$. Let $P_v$ be the $(s,v)$-path in $(V,F)$ for $v \in C$, which is unique due to Observation 17.3.2, and let $x^{P_v} \in \{0,1\}^E$ be the indicator vector of $P_v$. $y$ is the current dual solution. Then the following is valid for each $v \in C$.

1. **Dual feasibility:** $y$ is a dual feasible solution.
2. **Primal condition:** $x_e^{P_v} > 0$ implies $\sum_{S \in \mathcal{S}} y_S = \omega_e$.

3. **Dual condition:** $y_S > 0$ implies $\sum_{e \in \delta(S)} x_e^{P_v} = 1$.

The dual feasibility condition holds because of how $y$ is constructed at line 9. The primal condition is derived from that $P_v \subseteq F$ and how the edges of $F$ are chosen at line 10.

To see why the dual condition holds, suppose that there is $S \in \mathcal{S}$ with $y_S > 0$ and $\sum_{e \in \delta(S)} x_e^{P_v} \neq 1$. Then we have $\sum_{e \in \delta(S)} x_e^{P_v} \geq 2$; indeed, $y_S > 0$ implies that we chose $S$ at line 8 at some point and increasing $y_S$ made some edge $e$ tight at line 10. And such an edge $e$ must belong to $\delta(S)$ because otherwise the value of the left-hand side does not change. Therefore, $\sum_{e \in \delta(S)} x_e^{P_v} \geq 1$ and in particular $\sum_{e \in \delta(S)} x_e^{P_v} \geq 2$ by our assumption.

Note that $|\delta(S) \cap P_v| = \sum_{e \in \delta(S)} x_e^{P_v} \geq 2$ and let $e_1, e_2$ be two edges in $\delta(S) \cap P_v$. Without loss of generality, we assume that $e_2$ was added to $F$ later than $e_1$. Let $F_2$ be the set named as "$F$" at the iteration when $e_2$ was added and note that $e_1 \in F_2 \subseteq F$. On the other hand, $P_v$ contains a subpath containing $e_1$ and $e_2$ which takes at least one vertex of $V \setminus F_2$. This means that $F_2 \subseteq P_v$ contains a cycle while both $F_2$ and $P_v$ are subsets of $F$. This contradicts Observation 17.3.2, which says that $F$ is a tree. This completes the proof that the dual condition holds.

**Lemma 17.3.1.** *For each $v \in C$, the $(s, v)$-path $P_v$ of $F$ is a shortest $(s, v)$-path of $G$.*

**Proof:** (Proof sketch) Consider a linear program for the SHORTEST $(s, v)$-PATH problem and corresponding dual LP. It is an easy exercise to verify that $x^{P_v}$ and a simple modification of $y$ (how?) are primal and dual feasible solutions satisfying the primal and dual condition. By Complementary Slackness Condition, $x^{P_v}$ is an optimal primal feasible solution. In particular, $P_v$ is a shortest $(s, v)$-path of $G$. □

Finally, when the procedure terminates, the returned $(s, t)$-path at line 12 is a shortest $(s, t)$-path by Lemma 17.3.1.

## 17.4 STEINER FOREST PROBLEM: increasing multiple dual variables in synchronization

STEINER FOREST PROBLEM
**Instance:** a graph $G = (V, E)$ with an edge weight $\omega : E \to Q^+$, a set of $k$ terminal pairs $\{(s_i, t_i) : i \in [k]\}$.
**Goal:** Find an edge set $F' \subseteq E$ minimum weight so that in the subgraph $(V, F')$ of $G$, every terminal pair $(s_i, t_i)$ is connected.

STEINER FOREST PROBLEM is a generalization of both the SHORTEST $(s, t)$-PATH problem and the STEINER TREE (why?). In this section, we present a 2-approximation for STEINER FOREST PROBLEM using primal-dual method.

We call a vertex set $W \subseteq V$ active if $|W \cap \{s_i, t_i\}| = 1$ for some $i \in [k]$. The collection of all active sets is denoted by $\mathcal{S}$. It is not difficult to see that an edge set $F \subseteq E$ forms a steiner forest if and only if $(V, F)$ has no active connected component. From this, we obtain the following integer program for STEINER FOREST PROBLEM.

$$\begin{aligned} \min & \sum_{e \in E} \omega_e \cdot x_e & \\ \text{subject to} & \sum_{e \in \delta(S)} x_e \geq 1 & \forall S \subseteq \mathcal{S} \\ & x_e \in \{0,1\} & \forall e \in E \end{aligned}$$

One can replace the constraint $x_e \in \{0,1\}$ by the non-negativity constraint and the integral constraint. Therefore, a linear programming relaxation can be obtained by replacing each constraint $x_e \in \{0,1\}$ by $x_e \geq 0$. As the minimization

The dual LP is

$$\begin{aligned} \max & \sum_{S \in \mathcal{S}} y_S & \\ & \sum_{\substack{S \in \mathcal{S}: \\ e \in \delta(S)}} y_S \leq \omega_e & \forall e \in E \\ & y_S \geq 0 & \forall S \in \mathcal{S}. \end{aligned}$$

Let us consider a naive primal-dual algorithm for STEINER FOREST PROBLEM depicted in Algorithm 24.

**Algorithm 24** (Unsuccessful) Algorithm for STEINER FOREST PROBLEM

1: **procedure FirstAttempt**$(G, \omega, \{(s_i, t_i, ) : i \in [k]\})$
2: $y_S \leftarrow 0$ for all $S \in \mathcal{S}$.
3: $F \leftarrow \emptyset$.
4: **while** there is an active connected component $C$ in $(V, F)$ **do**
5: Increase $y_C$ till some edge $e$ becomes tight.
6: Append $F$ by $e$ at the end.
7: **Output** $F$.

In Algorithm 24, we maintain the dual feasibility and the primal condition, i.e. at any point during the execution of the algorithm, it holds $\sum_{S \in \mathcal{S}} y_S = \omega_e$ for every $e \in F$. However, the dual condition does not hold for any constant $c$. One can create an instance for which, when the algorithm terminates, there is a set $S \in \mathcal{S}$ such that $y_S > 0$ and $\sum_{e \in \delta(S)} x_e^F = |\delta(S) \cap F|$ is arbitrarily large.

We resolve this issue with two ideas. First, in the dual variable update phase, we increase multiple dual variables simultaneously. In this way, we can bound the average value of $|\delta(S) \cap F|$ for the active connected components at the beginning of this phase. Secondly, we discard unnecessary edges from $F$ after finding a set connecting all $(s_t, t_i)$ pairs. This backward deletion step turns out to be crucial for meeting a desired dual condition.

Consider the set $F$ and $y$ after performing line 9 at an arbitrary point during the execution of Procedure **SteinerForest**. It is easy to verify the following properties.

**Algorithm 25** Algorithm for STEINER FOREST PROBLEM

1: **procedure SteinerForest**$(G, \omega, \{(s_i, t_i, ) : i \in [k]\})$
2: $y_S \leftarrow 0$ for all $S \in \mathcal{S}$.
3: $F \leftarrow \emptyset$.
4: ▷ $y$ is dual feasible. We set these values only implicitly.
5: ▷ We grow $F$ until there is an $(s_i, t_i)$-path in the subgraph $(V, F)$ for every $i$.
6: **while** there is an active connected component in $(V, F)$ **do**
7: Let $\mathcal{C}$ be the collection of all active connected components in $(V, F)$.
8: For all $C \in \mathcal{C}$, increase $y_C$ **simutaneously** till some edge $e$ becomes tight.
9: Append $F$ by $e$ at the end.
10: Let $e_1, e_2, \ldots, e_\ell$ be the edges of $F$ in the order of insertion.
11: **for** $i = \ell$ down to 1 **do**
12: **if** there is no active connected component in $(V, F - e_i)$ **then** discard $e_i$ from $F$.
13: **Output** $F$.

1. **Dual feasibility:** $y$ is a dual feasible solution.
2. **Primal condition:** $x_e^F > 0$ (i.e. $e \in F$) implies $\sum_{S \in \mathcal{S}: e \in \delta(s)} y_S = \omega_e$.

Let $F'$ be the output set at line 13 and $y'$ be the final dual feasible solution at the end of the procedure. The key property for getting approximation ratio 2 for Procedure **SteinerForest** is the following.

**Lemma 17.4.1.** *Let $\mathcal{C}_i$ be the collection of active connected components identified at line 7 at $i$-th iteration of the while-loop. Then*

$$\frac{1}{|\mathcal{C}_i|} \sum_{C \in \mathcal{C}_i} |F' \cap \delta(C)| \leq 2.$$

Before proving Lemma 17.4.1, we want to show this statement together with the dual feasibility of $y$ and primal condition above establishes the claimed approximation ratio.

Let $\ell$ be the index of the last iteration of the while-loop. As the dual variables for the members of $\mathcal{C}_i$ get increased simultaneous, we can associate a single value, denoted as $\epsilon_i$, such that each $y_C$ for $C \in \mathcal{C}$ increases by $\epsilon_i$ during the $i$-th iteration of the while-loop. Moreover, the value $y'_S$ is the sum of all values $\epsilon_i$ over $i \in [\ell]$ such that the dual variable $y_S$ increased during the $i$-th iteration, or equivalently $S \in \mathcal{C}_i$. It readily follows

that $\sum_{S\in\mathcal{S}} y'_S = \sum_{i=1}^{\ell} \epsilon_i \cdot |\mathcal{C}_i|$. Therefore,

$$\begin{aligned}
\sum_{e\in E} \omega_e \cdot x_e^{F'} &= \sum_{e\in E} x_e^{F'} \cdot \big( \sum_{S\in\mathcal{S}: e\in\delta(S)} y'_S \big) && \because \text{primal condition} \\
&= \sum_{S\in\mathcal{S}} y'_S \cdot \big( \sum_{e\in\delta(S)} x_e^{F'} \big) \\
&= \sum_{S\in\mathcal{S}} y'_S \cdot |F' \cap \delta(S)| \\
&= \sum_{S\in\mathcal{S}} |F' \cap \delta(S)| \cdot \big( \sum_{i=1}^{\ell} \epsilon_i \cdot [S \in \mathcal{C}_i] \big) && \because y'_S = \sum_{i=1}^{\ell} \epsilon_i \cdot [S \in \mathcal{C}_i] \\
&= \sum_{i=1}^{\ell} \sum_{S\in\mathcal{S}} \epsilon_i \cdot [S \in \mathcal{C}_i] \cdot |F' \cap \delta(S)| \\
&= \sum_{i=1}^{\ell} \sum_{C\in\mathcal{C}_i} \epsilon_i \cdot |F' \cap \delta(C)| && \because \text{the term disappears for } S \notin \mathcal{C}_i \\
&= \sum_{i=1}^{\ell} \epsilon_i \cdot \big( \sum_{C\in\mathcal{C}_i} |F' \cap \delta(C)| \big) \\
&= 2 \cdot \sum_{i=1}^{\ell} \epsilon_i \cdot |\mathcal{C}_i| && \because \text{Lemma 17.4.1} \\
&= 2 \cdot \sum_{S\in\mathcal{S}} y'_S && \because \sum_{S\in\mathcal{S}} y'_S = \sum_{i=1}^{\ell} \epsilon_i \cdot |\mathcal{C}_i|
\end{aligned}$$

and especially, the objective value is bounded by $2 \cdot \omega(\texttt{opt})$.

The next observation is useful for establishing Lemma 17.4.1

**Observation 17.4.1.** *Let $T$ be a forest and let $R \subseteq V(T)$ be a set of tree nodes containing all leaves. Then $\sum_{v\in R} deg(v) \leq 2 \cdot |R|$.*

**Proof:** The claimed inequality is easily achieved by the following.

$$\begin{aligned}
\sum_{v\in R} deg(v) &= \sum_{v\in V(T)} deg(v) - \sum_{v\in V(T)\setminus R} deg(v) \\
&\leq 2 \cdot (|V(T)| - 1) - \sum_{v\in V(T)\setminus R} deg(v) \\
&\leq 2 \cdot (|V(T)| - 1) - 2 \cdot |V(T) \setminus R| && \because deg(v) \geq 2 \text{ for every non-leaf } v \\
&\leq 2 \cdot (|R| - 1)
\end{aligned}$$

□

**Proof of Lemma 17.4.1:** Let $F_i$ is the set of edges obtained at at line 9 at the $i$-th iteration of the while-loop for $i \geq 1$ and let $F_0 = \emptyset$.

Let $\bar{\mathcal{C}}_i$ be the set of the $i$-th iteration of the while-loop. Let $\bar{\mathcal{C}}_i$ be the set of connected components of $(V, F_i)$ which are not active. As $F'$ is a steiner forest, one can add $F' \setminus F_i$ to $F$ and obtain a subgraph $(V, F' \cup F_i)$ in which every terminal pair is connected.

To begin with, we argue that $(V, F_i)$ is a forest. We can see this by induction on the number of iterations. The base case $i = 0$ is trivial. At $i + 1$-st iteration, the only edges $f$ for which the value $\sum_{S\mathcal{S}:f\in\delta(S)} y_S$ changes (increases) by the synchronized dual update are the ones which have exactly one endpoint in some active connected component. As we add a (unique) tight edge $e$ to $F_i$, and such an edge $e$ can have at most one endpoint in an active component of $(V, F_i)$, the resulting edge set $F_{i+1} = F_i \cup e$ again forms a forest. This also implies that $(V, F_i \cup F')$ is a forest as $F_i \cup F'$ is a subset of $F_\ell$.

Let $H'_i$ be the forest obtained from $H_i$ by contracting contracting the edges of $F_i$. Notice that each connected component of $(V, F_i)$, i.e. each member of $\mathcal{C}_i \cup \bar{\mathcal{C}}_i$, becomes a single vertex and thus there is one-to-one correspondence between the vertices of $H'_i$ and $\mathcal{C}_i \cup \bar{\mathcal{C}}_i$. We abuse the notation and refer to the vertices of $H'_i$ by the corresponding connected components of $(V, F_i)$.

The key observation is that any leaf of $H'_i$ corresponds to an active connected component of $\mathcal{C}_i$. Suppose there is a leaf of $H'_i$ which corresponds to an inactive connected component. Then one can delete the (unique) edge of $F'$ connecting this leaf to its parent and the resulting edge set is a steiner forest, contradicting the assumption that $F'$ is the output set after the backward deletion step.

By Observation 17.4.1, we have

$$\sum_{C\in\mathcal{C}_i} |\delta(C) \cap F'| = \sum_{C\in\mathcal{C}_i} |\delta(C) \cap (F' \setminus F_i)| = \sum_{v\in\mathcal{C}_i} deg_{H'_i}(v) \leq 2 \cdot |\mathcal{C}_i|,$$

as claimed. □

## 17.5 EDGE MULTICUT ON TREES: solving the dual problem simultaneously

We present a 2-approximation algorithm using primal-dual method. The algorithm updates the primal solution so that it becomes feasible as well as the dual solution.

Let $P_i$ be the unique $(s_i, t_i)$-path on $T$ for $i \in [k]$. Recall that the linear programming relaxation for EDGE MULTICUT ON TREES and its dual are formulated as:

$$\begin{aligned} & \min \sum_{e\in E} \omega_e \cdot x_e & \\ \text{subject to} \quad & \sum_{e\in P_i} x_e \geq 1 & \forall i \in [k] \\ & x_e \geq 0 & \forall e \in E \end{aligned}$$

and

$$\begin{aligned} & \min \sum_{i \in [k]} f_i & \\ \text{subject to} \quad & \sum_{i \in [k]: e \in P_i} f_i \leq \omega_e & \forall e \in E \\ & f_i \geq 0 & \forall i \in [k]. \end{aligned}$$

**Algorithm 26** Algorithm for EDGE MULTICUT ON TREES

1: **procedure MulticutOnTrees**($T, \omega, \{(s_i, t_i) : t \in [k]\}$)
2: **if** some terminal pair $(s_i, t_i)$ are in distinct connected components of $T$ **then return** NO.
3: $f_i \leftarrow 0$ for all $i \in [k]$
4: Mark every vertex as unprocessed.
5: Mark every terminal pair as active.
6: $D \leftarrow \emptyset$.
7: **while** there is unprocessed vertex in $T$ **do**
8: Choose a lowermost unprocessed vertex $v$.
9: **while** there is an active pair $(s_j, t_j)$ whose least common ancestor is $v$ **do**.
10: Increase $f_j$ until some edge $e$ becomes tight, i.e. $\sum_{i \in [k]: e \in P_i} f_i = \omega_e$.
11: Append all tight edges to $D$.
12: Mark each terminal pairs inactive if the corresponding path on $T$ contains an edge of $D$.
13: Mark $v$ as processed.
▷ Now we want to delete some redundant edges from $D$ by scanning it backwardly
14: Let $e_1, \ldots, e_m$ be the edges in $D$ in the order of insertion.
15: **for all** $j = m$ down to 1 **do**
16: **if** $D - e_j$ is a multicut **then** discard $e_j$ from $D$.
17: **return** $D$ and the current flow $f$.

Let $D$ be the edge set found at the end of the first while-loop, i.e. the set of edges enumerated at line 14. and we denote by $D'$ the edge set returned at line 17. Note that the primal condition is maintained during the execution of the procedure due to update of $y$ and $D$ at lines 10-11.

(Primal condition) If $e \in D$, then the corresponding dual constraint is tight, i.e. $\sum_{i \in [k]: e \in P_i} f_i = \omega_e$.

The key observation is that after the backward deletion, a dual condition also holds.

(Dual condition) If $f_i > 0$, then $|P_i \cap D'| \leq 2$

**Lemma 17.5.1.** *If $f_i > 0$ for some $i \in [k]$, then $|P_i \cap D'| \leq 2$.*

**Proof:** We prove a slightly stronger statement:

> Let $v_i$ be the lowermost common ancestor of $(s_i, t_i)$. If $f_i > 0$, then the $(s_i, v_i)$-subpath and the $(t_i, v_i)$-subpath of $P_i$ have at most one edge of $D'$ respectively.

Suppose not, and without loss of generality let $(s_i, v_i)$-subpath of $P_i$ has two edges $e, e'$ Assume that $e'$ is deeper than $e$ on $T$.

If $e' \prec e$ in $D$ (inserted earlier to $D$ than $e$), then the pair $(s_i, t_i)$ is inactive when $v_i$ is processed and $f_i = 0$, a contradiction to the assumption that $f_i > 0$. Therefore, we have $e \prec e'$. The fact that $e'$ is not discarded during the backward deletion step when $e'$ was tested implies that there is a pair, say $(s_j, t_j)$, such that $e'$ is the only edge of $D$ intersecting $P_j$ at that moment.

Let $e''$ be an edge on $P_j$ which was added to $D$ when or before $v_j$ was processed and note that $e'' \prec e \prec e'$ because $v_j$ is processed before $v_i$ in the first while-loop. This, however, means that when $e'$ was tested during the backward deletion phase, both $e''$ and $e'$ are in $D$, a contradiction to the choice of $(s_j, t_j)$ as a pair which enforces $e'$ to survive with the backward deletion phase. □

To complete the analysis of the Procedure **MulticutOnTrees**, observe

$$\begin{aligned}
\sum_{e \in E} \omega_e \cdot x_e^{D'} &= \sum_{e \in E} x_e^{D'} \cdot \Big( \sum_{i \in [k] : e \in P_i} f_i \Big) \\
&= \sum_{i \in [k]} f_i \cdot \Big( \sum_{e \in P_i} x_e^{D'} \Big) \\
&= \sum_{i \in [k]} f_i \cdot |D' \cap P_i| \\
&\leq 2 \cdot \sum_{i \in [k]} f_i \\
&\leq 2 \cdot \omega(\mathtt{opt}),
\end{aligned}$$

that is, $D'$ is a 2-approximate solution to EDGE MULTICUT ON TREES. We also observe that $\sum_{i \in [k]} f_i \geq 0.5 \cdot \sum_{e \in E} \omega_e \cdot x_e^{D'} \geq 0.5 \cdot$ optimal flow value, and is a 0.5-approximate solution to the dual.

We remark that the dual as a fractional flow is polynomial-time solvable by solving the LP. But the restriction of maximum flow, where the edge weight is integral and we are asked to output integral flow called INTEGER MULTICOMMODITY FLOW ON TREES, is NP-hard on trees. The above analysis shows that for INTEGER MULTICOMMODITY FLOW ON TREES admits a 2-approximation algorithm which computes a 2-approximate multicut and 0.5-approximate integer flow simultaneously.